Sarawak Tun Openg University of Technology (SOTECH)

# IT PROJECT 2 REPORT

**CHAPTERS 1–6**

# Web-Based Preorder and Freshness Management System for Home-Based Entrepreneurs

**Christopher Hansen**

Student ID: 2402081406

Supervisor

Ms. Nur Elana Binti Mohd Saferinor

2026

# ABSTRACT

Home-based food businesses need an accessible way to coordinate preorders, dated ingredients, customer choices and handover. This project implements a single-vendor web application for Wrap & Bowl Kuch, a home-based business in Kuching, Sarawak. React provides customer and staff interfaces, Django enforces the application rules and PostgreSQL stores accepted orders, stock and evidence. Client feedback refined the system to a name/phone passkey account, morning delivery or collection, area-based delivery fees and a simpler actual-receipt workflow without required cooking-task planning.

Customers can browse a branded eight-menu catalogue, use optional explainable recommendations and a whole-basket date/quantity/budget finder, review their basket and submit QR/bank-transfer evidence or delivery cash. Placement captures accepted names, prices, recipes and packaging, then allocates eligible ingredients and packaging transactionally. Owners maintain documented shelf-life and storage evidence, review private payment proof separately from food receipt, print daily/order summaries and inspect sales, cost and contribution information. FreshCast remains separate decision support with explicitly identified local and external data boundaries.

Fresh verification at source 6edde5e ran 1,046 backend tests: 970 passed and 76 were skipped, comprising PostgreSQL-dependent and optional real-model checks. Thirty-seven local release/runtime checks and selected frontend suites, lint and build also passed. Retained browser evidence supports selected responsive, account, recovery and stock behaviours. These are developer verification results, not actual bank transactions, food-safety measurements or a completed user study. Informal client feedback informed development; editable client acceptance and student usability instruments are supplied for the formal evaluation still to be collected.

# TABLE OF CONTENTS

CHAPTER 1 ..... 1
1.1 INTRODUCTION ..... 1
1.2 PROBLEM STATEMENTS ..... 2
1.3 OBJECTIVES ..... 3
1.4 SCOPE ..... 3
1.5 REVENUE MODEL ..... 4
1.6 METHODOLOGY ..... 5
1.7 GANTT CHART ..... 5
1.8 SOFTWARE TOOLS ..... 5
1.9 CONCLUSION ..... 6
CHAPTER 2 ..... 7
2.1 INTRODUCTION ..... 7
2.2 BACKGROUND STUDY ..... 8
2.3 METHODOLOGY ..... 14
2.4 TECHNOLOGY ..... 16
2.5 IMPLEMENTATION IMPLICATIONS ..... 17
CHAPTER 3 ..... 19
3.1 INTRODUCTION ..... 19
3.2 ITERATIVE DEVELOPMENT AND REQUIREMENT REFINEMENT ..... 19
3.3 USE CASE DIAGRAMS ..... 20
3.4 USE CASE SPECIFICATIONS ..... 22
3.5 ACTIVITY DIAGRAMS ..... 30
3.6 SEQUENCE DIAGRAMS ..... 34
3.7 CLASS DIAGRAM ..... 39
3.8 ENTITY–RELATIONSHIP DIAGRAMS ..... 39
3.9 REQUIREMENT TRACEABILITY AND DESIGN LIMITS ..... 47
3.10 CONCLUSION ..... 47
CHAPTER 4 ..... 48
4.1 INTRODUCTION ..... 48
4.2 APPLICATION ARCHITECTURE AND TECHNOLOGY ..... 48
4.3 CLIENT REQUIREMENTS AND DEVELOPMENT ITERATIONS ..... 50
4.4 CUSTOMER INTERFACES ..... 51
4.5 OWNER INTERFACES ..... 62
4.6 CORE CALCULATIONS AND TRANSACTION RULES ..... 73
4.7 FRESHCAST AND SUPPORTING DECISION TOOLS ..... 75
4.8 REMINDERS, SECURITY AND DEPLOYMENT ..... 76
4.9 INTERFACE AND ACCESSIBILITY DECISIONS ..... 77

4.10 CONCLUSION....78
CHAPTER 5....79
5.1 INTRODUCTION....79
5.2 TESTING STRATEGY AND ENVIRONMENT....79
5.3 DECISION TABLES AND BOUNDARIES....81
5.4 EXECUTED FUNCTIONAL AND SECURITY CASES....83
5.5 CURRENT REGRESSION RESULTS....87
5.6 BROWSER FINDINGS, DEFECTS AND RETESTING....87
5.7 FORECAST EVALUATION AND DATA LIMITS....89
5.8 EVALUATION AGAINST THE UNCHANGED OBJECTIVES....90
5.9 CLIENT AND CUSTOMER EVALUATION PLAN....90
5.10 DISCUSSION IN RELATION TO THE LITERATURE....92
5.11 CONCLUSION....92
CHAPTER 6....94
6.1 INTRODUCTION....94
6.2 ACHIEVEMENT OF THE ORIGINAL OBJECTIVES....94
6.3 DEVELOPMENT CHALLENGES AND RESPONSES....96
6.4 STRENGTHS OF THE IMPLEMENTED SYSTEM....98
6.5 LIMITATIONS....98
6.6 PRIORITISED FUTURE WORK....99
6.7 HANDOVER AND MAINTENANCE....100
6.8 CONCLUSION....100
REFERENCES....102
APPENDIX A — SUPERVISOR CONSULTATION RECORD....106
APPENDIX B — ORIGINALITY / SIMILARITY RECORD....106
APPENDIX C — DECLARATION OF INTEGRITY....107
APPENDIX D — CURRENT APPLICATION DATA DICTIONARY....108
APPENDIX E — CLIENT FOLLOW-UP AND ACCEPTANCE FORM....125
APPENDIX F — STUDENT CUSTOMER-USABILITY QUESTIONNAIRE....128
APPENDIX G — IMPLEMENTATION AND TEST EVIDENCE MAP....131
APPENDIX H — SHORT OWNER OPERATING GUIDE....135

# LIST OF FIGURES

Figure 1.1: Original IT Project 1 proposal Gantt chart (retained; not an actual completion log)..... 5
Figure 2.1: Original literature map retained from IT Project 1.....7
Figure 3.1: Customer use cases for the current storefront and account workflow.....21
Figure 3.2: Owner/staff use cases for stock, orders, reporting and configuration.....22
Figure 3.3: Checkout validation and atomic ingredient/packaging allocation.....31
Figure 3.4: Batch date evidence, validation and stock eligibility.....32
Figure 3.5: Packaging receipts, menu mappings and placement deductions.....33
Figure 3.6: Separate payment review and actual customer receipt.....34
Figure 3.7: Customer passkey registration.....35
Figure 3.8: Authentication using a saved customer passkey.....35
Figure 3.9: Authoritative pricing, scoped evidence and transactional placement.....36
Figure 3.10: Private evidence review and manual payment recording.....37
Figure 3.11: Daily/sales reporting and read-only order-slip preview.....38
Figure 3.12: Authenticated, bounded owner handover reminder processing.....39
Figure 3.13: Selected core domain classes and their associations.....41
Figure 3.14: Identity, customer profile and passkey records.....42
Figure 3.15: Catalogue, recipe, batch and inventory-log relationships.....43
Figure 3.16: Accepted order terms, private proof and ingredient allocation.....44
Figure 3.17: Current and accepted packaging requirements and stock movements.....45
Figure 3.18: Payment, amendment, expense and wastage evidence.....46
Figure 4.1: Current customer/staff application architecture and external boundaries.....49
Figure 4.2: Founder-story page using the client-supplied business introduction (local developer capture)..... 52
Figure 4.3: Client menu, category filters and food cards with illustrative images.....53
Figure 4.4: Ordinary phone-viewport menu capture with compact cards and basket access.....54
Figure 4.5: Customer name/phone and passkey account dialog (ordinary phone capture). .....55
Figure 4.6: Separate kitchen staff sign-in and explicit customer-account return control. .....56
Figure 4.7: Date/quantity/budget finder and its one-menu versus mixed choice.....57
Figure 4.8: Basket quantities and optional item details without duplicate delivery forms. .....58
Figure 4.9: Checkout offered-day guidance, upcoming date shortcuts and minute-based time selection.....59
Figure 4.10: Checkout receipt-evidence state and enabled placement controls (synthetic local proof)..... 61

Figure 4.11: Customer order confirmation with separate food-receipt and payment states..... 62
Figure 4.12: Inventory overview with ingredient filters, recorded status and batch actions..... 63
Figure 4.13: Batch evidence form with conditional date inputs and a planning-deadline preview..... 64
Figure 4.14: Aligned stock-target controls and totals across the offered menus..... 65
Figure 4.15: Packaging stock balance and actions for current menu supplies..... 66
Figure 4.16: Packaging-assignment dialog with per-selling-unit quantities and aligned controls..... 66
Figure 4.17: Owner order queue with receipt groups, filters and deliberate per-order actions..... 67
Figure 4.18: Payment-review dialog showing proof, amount and payment history..... 68
Figure 4.19: Daily order/menu-unit summary for kitchen work..... 69
Figure 4.20: Order-slip print preview; an A4 preview is intentionally different from a web-page viewport..... 70
Figure 4.21: Sales overview with date filtering and separate food/delivery values..... 71
Figure 4.22: Read-only price/bulk-offer scenario and progressively disclosed cost records..... 72
Figure 4.23: Section-based kitchen settings with explicit save scope..... 73
Figure 4.24: Compact public recorded-stock counts with an optional date-based average explanation..... 74
Figure 4.25: Owner email configuration/status and optional execution details..... 76

# LIST OF TABLES

Table 2.1: Summary of findings on Malaysian online food delivery consumer behaviour....8
Table 2.2: Research areas, focus and identified gaps retained from the proposal....11
Table 2.3: Proposal-stage feature comparison; provider features/regions may change.. 13
Table 2.4: Agile and traditional comparison, adapted from the original proposal summary (Islam & Ferworn, 2020)....14
Table 2.5: Comparison of development dimensions, adapted from the proposal's Leau et al. (2012) summary....15
Table 3.1: Client-led design changes and their consequences....20
Table 4.1: Technology responsibilities in the current implementation....50
Table 4.2: Representative development problems and implemented responses....51
Table 4.3: Illustrative date calculation using explicit evidence....74
Table 5.1: Verification environment and interpretation....80
Table 5.2: Account eligibility and authentication decisions....81
Table 5.3: Current handover and payment decisions....81
Table 5.4: Stock allocation and history decisions....82
Table 5.5: Date evidence and recommendation boundaries....82
Table 5.6: Executed functional cases TC-01–TC-05....83
Table 5.7: Executed functional cases TC-06–TC-10....84
Table 5.8: Executed functional cases TC-11–TC-15....84
Table 5.9: Executed functional cases TC-16–TC-20....85
Table 5.10: Executed functional cases TC-21–TC-25....86
Table 5.11: Executed functional cases TC-26–TC-26....86
Table 5.12: Fresh verification summary at source 6edde5e....87
Table 5.13: Representative defects, corrections and observed retests....88
Table 5.14: Objective evidence and remaining validation....90
Table 6.1: Limitations and their practical implications....98

# LIST OF ABBREVIATIONS

| Abbreviation | Meaning |
|---|---|
| API | Application programming interface |
| CSRF | Cross-site request forgery |
| DRF | Django REST Framework |
| ERD | Entity–relationship diagram |
| FEFO | First-expiry, first-out |
| FK / PK | Foreign key / primary key |
| FYP | Final-year project |
| MAE | Mean absolute error |
| MYR / RM | Malaysian ringgit |
| ORM | Object-relational mapper |
| RPC | Remote procedure call |
| SMTP | Simple Mail Transfer Protocol |
| UAT | User acceptance testing |
| UML | Unified Modeling Language |
| WAPE | Weighted absolute percentage error |
| WCAG | Web Content Accessibility Guidelines |

# CHAPTER 1
# INTRODUCTION

## 1.1 INTRODUCTION

The home-based food industry experienced significant growth during and after the COVID-19 pandemic (Pallianysamy et al., 2025). According to Statista (2024), the online food delivery market in Malaysia is forecasted to reach USD 2.77 billion in 2024 and is projected to grow to USD 4.63 billion by 2029.

However, existing food ordering platforms may not adequately address the specific needs of home-based food entrepreneurs, particularly in supporting structured pre-order operations. Some local home-based food sellers might rely on WhatsApp for ordering — either through its structured cart feature or informal chat — or use large food delivery platforms such as GrabFood and FoodPanda. While the big platforms offer scheduled ordering, the proposal-stage comparison recorded advance booking windows of approximately 1–2 days, which may not align with the needs of home-based sellers who prepare food in batches and require confirmed orders further in advance. WhatsApp, meanwhile, has no proper mechanism for date-based preordering in either mode: in its cart feature, any date must be manually written into the product title or description, requiring the seller to read and verify each date themselves rather than having the system check availability or capacity automatically; chat-based arrangements, similarly, rely entirely on the seller manually tracking dates and capacity.

Neither WhatsApp nor large food ordering platforms such as GrabFood and FoodPanda offer any feature for monitoring ingredient freshness, leaving sellers without a structured way to track raw food expiry. This is particularly concerning given that storing large quantities of perishable goods increases the risk of microbial growth (Pallianysamy et al., 2025), a risk further worsened by Malaysia's indoor temperature range of 21°C to 37°C (Gou et al., 2018), which accelerates food spoilage. Real-life food poisoning incidents, such as the illness of 65 primary school students after consuming flat noodle soup in May 2024 ("Sixty-five Sungai Petani primary school pupils suffer food poisoning," 2024) and the deaths caused by spoiled fried noodles and eggs in Selangor, highlight the seriousness of this problem ("Food poisoning turns fatal," 2024).

In addition, GrabFood and FoodPanda place multiple competing sellers within the same marketplace environment, limiting opportunities for sellers to establish distinctive branding and direct customer relationships. WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence. Furthermore, the basic WhatsApp ordering workflow reviewed here does not provide the integrated analytics and menu-recommendation functions proposed for this system. Sellers who need to adopt a different platform for preordering, freshness monitoring, or a dedicated storefront would also benefit from integrated analytics and recommendation features, rather than relying on separate or absent tools. The absence of recommendation mechanisms may contribute to customer decision fatigue, which could negatively influence purchasing decisions and potentially increase the likelihood of purchase abandonment, while limited access to sales analytics restricts sellers' ability to monitor performance and make informed business decisions.

To address these issues, this proposal introduces a web-based food pre-ordering system that allows home-based food sellers to manage personalised storefronts and strengthen their branding efforts. The system also includes raw ingredient freshness monitoring based on storage duration, sales analytics to support better business decisions, and a menu recommendation system to assist customers in selecting suitable menu options and enhance the overall purchasing experience.

In conclusion, the proposed web-based system aims to assist home-based food sellers in managing pre-order operations more efficiently through a dedicated platform tailored to their business needs. The system further incorporates ingredient freshness monitoring to support food safety practices, while providing personalized storefront management, sales analytics, and menu recommendation features to enhance sellers' operational effectiveness and customer experience.

## 1.2 PROBLEM STATEMENTS

1. Lack of reliable advance preordering: The proposal-stage comparison recorded approximately 1–2-day advance windows for GrabFood and FoodPanda, which may not align with the needs of sellers preparing food in batches further ahead. WhatsApp's cart feature has no structured date field, requiring sellers to manually write and verify dates themselves rather than having the system check availability automatically. Chat-based ordering has no structure at all, leaving sellers to track every order manually.

2. Lack of ingredient freshness monitoring: The consumer ordering interfaces compared in this proposal do not provide the proposed kitchen-level ingredient shelf-life and storage records, limiting sellers' ability to proactively manage food safety.

3. Lack of dedicated single-vendor storefront: GrabFood and FoodPanda place competing sellers in the same multivendor environment, limiting branding opportunities. WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence.

4. Limited business and decision support tailored to small operators: The basic WhatsApp ordering workflow reviewed here does not provide these integrated analytics or menu-recommendation functions. Sellers who need to adopt a different platform for preordering, freshness monitoring, or a dedicated storefront would also benefit from integrated analytics and recommendation features, rather than relying on separate or absent tools.

## 1.3 OBJECTIVES

1. To design and develop a web application that supports flexible date-based food preordering and scheduling for home-based food entrepreneurs.

2. To implement a food stock freshness monitoring feature based on ingredient shelf life and storage duration.

3. To develop a personalised single-vendor storefront module that allows sellers to strengthen branding without direct competitor distraction.

4. To implement a sales analytics dashboard that supports data-driven business decision-making for home-based food operators.

5. To develop a menu recommendation system that reduces customer decision fatigue and improves purchase conversion rates.

## 1.4 SCOPE

### 1.4.1 Customer Scope

- Customer can view the seller's storefront and product listings with category filters
- Customer can select available preorder dates when placing orders
- Customer can view food recommendations and add items to cart

### 1.4.2 Food Entrepreneur Scope

- Seller can log in as admin and manage product listings
- Seller can manage preorder availability settings
- Seller can monitor ingredient freshness status based on remaining shelf life duration
- Seller can view sales analytics to support business decisions
- Seller can customise their storefront including store name, story, contact information and menu photographs

### 1.4.3 System Scope

- The system checks future preorder dates and kitchen availability, and distinguishes confirmed bookings from explicit owner-review requests
- The system tracks and displays ingredient freshness status based on remaining shelf life
- The system shows recommended and all product listings filterable by category
- The system displays relevant features based on user role — entrepreneur or customer

### 1.4.4 IMPLEMENTED SCOPE CLARIFICATION

The implemented case study is Wrap & Bowl Kuch, Kuching. Client feedback simplified ordering to a name/phone passkey account, permitted delivery or collection dates and morning handover, area-based fees, private QR/bank-transfer evidence or delivery cash, immediate ingredient/packaging allocation, and actual receipt recording. The client no longer needs cooking-task Planner confirmation in the operating workflow. Original proposal scope is retained above as historical context; Chapters 3–6 describe these refinements. The five objectives remain unchanged.

## 1.5 REVENUE MODEL

The proposed solution is designed to help home-based food entrepreneurs manage preordering operations through a structured and fixed-cost platform with record-based ingredient-date monitoring and personalised branding. The proposed commercial model has two revenue streams; automated seller subscription billing is outside the implemented system:

1. Upfront payment A one-time fee is charged for initial system access and setup.

2. Recurring fee Monthly maintenance and hosting fees are charged to sustain system operation.

## 1.6 METHODOLOGY

This project follows an Agile methodology due to its suitability for small-scale iterative development. Development is organised into iterative cycles where each cycle involves requirement gathering, design, implementation and testing of specific features. The process begins with identifying home-based food entrepreneurs' needs and customer pain points. Each feature — including product preordering, ingredient freshness monitoring, storefront management, sales analytics and menu recommendation — is then designed, implemented and tested iteratively. The system is refined based on feedback and testing outcomes throughout development.

## 1.7 GANTT CHART

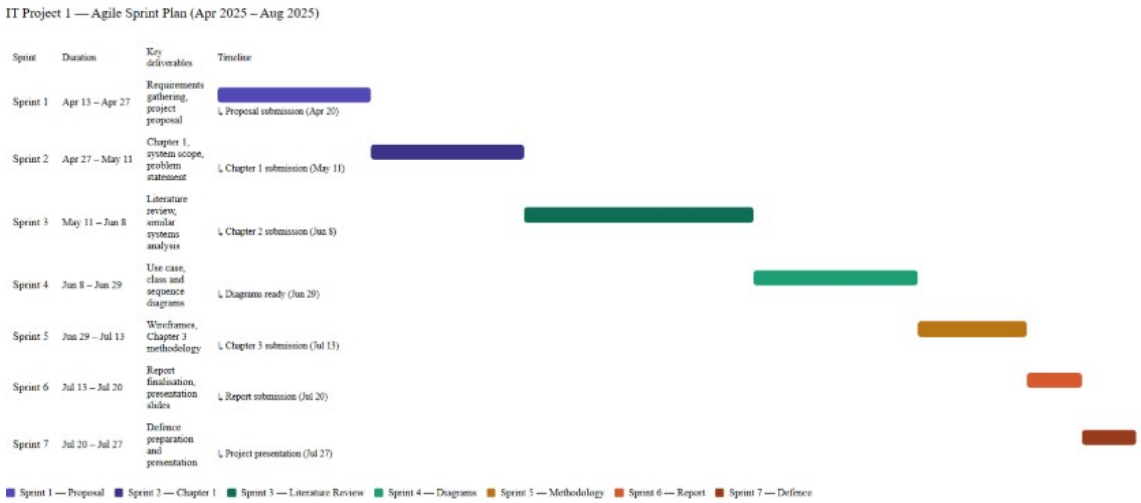



**Figure 1.1: Original IT Project 1 proposal Gantt chart (retained; not an actual completion log).**

## 1.8 SOFTWARE TOOLS

1. React — Frontend framework for building interactive user interfaces

2. Django — Backend framework supporting rapid development with built-in ORM and MVT architecture

3. PostgreSQL — Relational database suited for structured interconnected data entities

4. Visual Studio Code — Code editor with extensive Python and JavaScript extension support

5. GitHub — Version control platform for tracking code changes and managing development history

## 1.9 CONCLUSION

The proposed system aims to help home-based food entrepreneurs manage preorder operations more efficiently through a dedicated fixed-cost web platform tailored to their business needs.

By providing flexible date-based preordering, ingredient freshness monitoring, personalised storefront management, sales analytics and menu recommendation features, the system addresses key operational gaps faced by home-based food entrepreneurs. Operating through a dedicated platform allows sellers to maintain their own branded storefront and build direct customer relationships independent of large multivendor marketplace environments.

The project follows an Agile methodology to support iterative feature development and flexibility throughout the development process. Therefore, the proposed system supports the growth of local home-based food SMEs while contributing to food safety awareness and improved customer purchasing experience.

# CHAPTER 2

# LITERATURE REVIEW

## 2.1 INTRODUCTION

This chapter consists of three parts including background study, methodology, and technology.

The background study discusses the evolution of the food ordering industry, digital transformation in Malaysia's food and beverage (F&B) sector, and the research interest landscape related to preorder and freshness management systems. In addition, similar existing systems and current technological trends relevant to the proposed system will also be reviewed and discussed.

The methodology section discusses the comparison between Agile and Traditional software development approaches to determine the most suitable methodology for the proposed system.

Meanwhile, the technology section describes the framework and database technologies considered for the project. A comparison between Django and Express.js will be carried out to determine the most suitable framework for the proposed system. Moreover, a comparison between Firebase and PostgreSQL will also be conducted to identify the most suitable database solution for the proposed system.

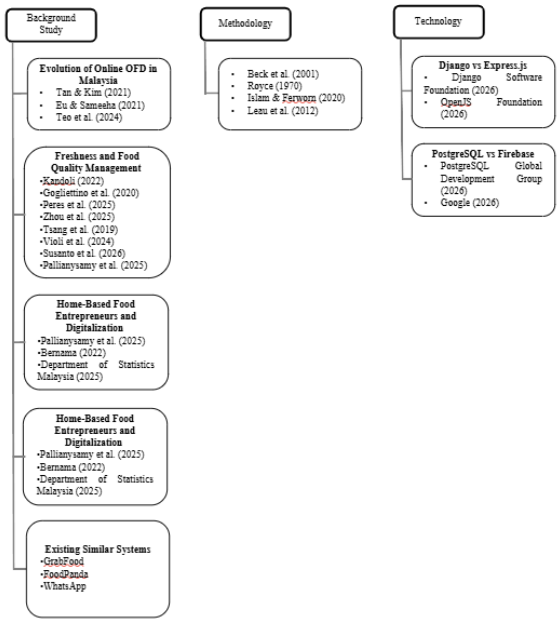



**Figure 2.1: Original literature map retained from IT Project 1.**

## 2.2 BACKGROUND STUDY

### 2.2.1 Evolution of Online Food Ordering Industry in Malaysia

The Online Food Ordering industry boomed in Malaysia in 2017, driven by smartphone adoption and internet accessibility (Tan & Kim, 2021). The COVID-19 pandemic further accelerated this growth as consumers shifted toward online food ordering amid the shutdown of non-essential services and dining restrictions (Eu & Sameeha, 2021). Within this growing market, research has identified food quality and health considerations as significant factors influencing Malaysian consumer behavior in OFD platforms (Teo et al., 2024).

Therefore, understanding consumer expectations regarding food quality and freshness within Malaysia's digital food ecosystem is important in identifying the relevance of the proposed web-based preorder and freshness management system. Table 2.1 presents an analysis of key findings and their implications for the Malaysian OFD landscape.

**Table 2.1: Summary of findings on Malaysian online food delivery consumer behaviour.**

| APA citation | Key finding | Relevance |
|---|---|---|
| Tan & Kim (2021) | Food quality, service approach and perceived price were linked to satisfaction. | Supports attention to food quality and service. |
| Eu & Sameeha (2021) | 76.9% perceived limited healthy options; 85.9% expressed willingness to purchase healthy food. | Highlights a gap in perceived choice/quality. |
| Teo et al. (2024) | Health, quality, service and value influence continuance intention. | Supports integrating quality and service information. |

Across these studies, food quality and healthy food availability consistently emerge as critical factors influencing Malaysian consumers' OFD usage and satisfaction. However, existing platforms are perceived as inadequately addressing these concerns. Given that freshness is identified as a core component of food quality (Tan & Kim, 2021), this suggests a gap for more practical freshness management features within digital food ordering systems.

### 2.2.2 Freshness and Food Quality Management in Digital Food Systems

Food storage is one method used to preserve food for future consumption (Kandoli, 2022). However, certain perishable food products with high nutrient content

remain vulnerable to microbial contamination and pathogen growth if storage conditions are not properly managed (Gogliettino et al., 2020). The presence and growth of these microorganisms influence the freshness status and quality of raw food, thereby requiring proper freshness management before food is served to consumers.

Food quality management has become a critical aspect of the food industry, particularly in ensuring food safety and freshness throughout the supply chain. Recent studies have explored the integration of Industry 4.0 technologies such as sensor systems, blockchain, and artificial intelligence to improve food monitoring, process automation, and traceability mechanisms (Peres et al., 2025). Furthermore, TTIs and FIs were explored in food shelf-life monitoring (Zhou et al., 2025).

However, these technologies are generally implemented within large-scale food supply chains and industrial environments where structured operational processes, dedicated quality control divisions, and larger financial resources are available. Such implementations commonly emphasize end-to-end supply chain monitoring rather than operational-level freshness management for small-scale food services or home-based food businesses.

Another study explored the integration of IoT, fuzzy logic, and zero-order quality degradation models into an Internet of Things-based Shelf Life Management System (Tsang et al., 2019). Nevertheless, the implementation complexity and infrastructure requirements of such systems may reduce their suitability for home-based food microbusinesses.

Similarly, Violi et al. (2024) proposed a mathematical inventory-routing model for perishable products. The model assists warehouses in determining delivery priorities to minimize spoilage while addressing stochastic demand using historical sales data and Conditional Value-at- Risk (CVaR) risk analysis. However, the computational complexity and integration requirements of such approaches may limit their applicability in small-scale academic projects and home-based food business operations. Furthermore, these systems may require IoT infrastructure to monitor physical inventory movement, potentially increasing implementation costs.

Susanto et al. (2026) also proposed a Quality Management System framework for food SMEs consisting of Quality Planning, Quality Assurance, and Quality Control and Improvement stages. However, the implementation scope of such systems may exceed the operational requirements of small-scale home-based food businesses.

Moreover, Pallianysamy et al. (2025) highlighted the need for monitoring and inspection solutions for home-based food operations from a food safety perspective. Nevertheless, implementing advanced monitoring technologies such as cameras, sensors, or IoT devices may introduce additional infrastructure and maintenance costs that are less suitable for lightweight small-scale systems.

Therefore, a research gap exists in the development of lightweight and cost-effective freshness management approaches that can be integrated into smaller-scale digital food systems such as online preorder and home-based food ordering platforms.

### 2.2.3 Home-Based Food Entrepreneurs and Digitalization

Home-based food businesses in Malaysia represent a growing segment particularly following COVID-19 (Pallianysamy et al., 2025). This growth coincides with significant expansion in Malaysia's digital food market — ecommerce transactions increased by 21.8 percent from 2020 to RM1.1 trillion in 2021 (Bernama, 2022), with revenue for the first nine months of 2025 reaching RM937.5 billion (Department of Statistics Malaysia, 2025).

However, as identified in Section 2.2.5, existing platforms present several limitations for home-based food operators. GrabFood and FoodPanda were recorded in the proposal comparison as offering approximately 1–2-day advance windows, which is insufficient for sellers who prepare food in batches and require confirmed orders further ahead of time. WhatsApp's Cart feature has no structured date field, requiring sellers to manually write and verify dates themselves rather than having the system check availability automatically, while informal chat-based ordering has no structure at all, making it difficult for sellers to reliably track orders and avoid missing them amid casual conversation. None of these platforms provide ingredient freshness monitoring features.

Furthermore, existing platforms lack dedicated personalised storefronts — GrabFood and FoodPanda offer limited personalisation within competitive multivendor environments, while WhatsApp offers either a fixed, generic Catalog template with no customizable layout, or informal product sharing via chat — neither gives sellers a distinct, independently branded storefront presence. While dedicated single-vendor ordering systems exist for large established food brands such as McDonald's, these systems are developed for large-scale commercial operations and are not accessible to informal home-based food micro-entrepreneurs as ready-to-use platforms.

Home-based entrepreneurs whose operational needs — flexible preordering, freshness monitoring, or a dedicated single-vendor storefront — are not met by GrabFood, FoodPanda, or WhatsApp would therefore need to adopt a different platform altogether; once on such a platform, they would also benefit from integrated sales analytics and recommendation features rather than having to rely on separate or absent tools. While GrabFood and FoodPanda do provide sales analytics through vendor dashboards (Table 2.3), and WhatsApp offers none at all, neither pathway currently combines these analytics and recommendation capabilities with the preorder, freshness, and storefront features that the proposed system addresses together as a single integrated solution.

### 2.2.4 Research Interest & Academic Relevance Landscape

Tan & Kim (2021) examined consumer satisfaction in Malaysian OFD, focusing on factors such as food quality and pricing. However, this study focuses on consumer satisfaction rather than operational tools for food businesses.

In addition, research on healthy food awareness in OFD platforms indicates that Malaysian consumers value healthier food options and food quality visibility (Eu & Sameeha, 2021). However, these studies mainly focus on healthy food availability perception and food choice behavior, rather than real-time freshness monitoring or freshness-status tracking within online food ordering systems.

Moreover, recent studies exploring digital technologies in food monitoring suggest increasing adoption of Artificial Intelligence (AI), Internet of Things (IoT), and smart sensors for food tracking and waste reduction (Zhou et al., 2025). However, these solutions are often complex and costly, making them less suitable for small-scale food businesses. Therefore, there is a need for a more lightweight and cost-effective approach to freshness management, such as a system based on shelf-life tracking and inventory entry time. Table 2.2 presents an analysis of existing research areas and their identified gaps relevant to the proposed system.

**Table 2.2: Research areas, focus and identified gaps retained from the proposal.**

| Research area | Study focus | Gap relevant to this project | Citation |
|---|---|---|---|
| Consumer satisfaction | Quality, price and service | Operational preorder/freshness tools | Tan & Kim (2021) |
| Healthy food availability | Perceptions of variety and price | Practical ingredient-date visibility | Eu & Sameeha (2021) |

| Research area | Study focus | Gap relevant to this project | Citation |
|---|---|---|---|
| Digital food monitoring | AI/IoT and larger supply chains | Lightweight tools for small operators | Zhou et al. (2025) |

Overall, the existing literature tends to focus either on consumer behavior, healthy food availability perception, or advanced technological solutions, with limited attention given to affordable, practical freshness management systems tailored for home-based food entrepreneurs. This identifies a clear research gap that the proposed web-based preorder and freshness management system aims to address.

### 2.2.5 Existing Similar Systems

The following platform descriptions and Table 2.3 preserve the proposal-stage comparison. Ordering windows and commercial features can vary by region, account and date; they were not revalidated as current provider promises for this implementation review. The comparison is not a universal claim that no other software supports these functions.

This section compares existing systems to identify features currently available and limitations that the proposed system aims to address.

#### *2.2.5.1 GrabFood*

GrabFood is the most popular food ordering platform among university students (Lok et al., 2024). It was founded in 2018 which provided online food delivery solutions in Southeast Asia (Grab, 2018). The platform primarily focuses on registered restaurant vendors and large-scale food businesses and was recorded in the proposal comparison as allowing advance preordering up to two days. Moreover it is a multivendor platform, therefore, they provide less freedom in personalized content. However, publicly available platform features indicate limited support for freshness-status monitoring and inventory- oriented analytics.

#### *2.2.5.2 FoodPanda*

FoodPanda was launched in 2012 before expanding across Southeast Asia (Foodpanda, 2012). It allows advance preordering up to 1 day. Same with GrabFood, the platform primarily focuses on registered restaurant vendors and large-scale food businesses. It allows limited preordering and does not provide personalized storefront. It also does not offer food freshness monitoring features, and provides limited sales

analytics through its vendor dashboard. Recommendation capabilities are relatively limited and primarily focused on vendor visibility rather than personalized freshness-oriented recommendations.

#### *2.2.5.3 WhatsApp*

WhatsApp may be used as an informal preorder communication platform among small- scale food businesses and microentrepreneurs in Southeast Asia. However, the preorder process is largely unstructured and dependent on manual chat management, which may increase the risk of missed orders and communication inefficiencies. In addition, WhatsApp does not provide dedicated features such as freshness monitoring, analytics dashboards, or personalized preorder management.

This section reviews several existing platforms related to online food ordering and preorder management to identify their functionalities, limitations, and relevance to the proposed system. The comparison focuses on preorder flexibility, personalization, freshness monitoring, sales analytics, and recommendation capabilities. Through this comparison, the limitations of current systems can be identified to justify the need for the proposed system.

**Table 2.3: Proposal-stage feature comparison; provider features/regions may change.**

| Feature | GrabFood | foodpanda | WhatsApp | McDonald's | Proposed system |
|---|---|---|---|---|---|
| Advance ordering | Up to 2 days | Up to 1 day | Manual date in conversation | Limited scheduling | Flexible seller rules |
| Single-vendor storefront | Marketplace | Marketplace | Basic catalogue/chat | Brand-owned | Dedicated kitchen |
| Ingredient-date monitoring | Not in compared interface | Not in compared interface | Manual records | Not in compared interface | Shelf-life records |
| Sales analytics | Vendor analytics | Vendor analytics | Not in basic ordering flow | Not evaluated | Integrated reports |
| Recommendations | Platform-based | Category-based | Not in compared flow | Not evaluated | Explainable ranking |

Note. McDonald's is included as a reference example of an existing single-vendor dedicated ordering system. It is not a practical alternative for home-based food entrepreneurs, as such systems are developed for large-scale commercial operations and are not accessible as ready-to-use platforms for informal micro-entrepreneurs.

## 2.3 METHODOLOGY

Agile methodology is an iterative software development approach where developers can always revisit earlier stages of the Software Development Life Cycle after feedback. According to Beck et al. (2001), the Agile Manifesto emphasizes four core values: individuals and interactions over processes and tools, working software over comprehensive documentation, customer collaboration over contract negotiation, and responding to change over following a plan. Individuals and interactions over processes and tools means the methodology capitalizes more on human resources rather than advanced technologies or tools. Working software over comprehensive documentation means the methodology makes use of early prototypes for testing purposes rather than planning everything from start through documentation. Customer collaboration over contract negotiation means the method prefers continuous customer feedback rather than a fixed negotiation contract at the beginning. Responding to change over following a plan means the methodology prioritises adapting to new information and feedback rather than rigidly adhering to an initial plan.

A traditional model is a software development process where each stage is rigidly planned from the beginning. According to Royce (1970) it consists of system requirements, software requirements, analysis, program design, coding, testing, operation. He, however, said that the model is so simple that it's very risky for larger and more costly software projects.

Agile methodology is an incremental software development process while traditional methodology is a more formal or plan-driven software development approach (Islam & Ferworn, 2020). The comparisons between the two are described in the table below.

**Table 2.4: Agile and traditional comparison, adapted from the original proposal summary (Islam & Ferworn, 2020).**

| Dimension | Agile (proposal summary) | Traditional (proposal summary) |
|---|---|---|
| Approach | Adaptive | Predictive |
| Success emphasis | Business value | Conformance to plan |
| Typical project context | Small / evolving | Larger / stable |
| Management emphasis | Decentralised | Centralised |
| Change | Adaptability | Controlled changes |

| Dimension | Agile (proposal summary) | Traditional (proposal summary) |
|---|---|---|
| Culture | Collaboration | Formal coordination |
| Documentation | Selected supporting records | Extensive records |
| Focus | People and iteration | Process and sequence |
| Cycles | Numerous | Limited |
| Domain | Exploratory | Predictable |
| Team | Small / adaptable | Larger / structured |
| Upfront planning | Incremental | Comprehensive |
| Return | Incremental delivery | Later completion |

This table summarises the proposal's comparison rather than establishing universal rules about documentation, team size or engineering skill. The present project uses iterative development with written requirements, diagrams and automated tests in every relevant increment.

Agile methodology is selected for this project due to its suitability for small-scale projects, support for iterative feature development, and flexibility to accommodate requirement changes throughout the development process. These characteristics align with the nature of this academic project, where features may be refined or extended based on feedback and testing outcomes. Leau et al. (2012) provide a comparative analysis between Agile and traditional methodologies regarding rework costs, testing protocols, and project scalability, as outlined in Table 2.5.

**Table 2.5: Comparison of development dimensions, adapted from the proposal's Leau et al. (2012) summary.**

| Dimension | Agile | Traditional |
|---|---|---|
| Requirements | Iteratively refined | Defined before implementation |
| Rework | Earlier feedback can reduce later rework | Late changes can be expensive |
| Direction | Changeable within iterations | More fixed baseline |
| Testing | Each iteration | Historically later stages |
| Customer involvement | Frequent | Often concentrated at milestones |
| Developer skills | Interpersonal and business knowledge | Technical and coordination skills still required |

| Dimension | Agile | Traditional |
|---|---|---|
| Project scale in proposal | Low to medium | Larger plan-driven context |

This table summarises the proposal's comparison rather than establishing universal rules about documentation, team size or engineering skill. The present project uses iterative development with written requirements, diagrams and automated tests in every relevant increment.

## 2.4 TECHNOLOGY

### 2.4.1 Django vs Express.js

According to the Django Software Foundation (2026), Django is a high-level Python web framework designed to support the rapid development of backend code. The framework is widely adopted for its Model-View-Template (MVT) architecture, which separates the application's core data, logic, and user interface. Within this architecture, a Model is defined as a Python class object that maps directly to a database table, providing an object-relational interface to database queries (Django Software Foundation, 2026). The View serves as a function or class that accepts web requests and returns the corresponding web responses. Finally, the Django architecture incorporates Templates, which consist of HTML code enhanced with template tags to dynamically display the data retrieved from the database to the end user (Django Software Foundation, 2026).

According to the OpenJS Foundation (2026), Express.js is a minimalist web framework designed to provide a robust set of features for web and application development. The framework relies primarily on two core functionalities: routing and middleware. Routing defines how application endpoints or URIs respond to specific incoming client requests, while middleware functionality allows developers to access and manipulate the request (req) and response (res) objects during the application's cycle (OpenJS Foundation, 2026).

Django was selected over Express.js due to prior development experience with the framework, which reduces learning overhead in a time-constrained academic project. Additionally, Django's MVT architecture enforces a clear project structure that supports code maintainability and organisation throughout development.

### 2.4.2 PostgreSQL vs Firebase

According to PostgreSQL Global Development Group (2026), PostgreSQL is an open- source object-relational database that has been developed for more than 35 years, making it a reputable database. It has many features that help developers in application building, administrators that protect data integrity, and support datasets of varying sizes. Furthermore, many features that are supported by SQL standard are also supported by PostgreSQL making it compliant with the SQL standard.

The Google (2026) comparison concerns Cloud Firestore, Firebase's document-oriented database, rather than every service under the Firebase platform. Meanwhile, it stores data in documents where each document contains key-value pairs. A document object is a type of JSON object that supports extra data types and has document size limits. Collection is the container that holds instances of the same data structure.

PostgreSQL is selected as its relational data model better suits structured data with interconnected entities, improving readability and maintainability compared to Firebase's document-based storage which may result in deeply nested data structures that are harder to navigate and maintain. Furthermore, Firebase's NoSQL structure is not natively supported by Django's ORM, which would require bypassing Django's built-in database management tools and increasing implementation complexity.

## 2.5 IMPLEMENTATION IMPLICATIONS

The literature rationale remains the same: a lightweight integrated tool is relevant to a home kitchen whose workflow is not represented by a short-notice marketplace checkout. The current React/Django implementation uses recorded date evidence, traceable ingredient and packaging allocations, a single-vendor storefront and explainable menu suggestions. Client feedback removed required cooking-task planning from the active workflow. These changes do not alter the proposal literature or the five objectives. Physical food safety, behavioural improvement and actual commercial outcomes require separate evidence, as discussed in Chapters 4–6.

# CHAPTER 3
# METHODOLOGY AND SYSTEM DESIGN

## 3.1 INTRODUCTION

This chapter retains the iterative development approach established in IT Project 1 and updates the design where the client's operating requirements changed. The five objectives in Section 1.3 are unchanged. The implemented case study is Wrap & Bowl Kuch, Ayu Shafinaz's home-based business in Kuching, Sarawak. Its catalogue comprises salad bowls, Wanpaku sandwiches and a Monday rice bowl. The client information used here consists of the menu poster, founder introduction and feature requests supplied by the project author. These materials are requirements evidence; they are not a transcribed interview or a completed acceptance study.

The principal refinement is a simpler operating workflow. Scheduling means choosing a permitted future delivery or collection date and time, rather than requiring the owner to define cooking tasks or confirm a preparation plan. Customer access uses a name/phone profile with a passkey. The owner retains separate staff authentication. Ingredient and packaging allocations occur when an order is accepted. Payment verification and actual food receipt remain distinct events. Earlier scheduling and payment components remain in the codebase for compatibility, but they are not the active client checkout design.

## 3.2 ITERATIVE DEVELOPMENT AND REQUIREMENT REFINEMENT

Development followed small requirement, implementation, verification and feedback cycles. The original problem, literature rationale and objectives provided the baseline. Client feedback subsequently prioritised name-and-phone contact, packaging stock, payment-proof upload, area fees, daily order summaries, printing and a received/not-received presentation. Additional iterations addressed phone normalisation, accepted-order snapshots, stock shortages, passkey recovery limitations and consistent interfaces.

The project used iterations rather than claiming formal team Scrum ceremonies that did not take place. A working feature was reviewed through its visible interaction and its server-side rules. Regression checks were rerun when a change could affect existing orders, stock or permissions. The author's client meetings informed these revisions, but no attendance count, signed approval or interview answer is inferred from

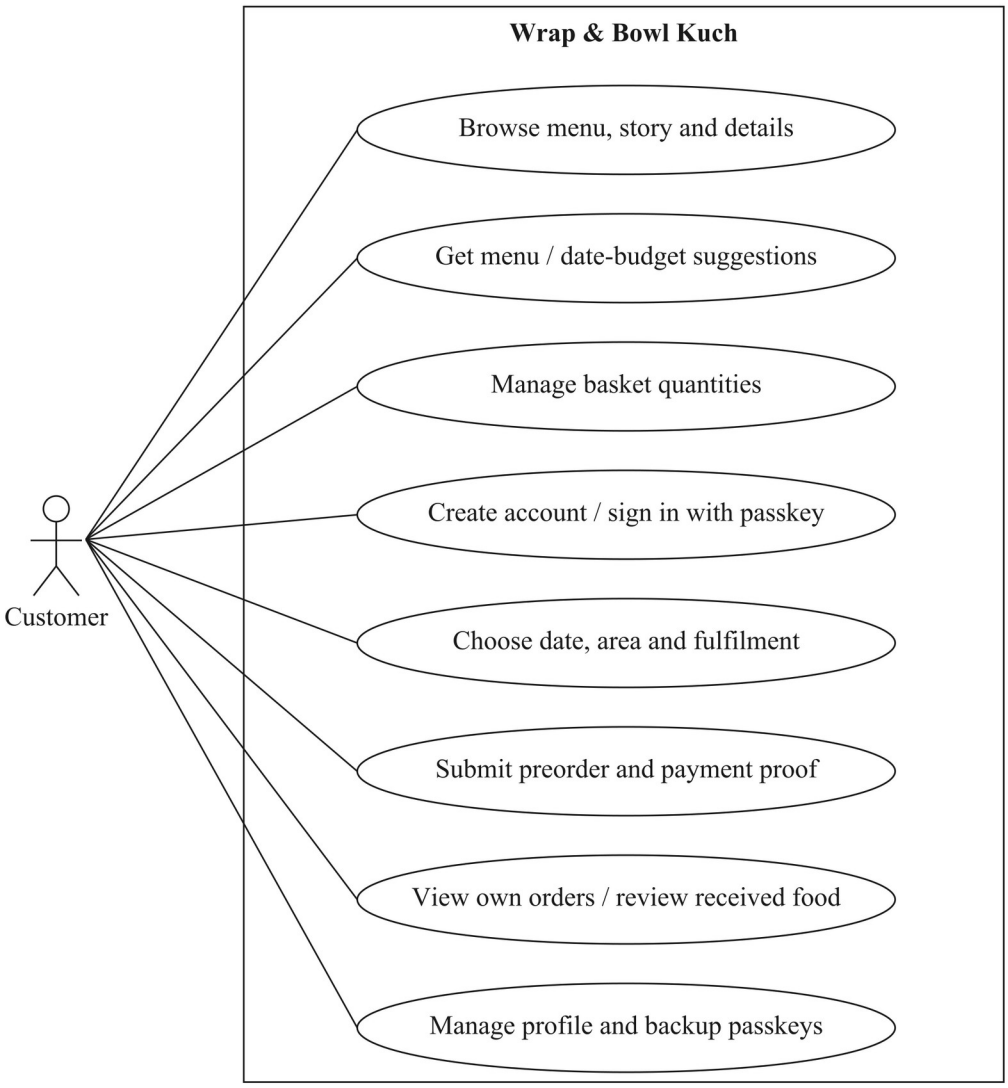
the code history. Chapter 4 explains the implementation decisions and Chapter 5 separates developer verification from the evaluation still to be collected.

**Table 3.1: Client-led design changes and their consequences**

| Requirement change | Current design consequence | Preserved constraint |
|---|---|---|
| Reduce registration effort | Name and phone profile with native passkey registration/sign-in | A phone number is contact data, not proof of identity |
| No cooking Planner in daily use | Future handover date/time, offered days and cutoffs; no cooking reservation | A date must still be permitted and within the supported horizon |
| Track packaging as well as ingredients | Packaging quantities, menu assignments and per-order allocations | A shortage must prevent the whole placement |
| Upload payment evidence | Private sanitised receipt image associated with the quoted selection | An upload never means the bank payment has been verified |
| Morning deliveries and area fees | Editable minute-based hours and delivery-area price mapping | The server calculates the accepted fee |
| Simplify order handling | Received / Not received, with cancellation and audited corrections | Food receipt does not record money received |

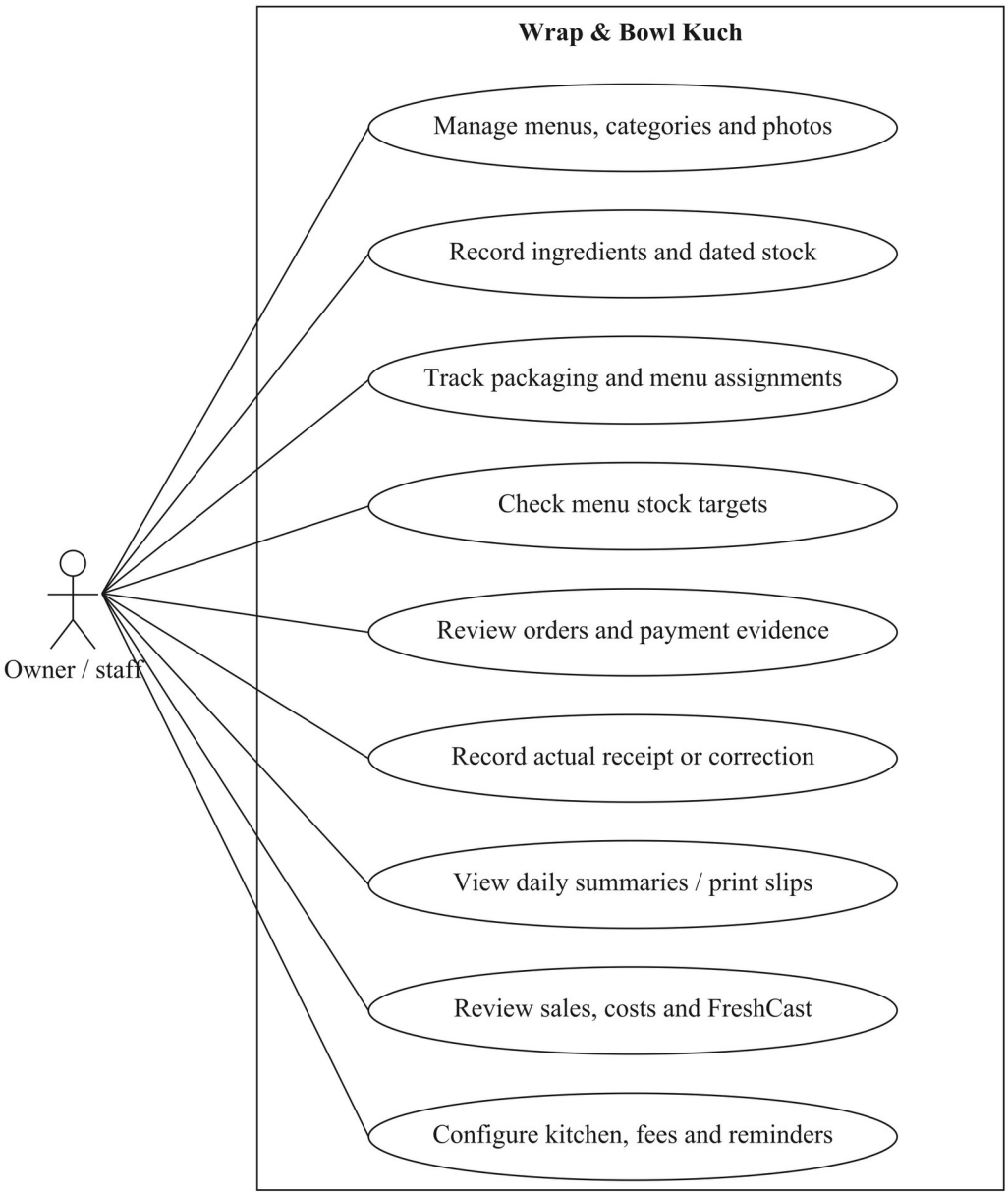
## 3.3 USE CASE DIAGRAMS

The customer diagram uses one Customer actor consistently. Public browsing does not require an account; checkout and private history do. The owner diagram represents authenticated staff. A bank or courier is not drawn as an automatically connected application actor because the current website does not call either provider's operational API. The customer uses a separate banking application, and the kitchen arranges the rider externally.




Browsing is public. Ordering requires an account. Bank transfer occurs outside the website.

**Figure 3.1: Customer use cases for the current storefront and account workflow.**




Staff password sign-in is required. A reminder or uploaded receipt never confirms delivery or payment.

**Figure 3.2: Owner/staff use cases for stock, orders, reporting and configuration.**

## 3.4 USE CASE SPECIFICATIONS

The specifications below describe the current client workflow. Shared preconditions include a valid session, role-appropriate permissions and current saved configuration. An error must leave an order, payment or stock operation either fully recorded or unrecorded; the interface must not treat an uncertain response as a successful second submission.

### 3.4.1 BROWSE MENU

| Use case | Current specification |
|---|---|
| ID / actor | UC-01 — Visitor/customer |
| Precondition | Public storefront available. |
| Main flow | Open the menu; search/filter categories; inspect price, contents and offered days. |
| Alternative / exception | Empty results or unavailable records show a recovery state. |
| Postcondition | No order or stock change. |

### 3.4.2 CREATE CUSTOMER ACCOUNT

| Use case | Current specification |
|---|---|
| ID / actor | UC-02 — Customer |
| Precondition | Browsing session; supported passkey device. |
| Main flow | Enter name and full phone; request registration; complete native verification. |
| Alternative / exception | Invalid phone, unsupported device, rejected origin/challenge or cancelled ceremony does not create false authentication. |
| Postcondition | Registered account preserves its existing basket; no website password/email required. |

### 3.4.3 SIGN IN

| Use case | Current specification |
|---|---|
| ID / actor | UC-03 — Customer / staff |
| Precondition | Registered passkey or staff credentials. |
| Main flow | Use the customer passkey; staff use the separate protected password form. |
| Alternative / exception | Wrong credential remains an error; the phone alone never opens another account. |
| Postcondition | Verified session and role-appropriate access. |

### 3.4.4 MANAGE PROFILE AND PASSKEYS

| Use case | Current specification |
|---|---|
| ID / actor | UC-04 — Customer |
| Precondition | Authenticated account. |
| Main flow | Review saved contact; reauthenticate for sensitive changes; add or remove a backup key. |
| Alternative / exception | Last-key removal and unauthorised/stale verification are rejected. |
| Postcondition | Profile/key change is saved without rewriting accepted order contacts. |

### 3.4.5 GET MENU SUGGESTIONS

| Use case | Current specification |
|---|---|
| ID / actor | UC-05 — Customer |
| Precondition | Menus configured; date finder needs a valid selection. |
| Main flow | Browse date-free cards or choose date, total units, mix/same and optional budget; inspect up to three alternatives. |
| Alternative / exception | No feasible match is explained; replacing a basket needs deliberate confirmation. |
| Postcondition | Proposed basket obeys relevant stock/pricing rules; no handover hour reserved. |

### 3.4.6 MANAGE BASKET

| Use case | Current specification |
|---|---|
| ID / actor | UC-06 — Customer |
| Precondition | Browsing or registered session. |
| Main flow | Add food, adjust quantity, remove a line and review subtotal. |
| Alternative / exception | Failures retain truthful current/error state; no duplicate successful add is invented. |
| Postcondition | Basket changes only; accepted stock is not yet consumed. |

### 3.4.7 CHOOSE FULFILMENT AND DATE

| Use case | Current specification |
|---|---|
| ID / actor | UC-07 — Customer |
| Precondition | Registered account and nonempty basket. |
| Main flow | Select delivery/collection, configured area if needed, common offered date and allowed time. |
| Alternative / exception | Invalid weekdays, cutoffs, time bounds or restricted area block a valid quote. |
| Postcondition | Current server-priced fulfilment selection; no accepted order yet. |

### 3.4.8 UPLOAD PAYMENT PROOF

| Use case | Current specification |
|---|---|
| ID / actor | UC-08 — Customer |
| Precondition | Manual payment available; confirmed quote and account. |
| Main flow | Pay externally if appropriate; select an eligible receipt image and upload it. |
| Alternative / exception | Nonimage, excessive size, wrong scope, unauthorised/private read or reused evidence is rejected. |
| Postcondition | Private scoped proof ID; payment is still unverified. |

### 3.4.9 PLACE PREORDER

| Use case | Current specification |
|---|---|
| ID / actor | UC-09 — Customer |
| Precondition | Account, valid fulfilment/contact, accepted price and required proof or delivery cash. |
| Main flow | Submit one request ID; server locks/rechecks current state and commits order plus allocations. |
| Alternative / exception | Any shortage, stale scope or later write error rolls back the entire placement; retry is idempotent. |
| Postcondition | Accepted snapshots, one ingredient/packaging deduction and saved order confirmation. |

### 3.4.10 VIEW OWN ORDER AND REVIEW FOOD

| Use case | Current specification |
|---|---|
| ID / actor | UC-10 — Customer |
| Precondition | Authenticated owner of order. |
| Main flow | Open My orders; inspect saved details/payment/receipt; use an eligible review or change action. |
| Alternative / exception | Another account cannot access private history; ineligible changes are rejected. |
| Postcondition | Read-only display or a separately validated audited change. |

### 3.4.11 MANAGE CATALOGUE

| Use case | Current specification |
|---|---|
| ID / actor | UC-11 — Owner/staff |
| Precondition | Staff session. |
| Main flow | Create/edit category/menu, price, unit contents, offered dates, image, recipe and selling status. |
| Alternative / exception | Invalid fields show form errors; protected historic relationships prevent unsafe deletion. |
| Postcondition | Current catalogue updates; accepted order names/prices remain unchanged. |

### 3.4.12 RECEIVE OR ADJUST INGREDIENT BATCH

| Use case | Current specification |
|---|---|
| ID / actor | UC-12 — Owner/staff |
| Precondition | Ingredient unit and actual received evidence available. |
| Main flow | Choose date basis, storage and handling; enter quantity; review deadline and save with a reason where needed. |
| Alternative / exception | Invalid evidence or stale edit version is rejected. |
| Postcondition | Dated batch and audit record; held/expired stock excluded. |

### 3.4.13 CHECK A STOCK TARGET

| Use case | Current specification |
|---|---|
| ID / actor | UC-13 — Owner/staff |
| Precondition | Current recipes and packaging configured. |
| Main flow | Choose a date and 1–100 units per offered menu; inspect totals, shortfalls and included menus. |
| Alternative / exception | Missing recipes/settings and no offered menus are reported distinctly. |
| Postcondition | Read-only target/worksheet; no receipt, reservation or deduction. |

### 3.4.14 MANAGE PACKAGING

| Use case | Current specification |
|---|---|
| ID / actor | UC-14 — Owner/staff |
| Precondition | Staff session. |
| Main flow | Create packaging, record received pieces/cost, assign per-menu pieces and review movements. |
| Alternative / exception | Invalid removal or missing supplies is rejected; removals require review. |
| Postcondition | Current pieces and menu mappings saved; old accepted allocations preserved. |

### 3.4.15 RECORD VERIFIED PAYMENT

| Use case | Current specification |
|---|---|
| ID / actor | UC-15 — Owner/staff |
| Precondition | Accepted order and bank/cash evidence. |
| Main flow | Inspect private proof; check external receipt; record full payment reference and outcome. |
| Alternative / exception | An upload alone is insufficient; duplicate/conflicting events are validated. |
| Postcondition | Payment event/state updated; food receipt unchanged. |

### 3.4.16 CONFIRM ACTUAL FOOD RECEIPT

| Use case | Current specification |
|---|---|
| ID / actor | UC-16 — Owner/staff |
| Precondition | Eligible current order(s); actual handover confirmed. |
| Main flow | Select one or more orders; review exact set and confirm food received. |
| Alternative / exception | Stale/ineligible selected order prevents an unreviewed partial update. |
| Postcondition | Actual receipt/audit state updated; no invented payment. |

### 3.4.17 REVIEW DAILY SUMMARY AND PRINT

| Use case | Current specification |
|---|---|
| ID / actor | UC-17 — Owner/staff |
| Precondition | Orders and a chosen handover date. |
| Main flow | Open daily menu totals; inspect orders; open a slip and explicitly print/save PDF. |
| Alternative / exception | Undated records and missing slips are handled without invented dates. |
| Postcondition | Read-only report/preview with accepted name/address/phone. |

### 3.4.18 REVIEW SALES AND PRICING

| Use case | Current specification |
|---|---|
| ID / actor | UC-18 — Owner/staff |
| Precondition | Relevant recorded orders and known costs where required. |
| Main flow | Select period; distinguish operational value from paid/received sales; compare a price/bulk scenario. |
| Alternative / exception | Unknown costs remain unavailable; CSV ranges and formula text are bounded/sanitised. |
| Postcondition | Read-only report/export/scenario; live prices unchanged. |

### 3.4.19 RECORD WASTE OR EXPENSE

| Use case | Current specification |
|---|---|
| ID / actor | UC-19 — Owner/staff |
| Precondition | Batch quantity or overhead evidence. |
| Main flow | Review a waste deduction with reason, or record/void an eligible operating expense. |
| Alternative / exception | Do not double-count cooking deductions or ingredients as overhead; invalid quantities rejected. |
| Postcondition | Waste adjusts stock with cost evidence; expense ledger remains separate. |

### 3.4.20 MANAGE KITCHEN SETTINGS

| Use case | Current specification |
|---|---|
| ID / actor | UC-20 — Owner/staff |
| Precondition | Staff session. |
| Main flow | Save identity/story, area fees/hours, enabled collection, receiving QR and promotion section. |
| Alternative / exception | Invalid configuration is rejected; one section save preserves unrelated drafts. |
| Postcondition | New quotes use settings; accepted terms remain snapshots. |

### 3.4.21 PROCESS OWNER REMINDER

| Use case | Current specification |
|---|---|
| ID / actor | UC-21 — Scheduler / owner |
| Precondition | Authenticated worker and configured SMTP. |
| Main flow | Check due delivery/collection events and receipts; recheck under lock; attempt bounded sends. |
| Alternative / exception | Unauthorised calls rejected; failed sends do not save a success receipt. |
| Postcondition | Run metadata/accepted-send receipt only; order/payment/stock not changed. |

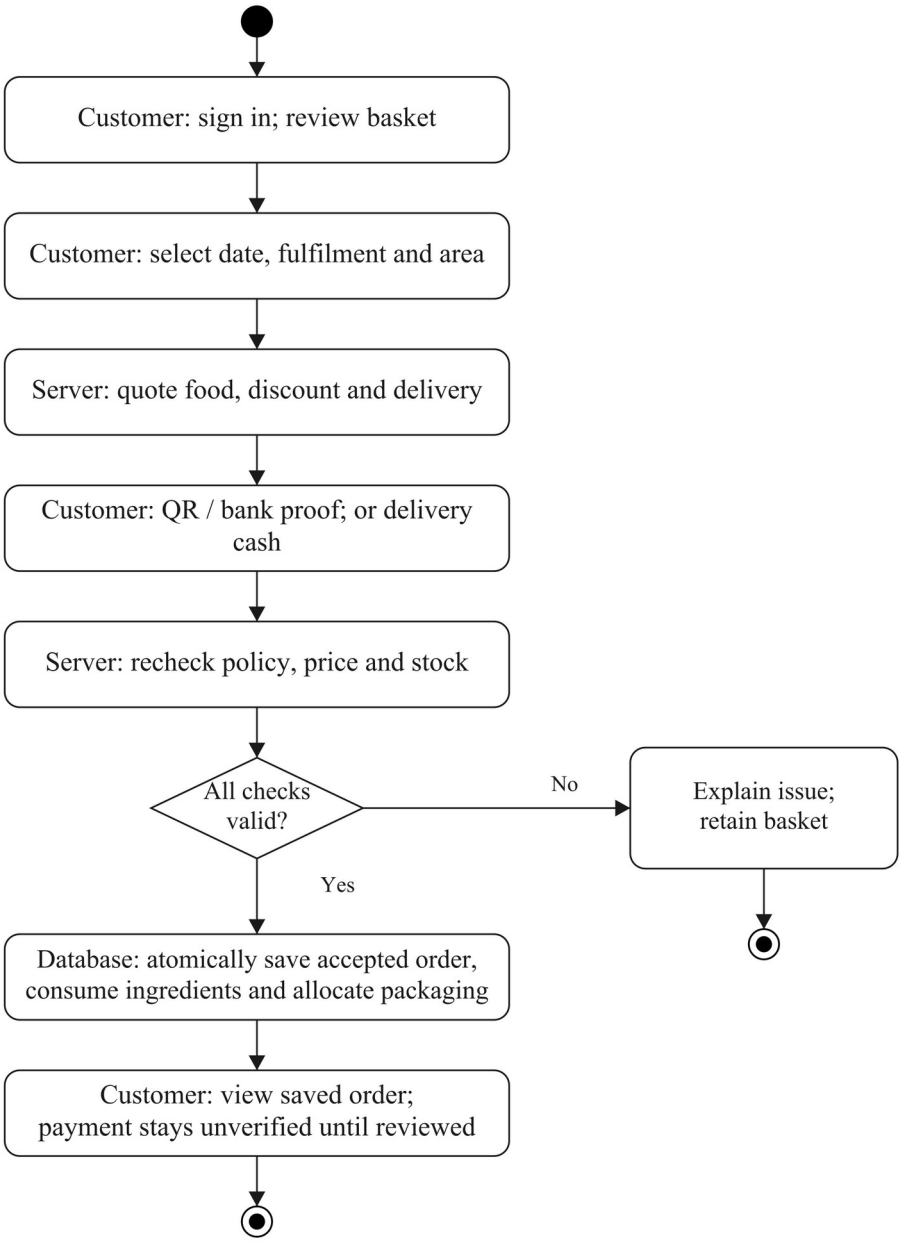
### 3.4.22 USE KITCHEN HELP AND FRESHCAST

| Use case | Current specification |
|---|---|
| ID / actor | UC-22 — Customer / staff respectively |
| Precondition | Relevant public help or staff decision route available. |
| Main flow | Use supported help/contact guidance; staff select clearly labelled decision data/scenarios. |
| Alternative / exception | Service failure provides recovery; local, external and illustrative evidence are not mixed. |
| Postcondition | Assistance or hypothetical analysis; no automatic banking/courier/procurement action. |

## 3.5 ACTIVITY DIAGRAMS

### 3.5.1 CUSTOMER CHECKOUT AND STOCK COMMITMENT

The customer selects delivery or available collection, an offered date and an allowed time. Delivery requires an area from the kitchen's configured list when that list is restricted. The server quotes food, bulk discount and delivery together. Delivery can use an available QR/bank-transfer method or cash. Collection requires a receiving QR and payment evidence. Placement repeats the relevant checks inside the order transaction and creates accepted terms before allocating stock. A failure must preserve the basket without a partial order or partial deduction.

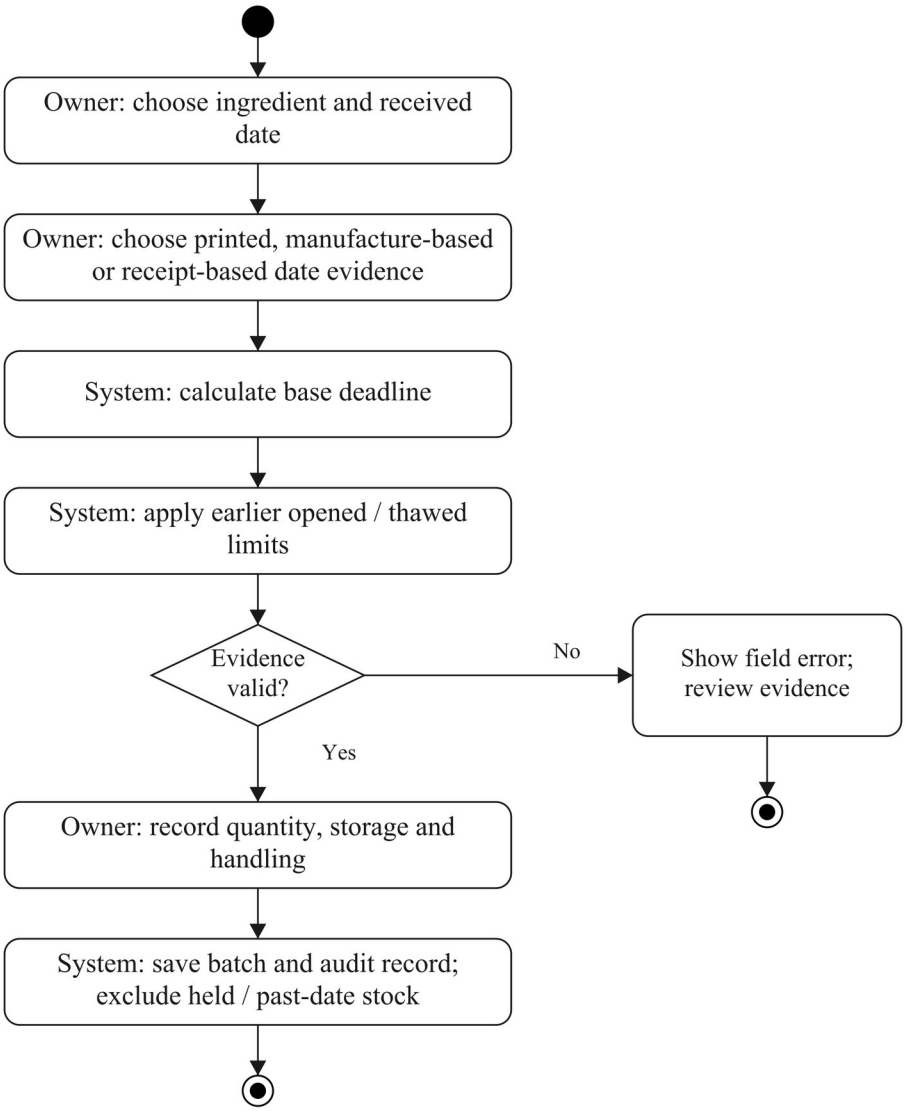



Collection requires QR proof. A stock failure rolls back the entire placement; a repeated request does not deduct twice.

**Figure 3.3: Checkout validation and atomic ingredient/packaging allocation.**

### 3.5.2 DATED INGREDIENT BATCHES

An ingredient definition supplies its inventory unit and optional documented storage guidance. A received batch records its own quantity, dates, actual storage and handling evidence. Printed-date, manufacture-plus-life and receipt-plus-storage-life modes use different anchors. Opening and thawing limits can shorten the base deadline. Uncertain handling or a known breach holds the batch. Selecting a wet-market source does not itself prove that the product has no usable supplier label or establish a safe duration.

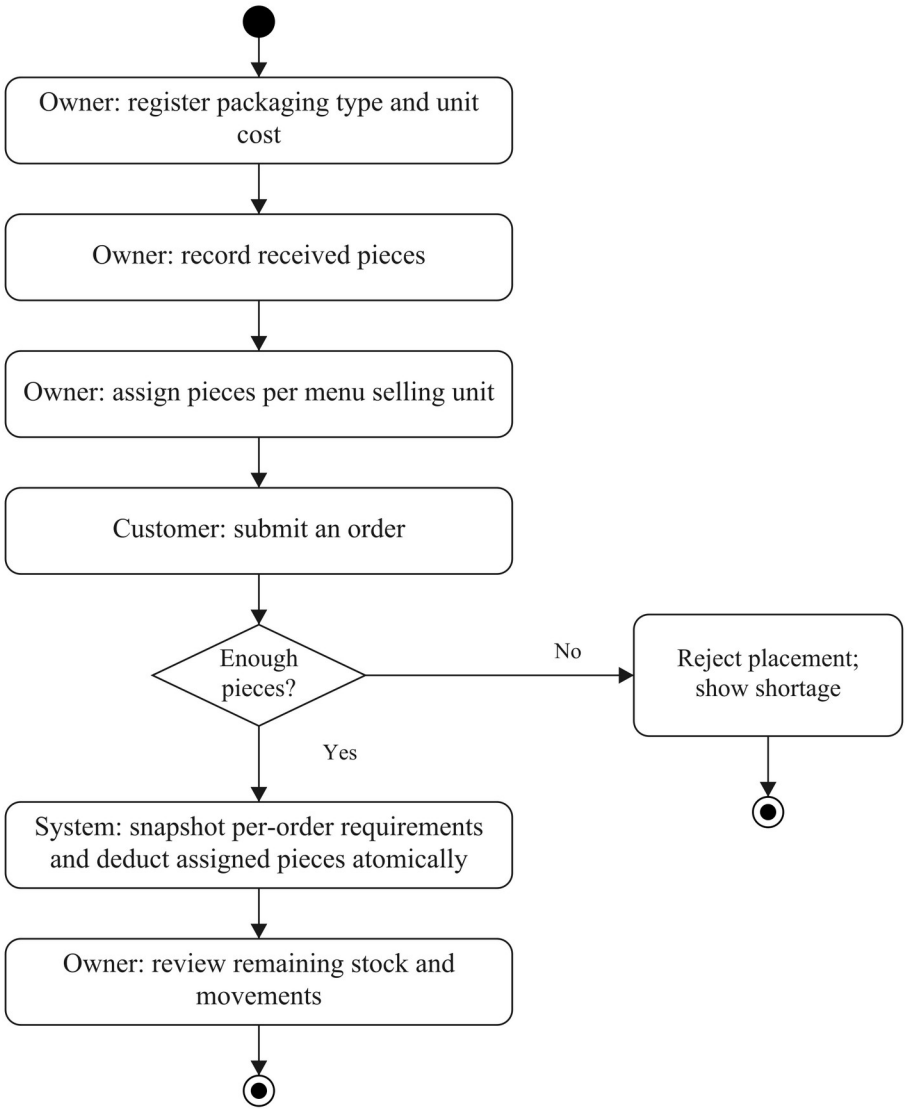



Documented shelf life is a calendar planning rule, not a physical freshness or food-safety measurement.

**Figure 3.4: Batch date evidence, validation and stock eligibility.**

### 3.5.3 PACKAGING STOCK

Packaging is tracked in whole pieces independently of edible ingredients. Each menu maps the pieces required for one selling unit. Placement multiplies these requirements by the ordered quantity and captures the accepted packaging names, quantities and costs. A later menu edit cannot rewrite that accepted allocation.

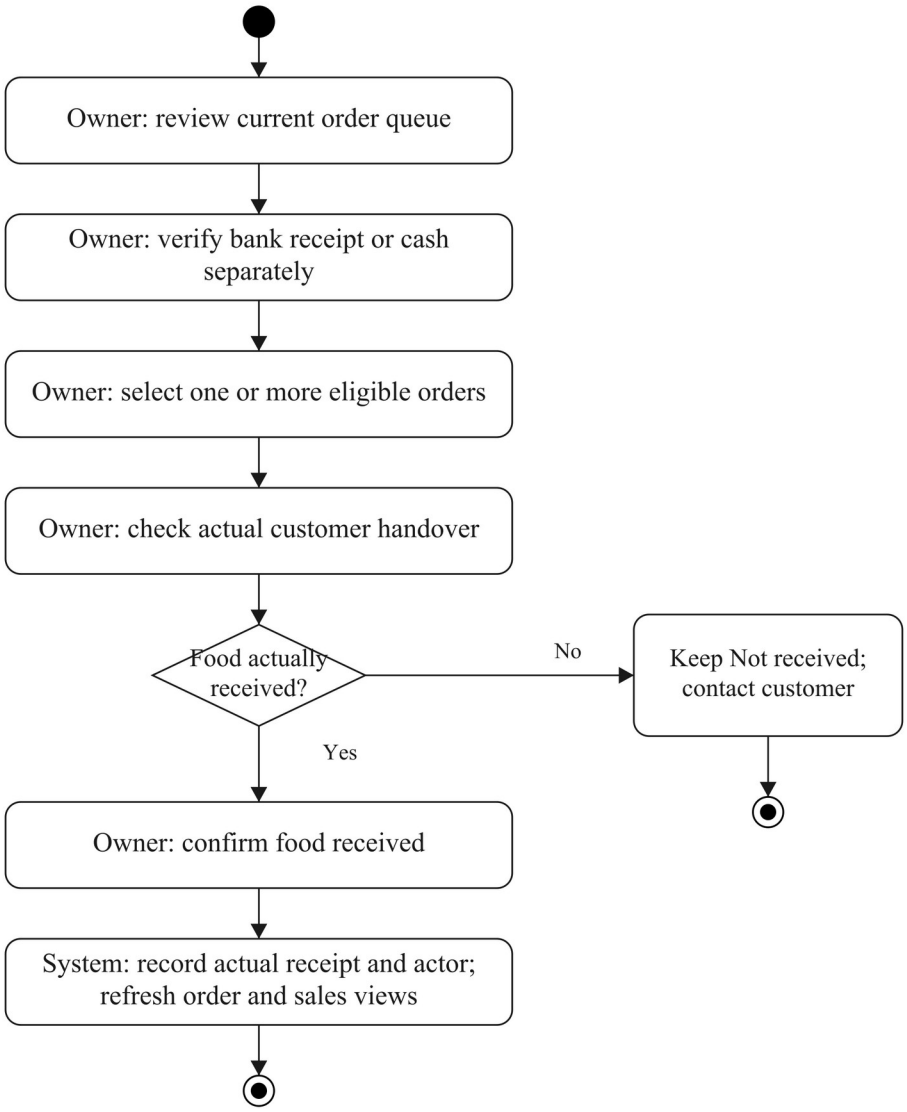



A low-stock warning does not create stock. Accepted packaging requirements survive later menu edits.

**Figure 3.5: Packaging receipts, menu mappings and placement deductions.**

### 3.5.4 ACTUAL FOOD RECEIPT

The owner confirms actual receipt only when the customer has received or collected the food. Several eligible orders can be selected together, but the selected set and current versions must be rechecked. A planned time, bank receipt, reminder or timer cannot establish that the food was received. Corrections and cancellation use their own audited operations.




The clock and email worker never mark food received. Receipt of food and receipt of money are separate facts.

**Figure 3.6: Separate payment review and actual customer receipt.**

## 3.6 SEQUENCE DIAGRAMS

### 3.6.1 CUSTOMER PASSKEY ACCESS

Registration and authentication use session-bound, short-lived WebAuthn challenges. The device obtains user verification and signs a response; Django verifies the relevant challenge, origin, relying party and credential data. The server stores a public key, credential identifier and account association. It does not receive the customer's fingerprint, face template, private key or device PIN. A matching phone number is not used to take over an existing account.

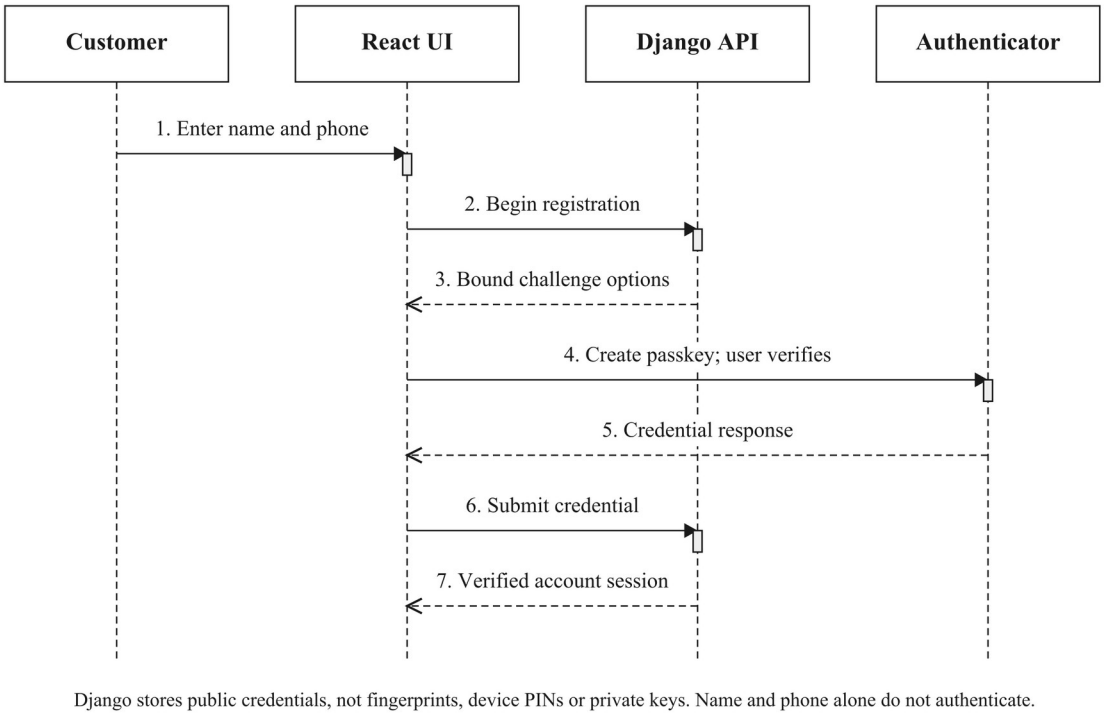



**Figure 3.7: Customer passkey registration.**

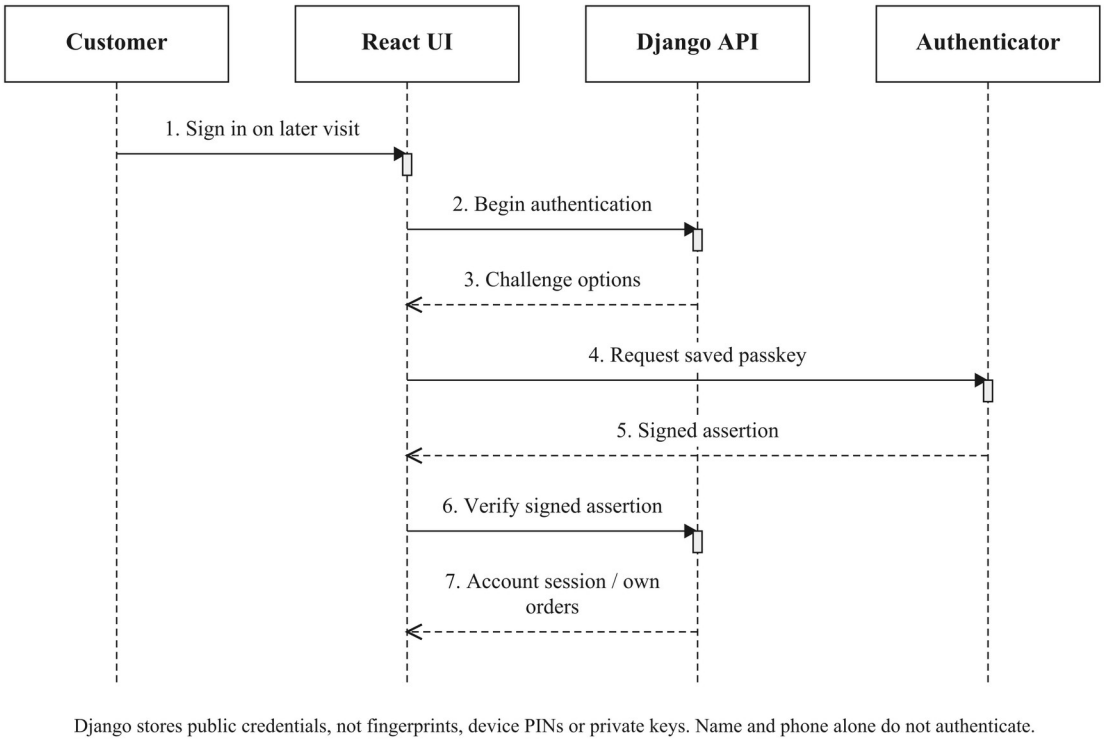



**Figure 3.8: Authentication using a saved customer passkey.**

### 3.6.2 QUOTE, PRIVATE PROOF AND ORDER PLACEMENT

The quote is advisory until placement succeeds. Its pricing uses the current basket and fulfilment selection. A receipt upload is authenticated, sanitised and bound to a price/selection scope. Changing the basket, recipient configuration or relevant

fulfilment details can require the existing receipt to be reviewed or uploaded again; it must not prompt a second bank payment automatically. Placement uses a request identifier and locks current state so replay does not allocate ingredients or packaging twice.

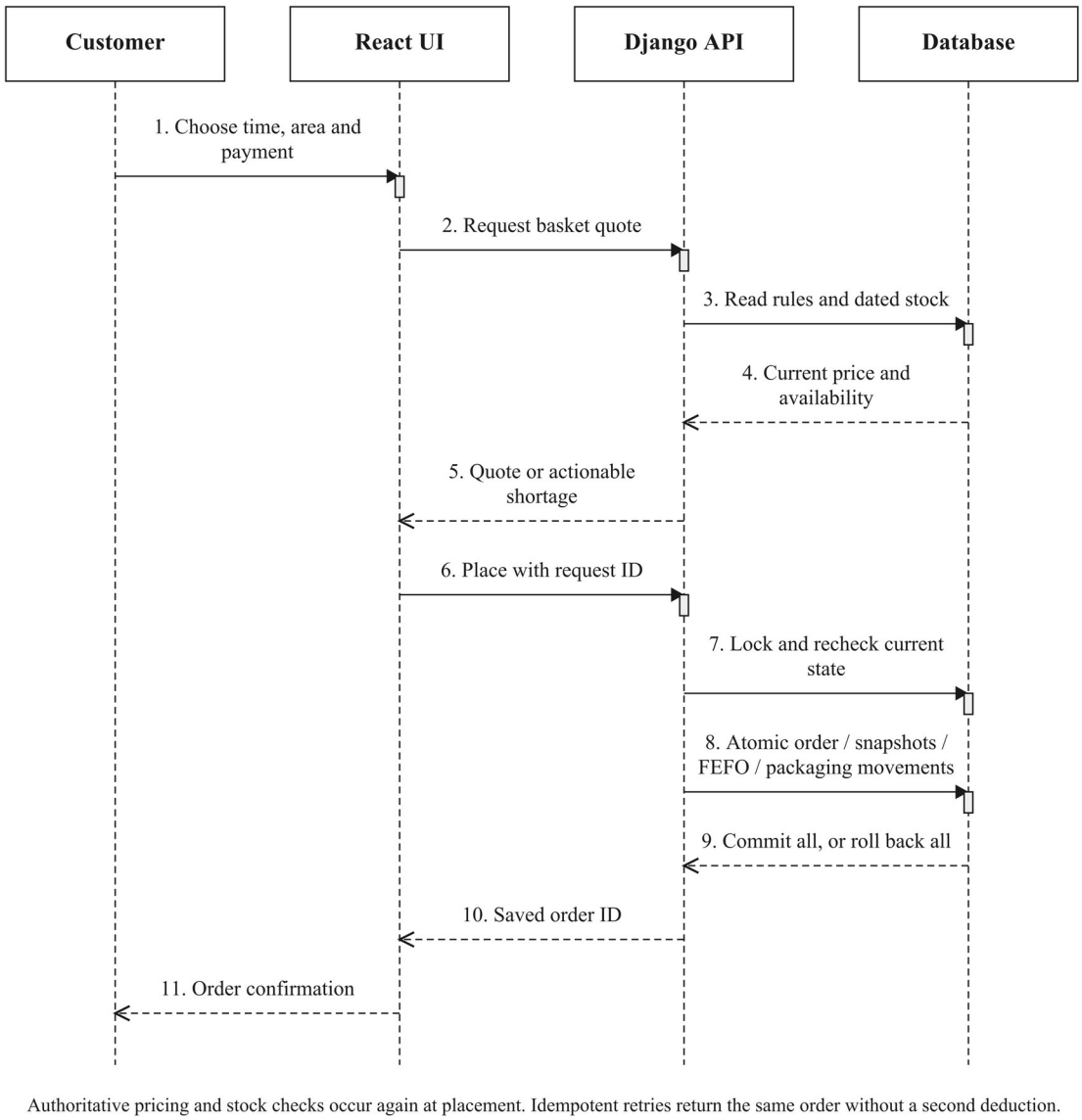



**Figure 3.9: Authoritative pricing, scoped evidence and transactional placement.**

### 3.6.3 OWNER PAYMENT VERIFICATION

The kitchen reviews the uploaded image and its bank records outside the website. Recording a completed full receipt creates a payment event and updates the payment state under server validation. A refund record describes money already returned externally; it is not a bank-transfer instruction. The payment operation does not establish physical food receipt.

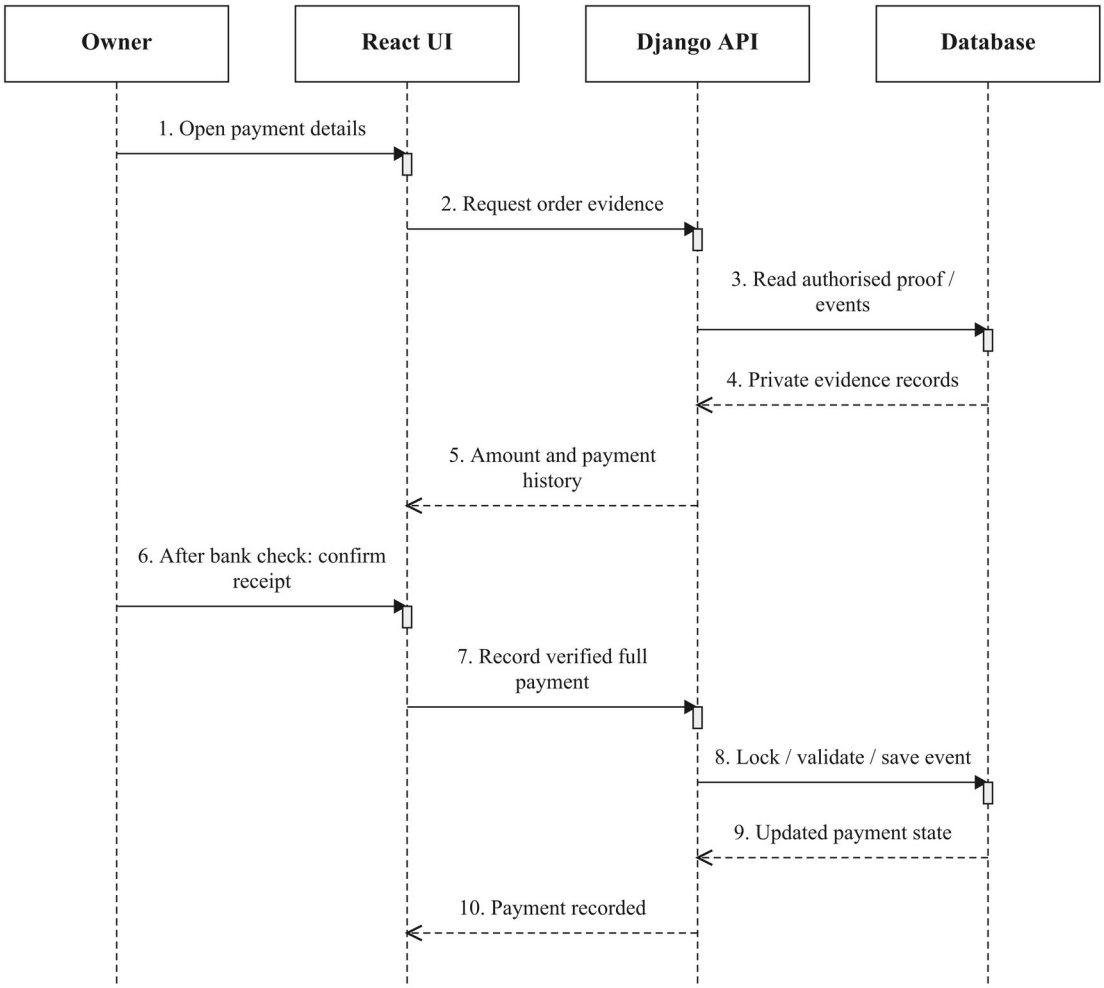



**Figure 3.10: Private evidence review and manual payment recording.**

### 3.6.4 REPORTING AND PRINT PREVIEW

Daily reports aggregate accepted line names and quantities for the selected handover date. Sales reporting applies its own paid-and-received inclusion rules. A printable slip uses the saved customer, phone, address and collection/delivery details. Opening its preview does not change the order, and printing starts only from an explicit print action.

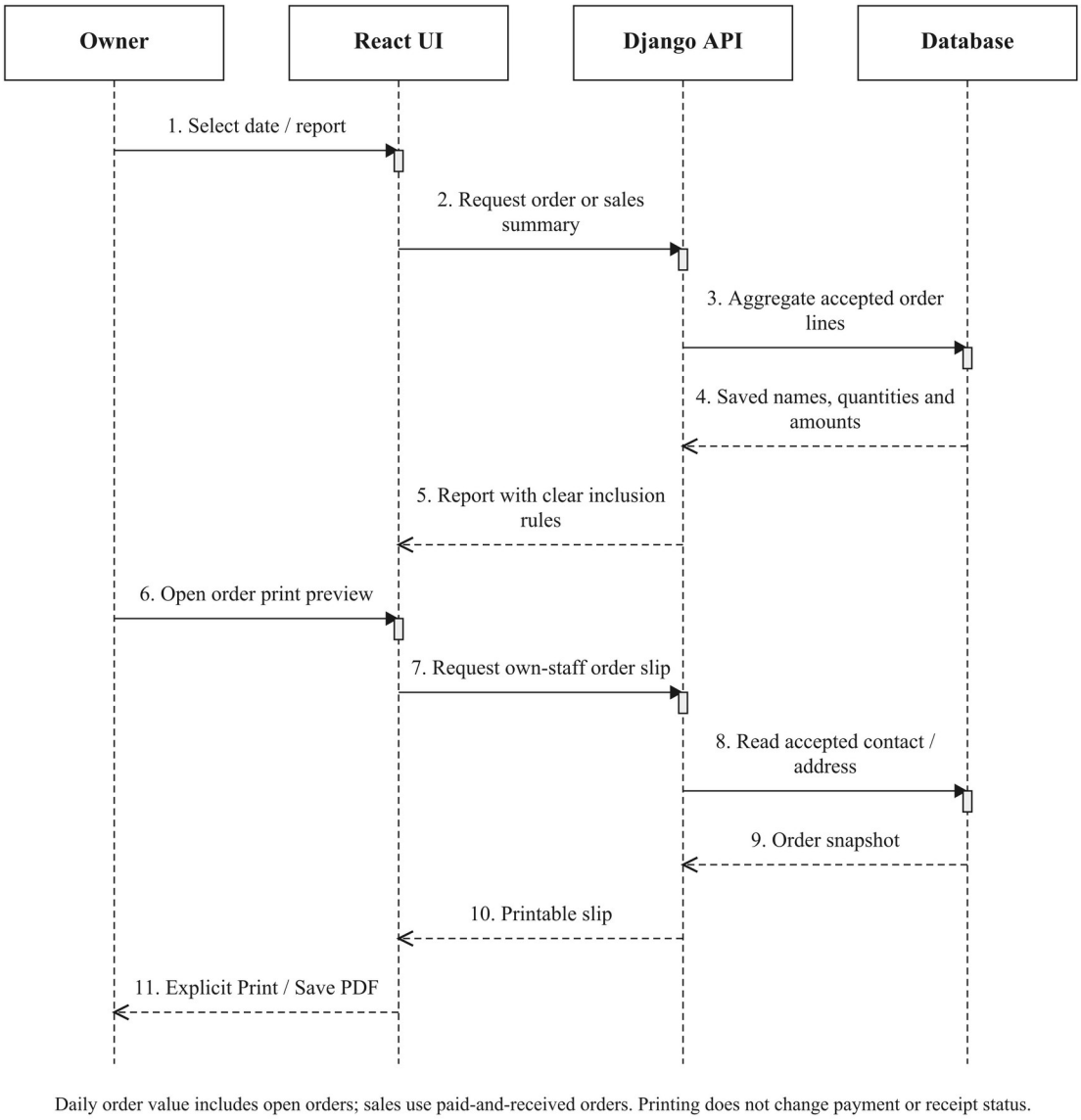



**Figure 3.11: Daily/sales reporting and read-only order-slip preview.**

### 3.6.5 OWNER HANDOVER REMINDERS

One authenticated worker checks eligible reminder events. In simple mode, the current owner queue concerns delivery/collection rather than preparation. Successful provider acceptance is followed by a saved reminder receipt; failed attempts can be retried. The worker has a bounded attempt budget. Provider acceptance is not proof of inbox receipt, and a crash between acceptance and saving the receipt can still cause a duplicate on retry.

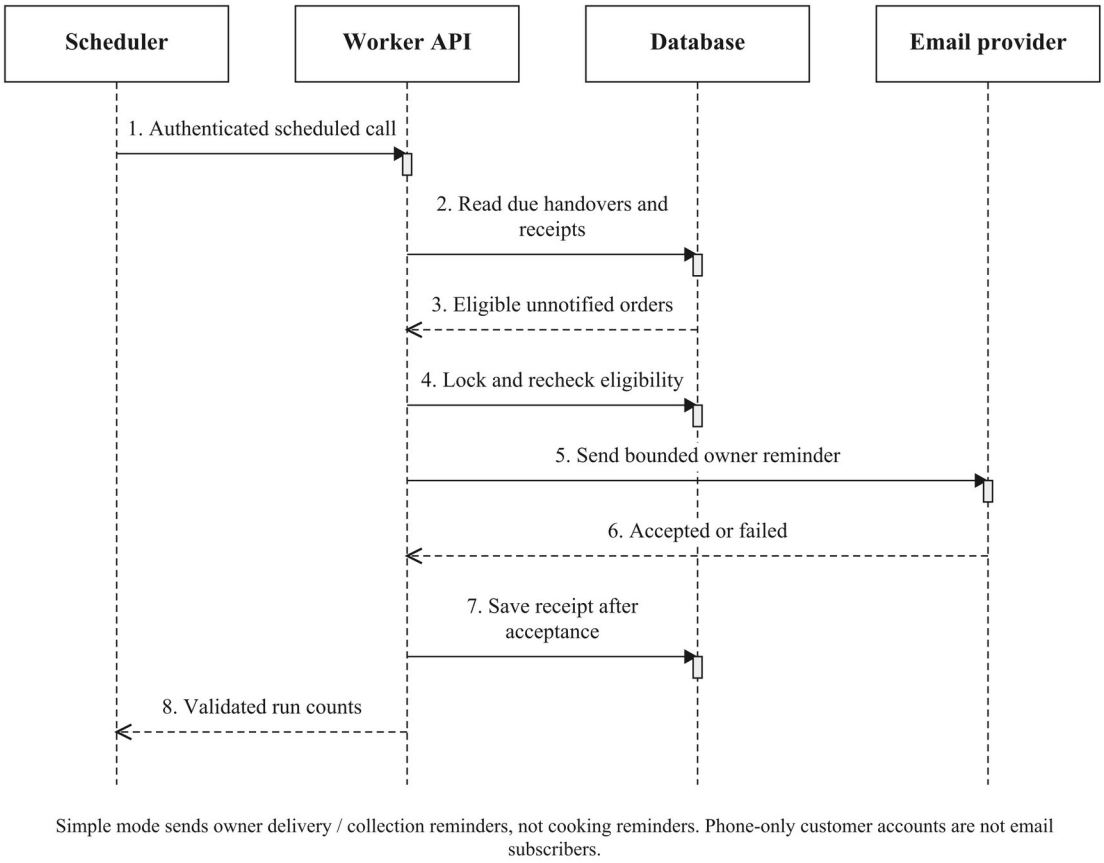



**Figure 3.12: Authenticated, bounded owner handover reminder processing.**

## 3.7 CLASS DIAGRAM

Figure 3.13 shows a selected core of the implemented Django domain, rather than inventing a separate class for every screen. Product represents the current menu, while OrderItem records accepted purchase terms. ProductIngredient connects current recipes to RawMaterial; InventoryItem records dated lots. The full application contains additional account, packaging, payment and operational models described in the ERDs and Appendix D. Methods shown correspond to the actual implementation.

## 3.8 ENTITY–RELATIONSHIP DIAGRAMS

The ERDs show selected attributes and actual relationship columns. Multiplicities apply at the indicated entity ends. Database fields for earlier scheduling, wallet and external-payment experiments still exist for compatibility. Their presence does not imply that those methods are available in the current client checkout. Appendix D distinguishes current and supporting/legacy models.

### 3.8.1 IDENTITY AND ACCOUNT ACCESS

Django’s auth_user remains the underlying identity table. A browsing session can use an opaque GuestCustomer identity with an unusable password. A registered

customer has a CustomerAccount and one or more public CustomerPasskey credentials. PasskeyCeremony binds a short-lived action to its session and, where applicable, user. Staff continue to use the protected staff credential flow. Address is retained as a supporting/legacy address entity, while current orders preserve accepted contact/address snapshots.

### 3.8.2 CATALOGUE, RECIPES AND INVENTORY

A category contains current menus. A recipe joins a menu and ingredient with a positive quantity in that ingredient's inventory unit. A material can have many dated batches and each batch can have many inventory log entries. Batch quantity and date eligibility are authoritative for the stock check; a displayed stock-supported menu quantity is derived, not a second manually maintained physical count.

### 3.8.3 ACCEPTED ORDERS AND CONSUMPTION

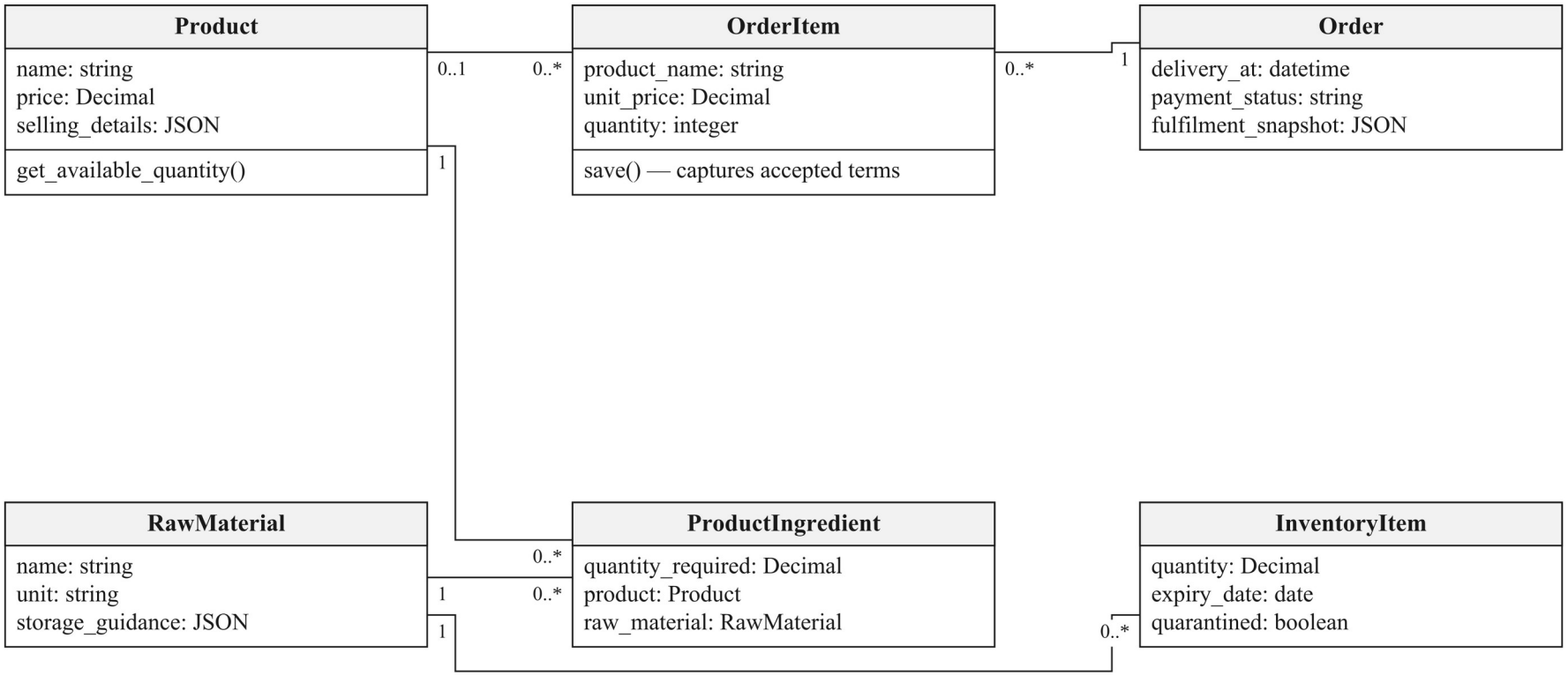
OrderItem has both a nullable Product foreign key and its own product_name, unit_price and quantity. History displays the saved name and price, so a menu rename does not rewrite a previous purchase. OrderIngredient preserves the accepted recipe requirement. IngredientConsumption records actual batch allocations against order items, including quantity, cost and recorded expiry evidence. CheckoutProof is private evidence belonging to its user and, after acceptance, its associated order.

### 3.8.4 PACKAGING RELATIONSHIPS

ProductPackaging defines the current per-menu requirement. OrderPackaging preserves the accepted per-order-line requirement, including the packaging name and cost where known. PackagingMovement records stock changes and can refer to an accepted packaging allocation. Names and quantities are deliberately snapshotted so later catalogue edits do not alter history.

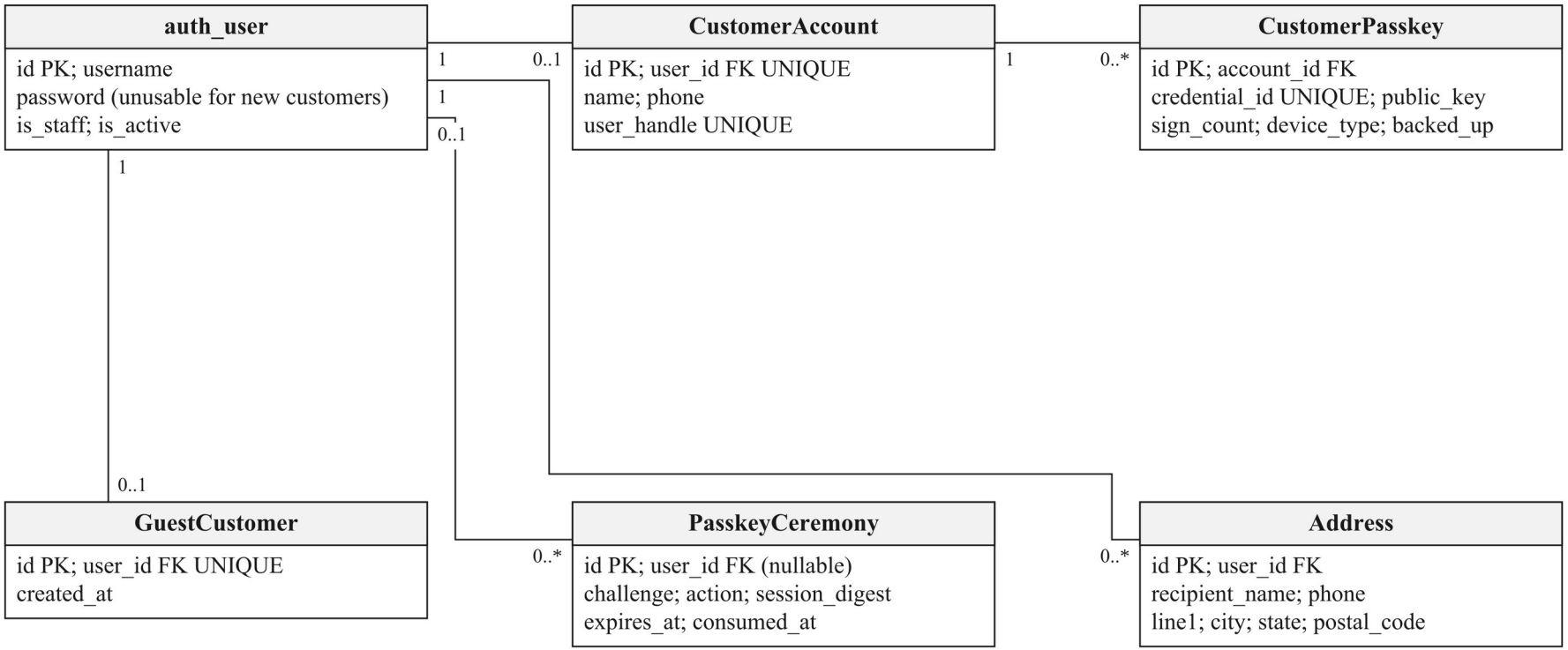
### 3.8.5 FINANCIAL AND AUDIT RECORDS

PaymentEvent stores completed/pending/failed receipt or refund evidence. OrderAmendment stores the actor and before/after documents for relevant corrections. OperatingExpense records overhead rather than ingredient purchases. WasteRecord associates discarded quantities and estimated cost with an inventory batch. These records support an audit trail; they do not replace bank reconciliation or establish net profit.



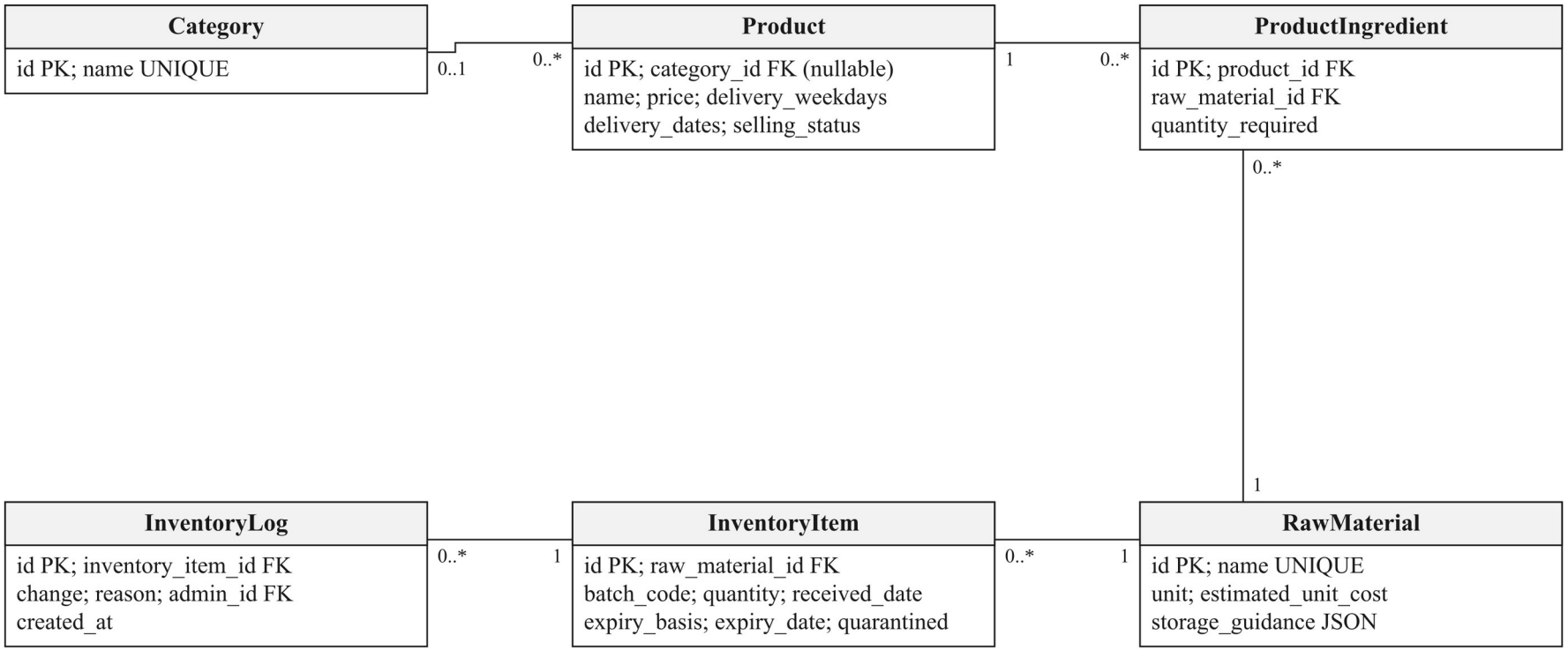

Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

**Figure 3.13: Selected core domain classes and their associations.**



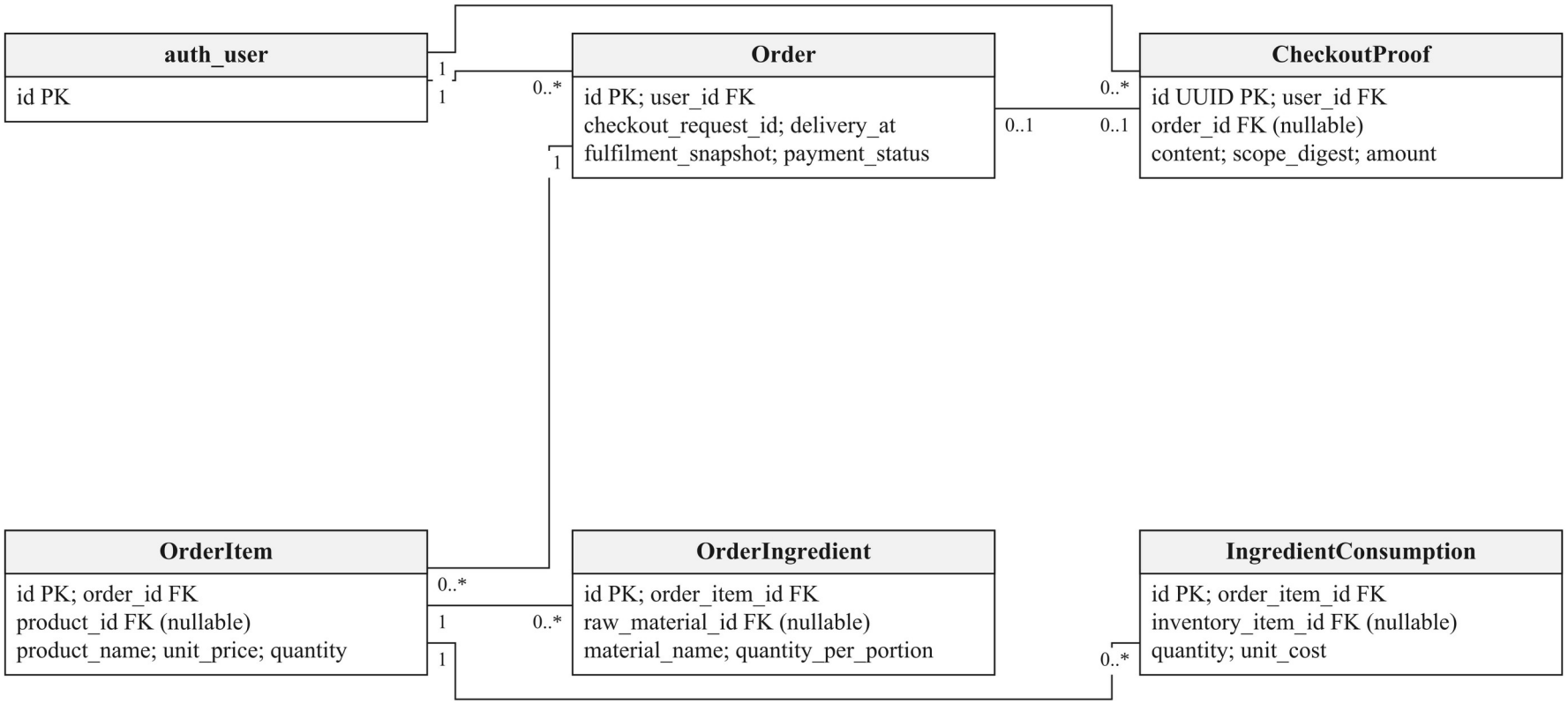

Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

**Figure 3.14: Identity, customer profile and passkey records.**



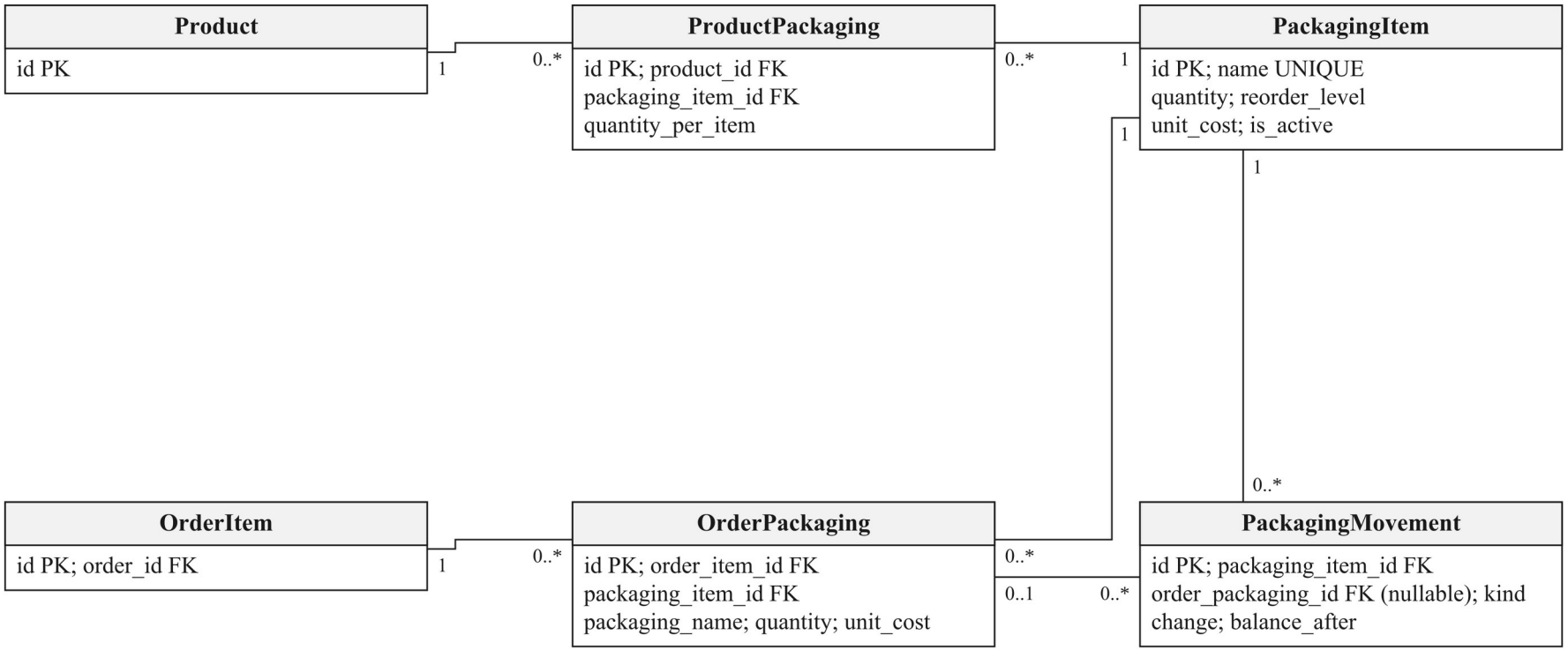

Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

**Figure 3.15: Catalogue, recipe, batch and inventory-log relationships.**



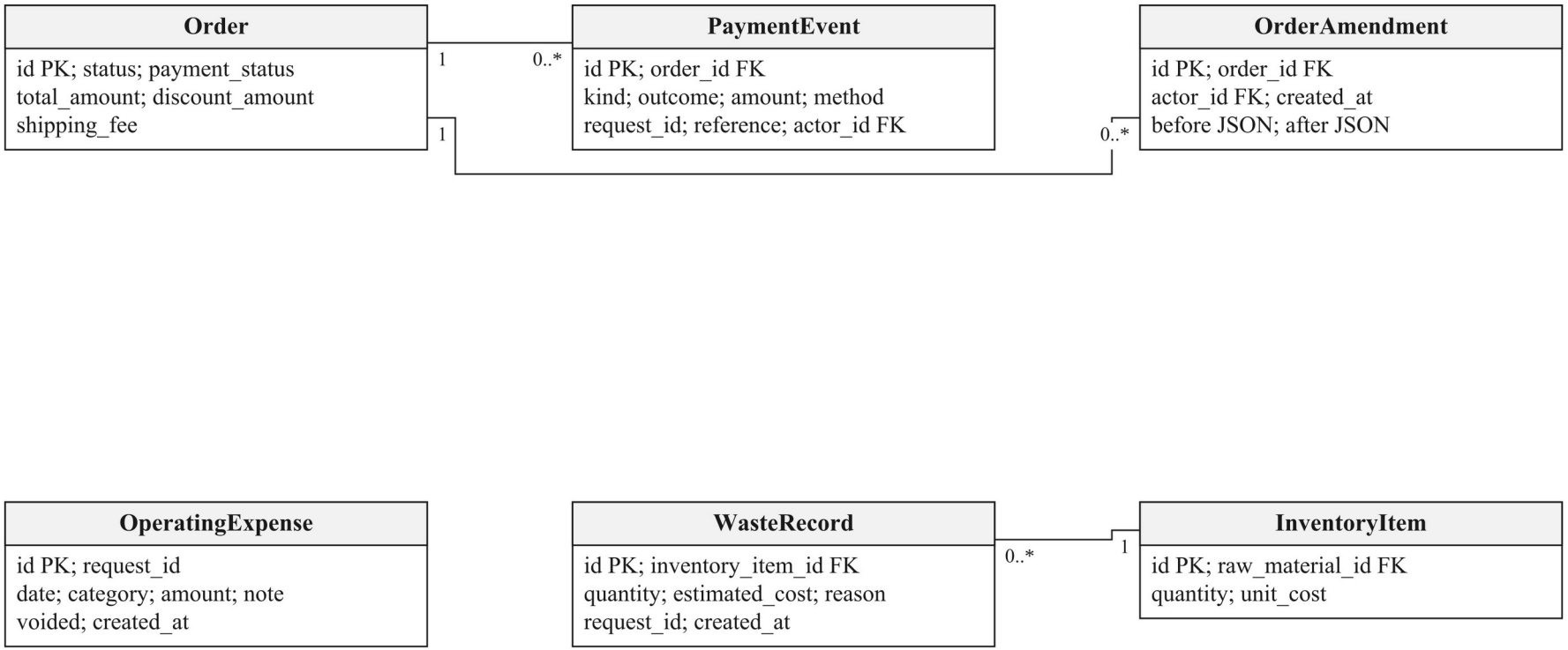

Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

**Figure 3.16: Accepted order terms, private proof and ingredient allocation.**




Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

**Figure 3.17: Current and accepted packaging requirements and stock movements.**




Selected fields shown. PK = primary key; FK = foreign key; multiplicities are at the linked ends.

**Figure 3.18: Payment, amendment, expense and wastage evidence.**

## 3.9 REQUIREMENT TRACEABILITY AND DESIGN LIMITS

The first objective is realised through permitted future handover dates, editable delivery/collection hours and date-dependent menu checks. It does not require automatic cooking-task allocation after the client removed that workflow. The second objective is implemented through recorded shelf-life evidence and storage duration; it is not a sensor-based food-safety claim. The single-vendor storefront supports the third objective. Sales and cost evidence support the fourth, and the two recommendation surfaces implement the fifth. Whether recommendations reduce decision time or increase conversion must be evaluated with users rather than inferred from successful test requests.

The design intentionally leaves transfers, rider booking and physical handover outside automatic verification. It provides clear records and contact routes around those activities. Supported browser/device access, honest stock records, correct ingredient quantities and owner configuration remain prerequisites. Chapter 5 identifies the checks performed and the empirical claims that remain unmeasured.

## 3.10 CONCLUSION

The revised design reflects the client's simpler ordering model while keeping the original project objectives. Accepted-order snapshots protect historical terms, and the placement transaction coordinates ingredient and packaging records. The next chapter explains the implemented interfaces, calculations and development decisions using this design.

# CHAPTER 4

# INTERFACE AND SYSTEM IMPLEMENTATION

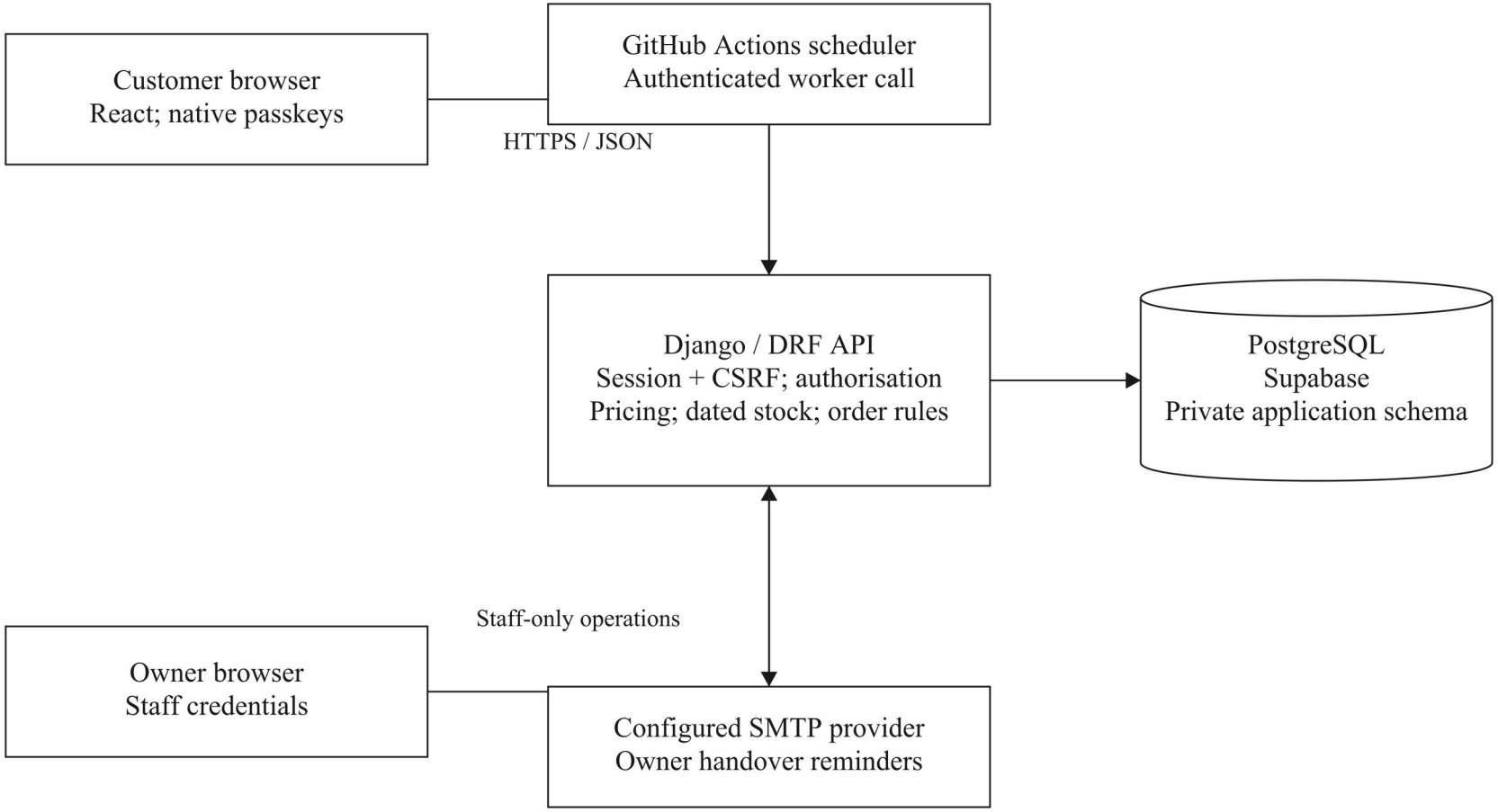
## 4.1 INTRODUCTION

The implemented application combines a customer storefront and a staff workspace for Wrap & Bowl Kuch. This chapter describes the current source revision, 6edde5e, rather than the earlier Dapur Kita report baseline. The client's public menu poster and founder story supplied the business identity, eight products, advertised prices and offered days. The project author authorised editable recipe estimates, standard packaging assignments and fictional operational records for demonstration and testing. These are implementation inputs, not measured production yields, actual sales or certified storage guidance.

The supplied senior report organises its interface chapter around customer and administrator tasks. This chapter follows that useful structure while also explaining the development decisions and problems highlighted by the briefing rubric. The screenshots are actual retained browser captures from isolated developer scenarios, not hand-drawn interface mock-ups. Desktop and phone views use ordinary viewport captures; an order-slip image is a print preview. Different captures document different development checks, so they are not presented as photographs of the client's live database or as proof that the latest Vercel deployment was observed.

## 4.2 APPLICATION ARCHITECTURE AND TECHNOLOGY

React renders the storefront, account dialogs and staff pages. Django with Django REST Framework validates requests, authenticates sessions, calculates prices and enforces stock/order rules. PostgreSQL, hosted through Supabase for the deployed setup, provides persistent application records. Vercel hosts the application deployment; it is not a substitute database for the order and inventory tables. Local verification uses isolated SQLite, with PostgreSQL-specific tests identified separately. Session writes use CSRF protection and privileged operations require the appropriate staff role.




QR transfers and courier bookings happen in external apps. There is no live bank or GrabExpress API connection.

**Figure 4.1: Current customer/staff application architecture and external boundaries.**

The architecture keeps user-interface convenience separate from authority. A disabled button is helpful feedback, but it does not replace server validation. A client-supplied total is compared against server-calculated prices. Stock is rechecked at acceptance, receipt images are served privately, and a customer cannot read another customer's history by changing an identifier. Request cancellation and session-current guards prevent late responses from repopulating data after an account change.

**Table 4.1: Technology responsibilities in the current implementation**

| Component | Responsibility | Important boundary |
|---|---|---|
| React and React Router | Responsive pages, dialogs, form feedback and navigation | Client state does not authorise a payment or stock mutation |
| Django / DRF | Sessions, role checks, validation, pricing and transactions | Current rules are enforced again on submission |
| PostgreSQL / Supabase | Orders, accepted terms, stock, public credentials and evidence | Private application tables and binary proof are not public download folders |
| Native WebAuthn | Device-mediated customer registration and sign-in | The website never receives biometric templates or a device PIN |
| SMTP and scheduled worker | Bounded owner delivery/collection reminders | An accepted send is not an inbox or delivery guarantee |
| External banking/courier apps | Actual transfer and rider booking | There is no active bank reconciliation or GrabExpress API integration |

## 4.3 CLIENT REQUIREMENTS AND DEVELOPMENT ITERATIONS

The original system explored detailed kitchen scheduling and additional payment mechanisms. The client's requirements shifted the operating interface towards a Google-Form-like contact experience, morning preorders and simpler fulfilment handling. The implementation retained security through passkeys rather than allowing anyone who knows a phone number to open that person's order history. It also retained the recipe records needed for ingredient calculations while removing cooking-plan confirmation from the client's normal workflow.

The changes were delivered in reviewable increments. Packaging stock and order allocations were introduced alongside placement deduction. Reporting added daily menu-unit totals, paid/received sales and a preview-first print slip. Follow-up iterations synchronised area fees and checkout totals, corrected phone formats, preserved profile

defaults, required the customer account before ordering and clarified the distinction between received food and received money. Later interface passes reduced dense settings content, moved date selection to checkout, aligned inputs/actions and repaired dialog scroll-lock cleanup.

**Table 4.2: Representative development problems and implemented responses**

| Observed issue | Implemented response | Verification approach |
|---|---|---|
| Contact-only login could expose another account | Native passkey authentication; phone remains contact information | Challenge/origin/signature/ session tests and virtual-device browser flows |
| Delivery fee looked free after an area selection | Authoritative area pricing, refresh state and stale-total rejection | Quote/proof/placement reconciliation and browser checks |
| Ingredient and packaging records could diverge from ordering | Shared accepted requirements and transactional placement allocation | Exact stock-deduction, rollback and replay tests |
| Customer could choose an unoffered day | Shared basket ordering-day guidance and upcoming date shortcuts | Weekday intersection, invalid-date and quote-clearing checks |
| An uploaded proof was confused with paid status | Explicit unverified evidence and separate owner payment recording | Permission, private-file and payment-event tests |
| Admin sign-in occasionally left scrolling disabled | Shared dialog scroll-lock ownership and final-owner cleanup | Both dialog closing orders, repeated switches and actual wheel scrolling |
| Stock-check form labels and action were unclear/misaligned | Date to check, Units per menu, target example and scoped grid | Measured desktop/tablet alignment and stacked phone controls |

These iterations are evidence of a responsive development process. They do not establish that all usability defects have been eliminated. The client acceptance form and customer task study in Chapter 5 provide a route for checking the revised interfaces with the intended users.

## 4.4 CUSTOMER INTERFACES

### 4.4.1 BRANDED STOREFRONT AND MENU

The storefront presents one kitchen's identity without competitor listings. Settings supply the kitchen name, introduction, founder name, story, food philosophy, signature food, service-area description and contact route. The case-study story identifies Ayu Shafinaz and describes the cooking hobby that became Wrap & Bowl Kuch. No founder

photograph or nutritional certification is invented. Illustrative menu graphics can be replaced through the menu photo upload controls.

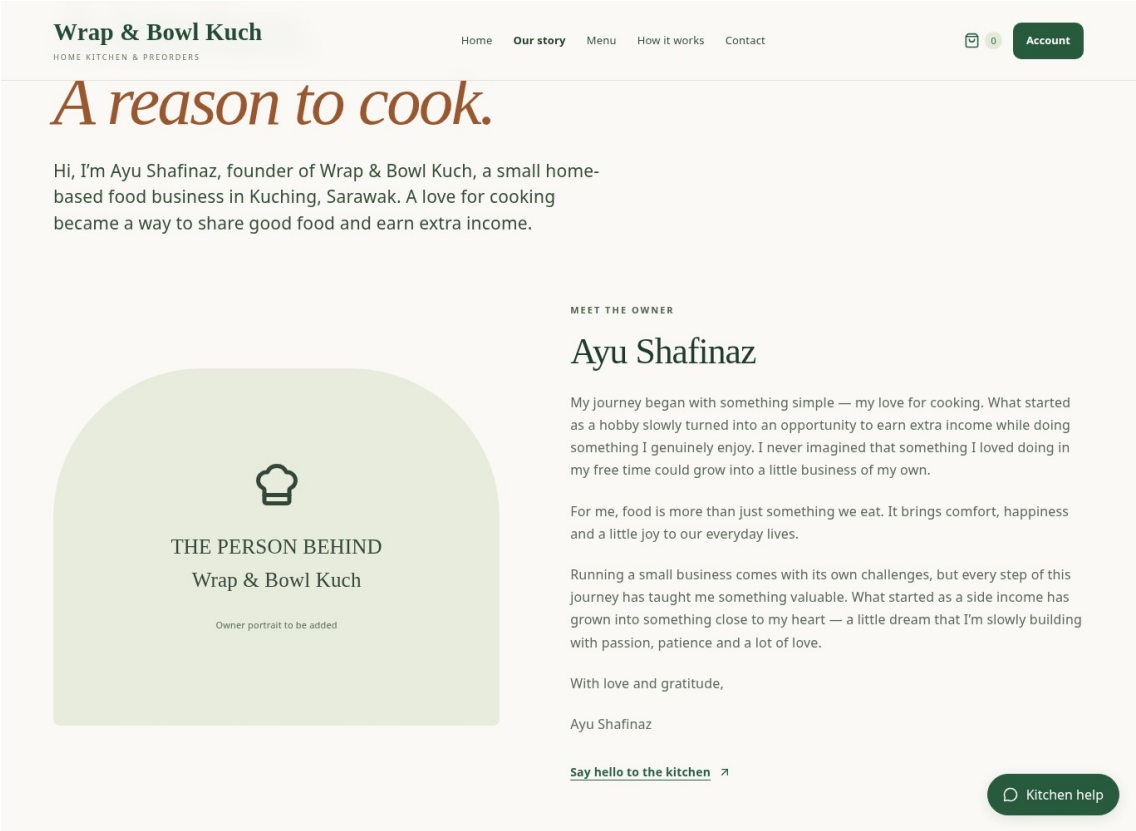



**Figure 4.2: Founder-story page using the client-supplied business introduction (local developer capture).**

The eight-menu catalogue contains Chicken Grilled Salad (RM13), Chicken Popcorn Salad (RM13), Eggsquisite Salad (RM12), Beef Sukiya Salad (RM18), Wanpaku Chicken Katsu (RM12), Wanpaku Chicken Grilled (RM12), Salmon Grilled Salad Bowl (RM28) and Sukiya Beef Rice Bowl (RM20). The first seven are offered Monday–Friday; the rice bowl is Monday only in the supplied menu. Categories group the catalogue into salad bowls, sandwiches and rice bowls. These settings remain editable by the owner.

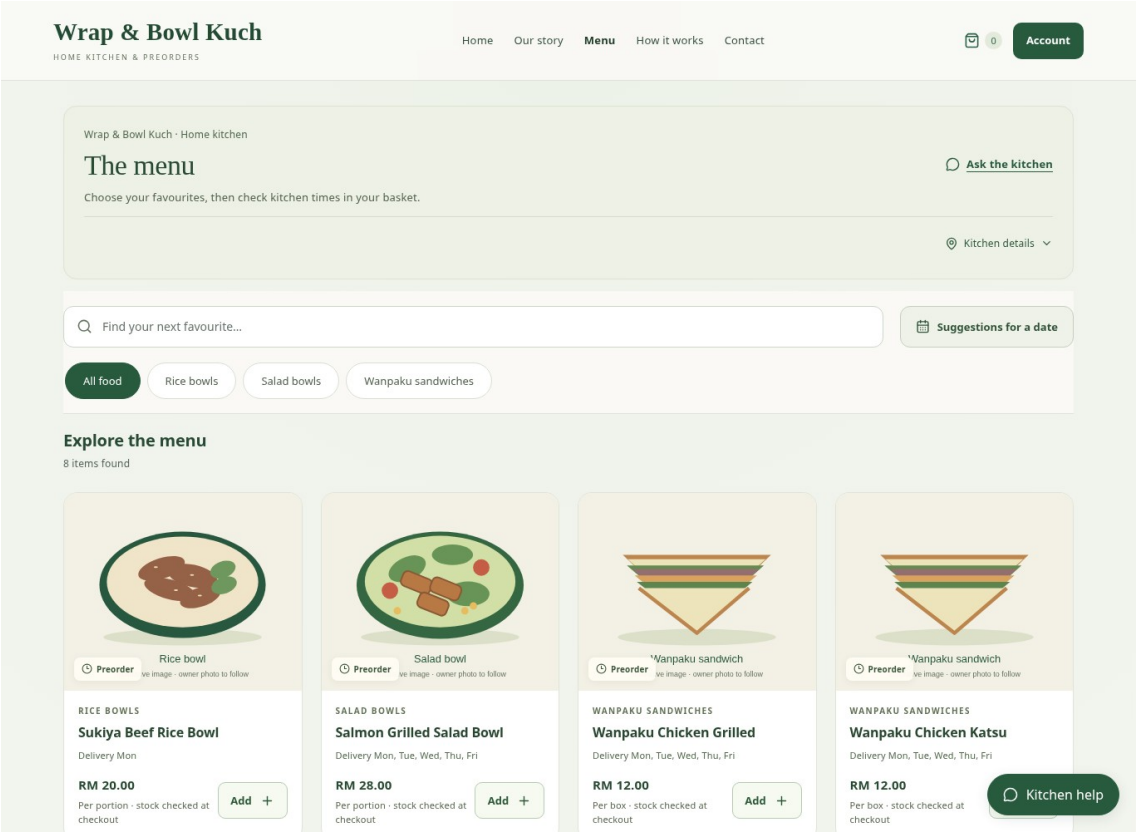



**Figure 4.3: Client menu, category filters and food cards with illustrative images.**

Search and category filters support direct browsing. A card presents its selling unit, price and availability information before the add action. A quantity already in the basket can be shown as a quantity control rather than another unexplained Add button. Stock-supported units are calculated from recorded ingredients and packaging and shared across menus; adding one menu can affect the supportable quantity of another. The displayed quantity is therefore an indication, while checkout checks the complete basket.

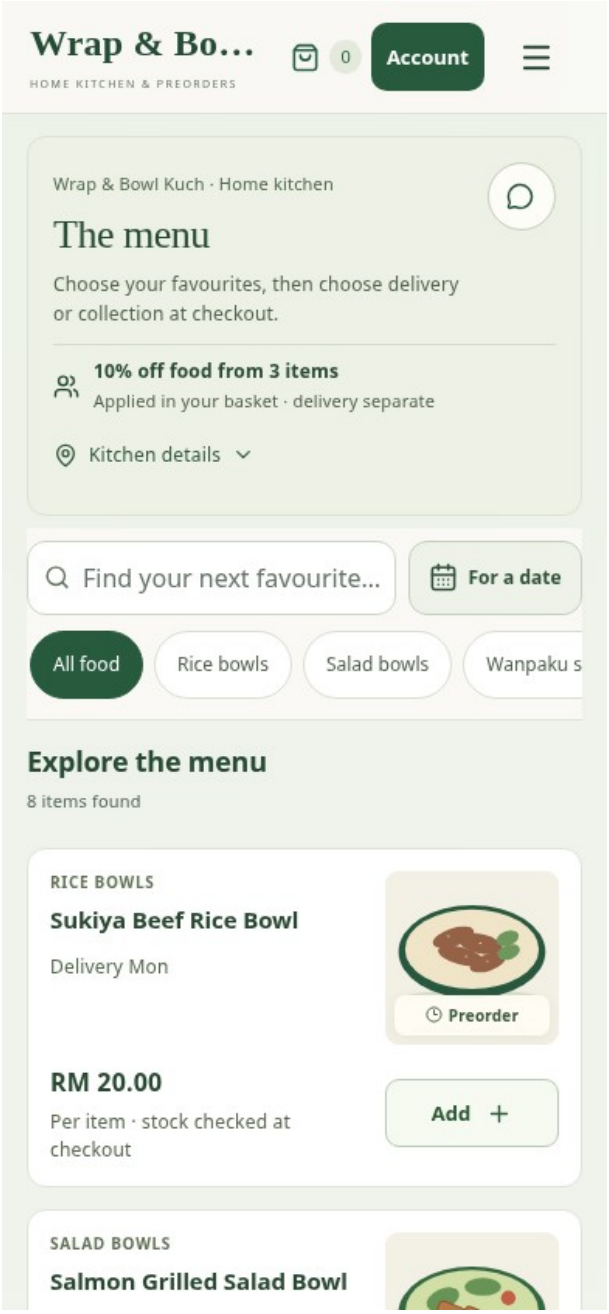



**Figure 4.4: Ordinary phone-viewport menu capture with compact cards and basket access.**

### 4.4.2 CUSTOMER ACCOUNT AND STAFF ACCESS

Public browsing can start without customer registration. Before placing an order or uploading private payment proof, the customer creates or signs into a passkey account. Registration collects a name and full phone number, then opens the browser/device passkey ceremony. Malaysian local and international spellings are normalised consistently. A prefix such as 011 alone is incomplete; a full valid number is required. The phone is stored as a contact route, not authenticated through SMS.

The device may use a fingerprint, face verification, screen-lock PIN or security key as supported by its passkey manager. The user does not need to memorise a website

username and password. That convenience depends on a compatible device/browser and access to a saved or synchronised passkey. Backup passkeys and an explicit sign-in path are important because a name and phone number alone cannot securely recover an account on a different device.

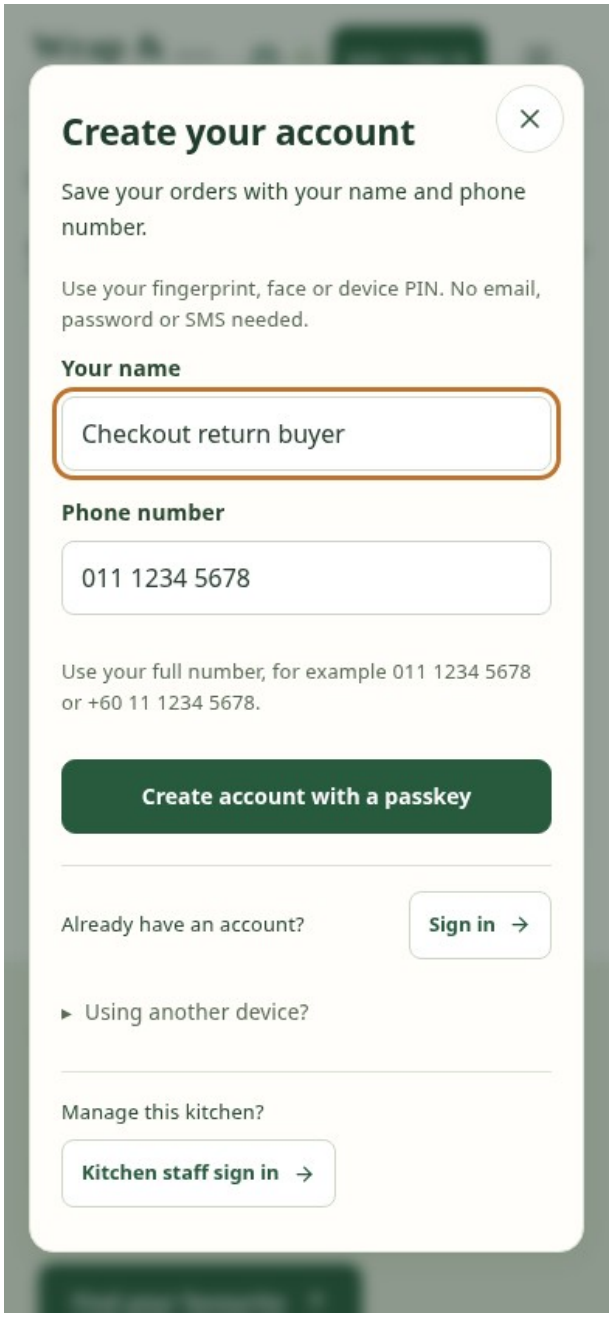



**Figure 4.5: Customer name/phone and passkey account dialog (ordinary phone capture).**

Staff access is intentionally separate. The kitchen signs in with its existing staff username/email and password. The dialog provides a clear return to customer account creation and blocks dismissal while an authentication write is pending. Staff password recovery depends on a configured recoverable staff email and working mail setup; it is not recovery for a phone-only customer passkey. Returning between dialogs preserves an unsent customer draft during that session, while closing the customer dialog clears it.

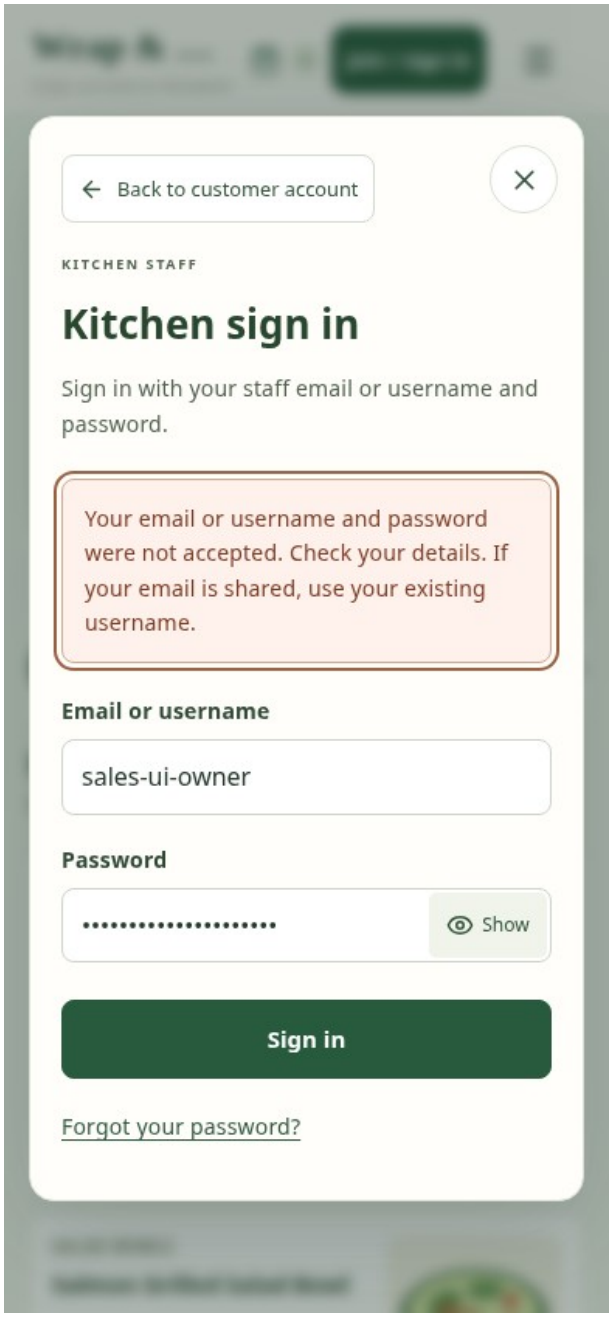



**Figure 4.6: Separate kitchen staff sign-in and explicit customer-account return control.**

The customer account page shows saved profile details and public passkey records. Adding a backup key or changing sensitive profile data requires recent native verification. Removing a key identifies the selected key and preserves access to the last remaining key. The application does not provide a staff shortcut that authenticates a customer merely by entering their phone number.

### 4.4.3 COMPLEMENTARY RECOMMENDATION SURFACES

Recommended for you provides a small, date-free discovery set using available purchase/category and popularity information. It supports browsing without forcing a form ahead of the menu. These suggestions do not constitute a promise that all items

can be accepted for a future date. The optional Suggestions for a date finder adds delivery date, total item count, category and an optional whole-order budget. A meal, box or pack counts as one selling unit; the count is not a number of diners.

The finder supports Allow a mix and One menu only. A mixed result is a complete proposed basket, retaining existing items where required. A one-menu result offers alternative baskets consisting of the requested quantity of a single menu; it does not mean that only one alternative should be displayed. Replacement asks for confirmation rather than silently removing existing basket items. Up to three alternatives are shown after eligibility and budget partitioning.

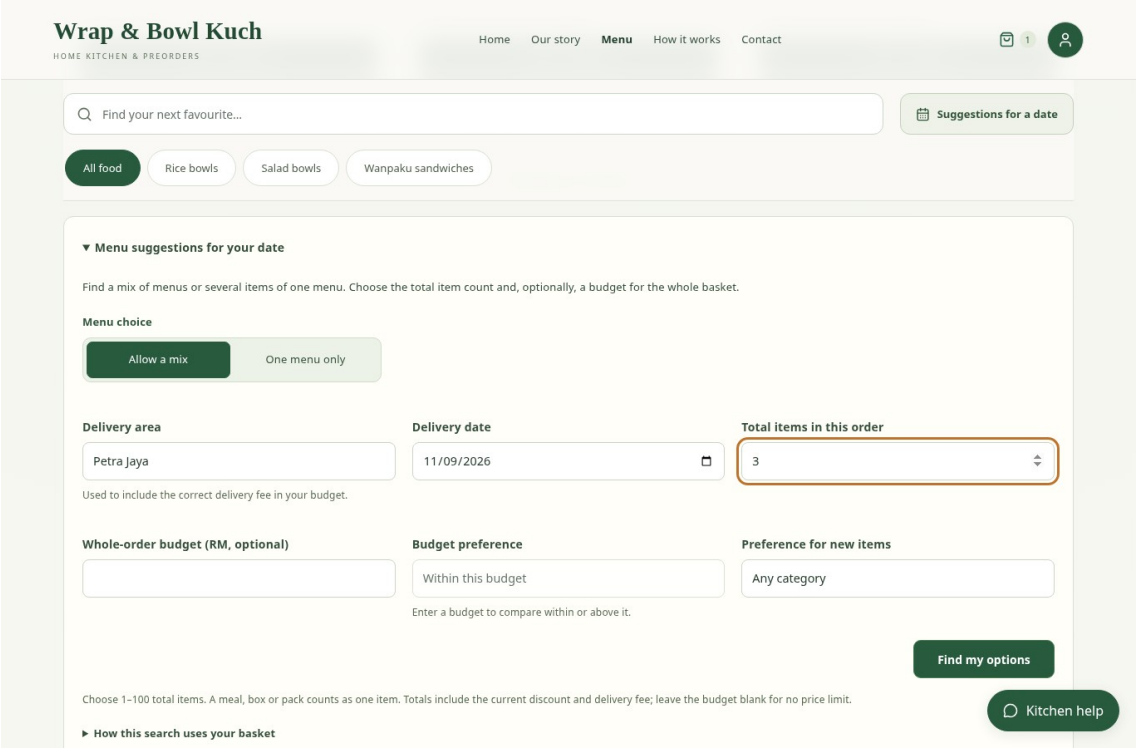



**Figure 4.7: Date/quantity/budget finder and its one-menu versus mixed choice.**

The current whole-order comparison includes food, the applicable bulk discount and the selected delivery fee. Leaving the budget blank means no price limit, so the within/above selector remains disabled until a budget is supplied. Within includes an exact match; Above is strictly greater. Where no above-budget basket is feasible, the interface explains that absence rather than inventing an expensive recommendation. Choosing times remains a checkout task; the finder checks the proposed date and complete basket without reserving an hour.

Personalisation is explainable through the available recorded history and category signals. It is not described as a trained dietary model or a proven causal conversion

optimiser. A recommendation also does not establish allergen suitability; the customer is directed to check dietary needs with the kitchen.

### 4.4.4 BASKET REVIEW

The basket focuses on food, quantities, the food subtotal and continuation to checkout. Item details contain pack contents and individual menu schedules, while the compact ordering-day summary identifies common offered days. It does not repeat the delivery-area and time forms. Continuing from this page is therefore valid before those checkout-only fields are completed.

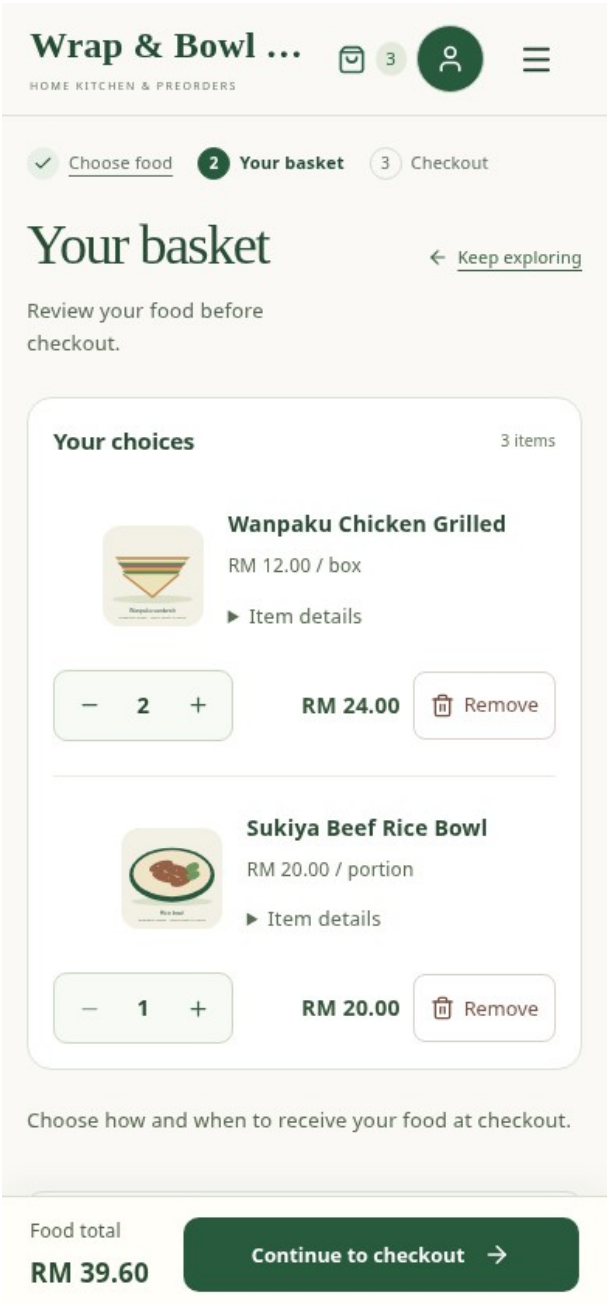



**Figure 4.8: Basket quantities and optional item details without duplicate delivery forms.**

A persistent basket shortcut is a navigation aid rather than a success notification. It can remain visible while the basket has items, provided it does not obscure content. Success notifications such as adding food or saving settings are transient; errors remain readable and actionable. The date finder keeps the basket shortcut in page flow while its larger form is open to avoid covering its controls.

### 4.4.5 DELIVERY, COLLECTION AND DATE SELECTION

Checkout first establishes fulfilment and time, then contact/address and payment. Delivery uses configured area choices and their fees. Collection appears only when the owner has enabled it and supplied a usable collection arrangement. A collection address on its own is not equivalent to an enabled service. The default client delivery period is 07:30–11:00 Malaysia time; owners can edit the supported minute-based bounds. Collection retains independently saved bounds.

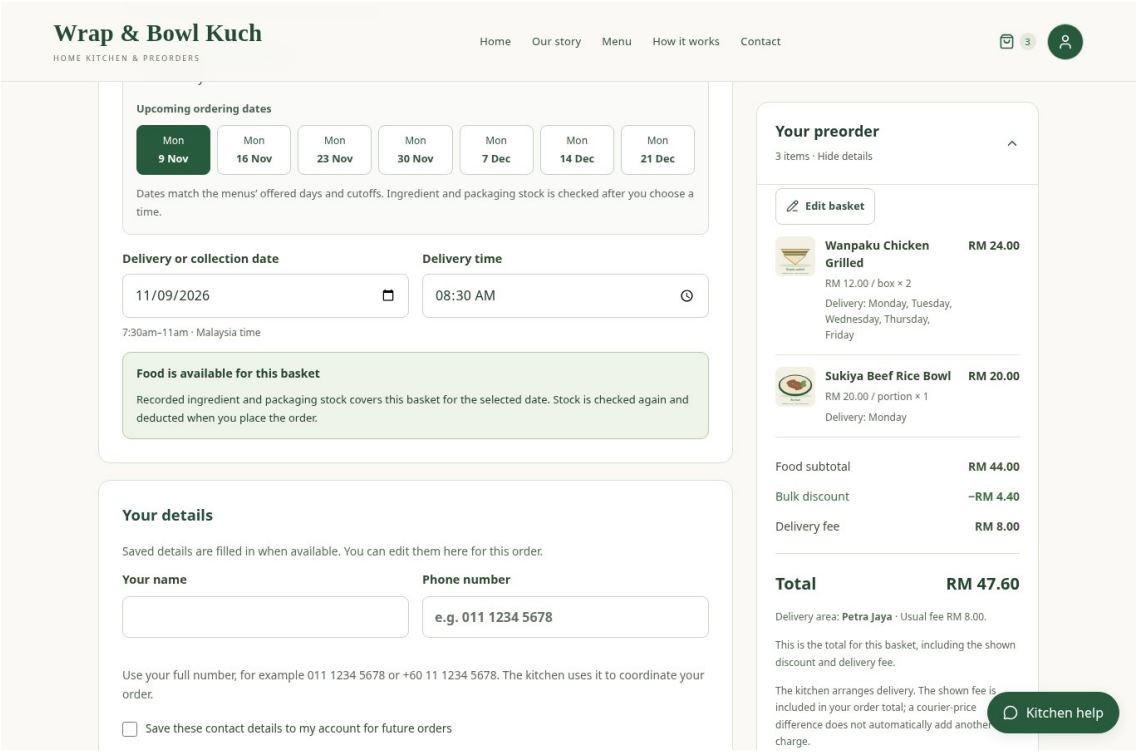



**Figure 4.9: Checkout offered-day guidance, upcoming date shortcuts and minute-based time selection.**

Offered weekdays, specific sale dates and cutoffs are combined across the basket. Upcoming date shortcuts make the common dates visible beside the date control rather than relying on customers to remember a note elsewhere. An invalid manual choice clears the old availability/quote before a new valid check. A date shortcut only satisfies the calendar rules; ingredient and packaging availability still need confirmation. New simple orders do not consult kitchen labour, oven bookings or cooking-task durations.

A complete delivery address and an order phone support handover. Saved name/phone values are prefilled; a customer can change this order's details without being forced to leave checkout. Updating the account profile is explicit and, where required, reauthenticated. The accepted order stores a contact snapshot so subsequent profile changes do not rewrite the delivery instructions on an earlier order.

### 4.4.6 QR/BANK PAYMENT AND RECEIPT UPLOAD

For delivery, the current options are enabled QR/bank-transfer evidence or cash. Collection requires a receiving QR and proof. The kitchen supplies approved receiving details in Settings. The customer checks the recipient and exact amount in their own banking application. The website does not call a bank API, automatically inspect a bank statement or initiate a transfer. QR payment is not restricted to the Maybank brand; the receiving code must be supported by the paying application.

Payment evidence is an image upload, not an unrestricted document attachment. The current implementation accepts a single-frame JPG, PNG or WebP image up to 4 MB, decodes and re-encodes it, removes original-file metadata and limits the stored image size. PDF payment receipts are not an implemented checkout upload format. Customers must use an image/export supported by the form rather than assuming every PDF can be uploaded.

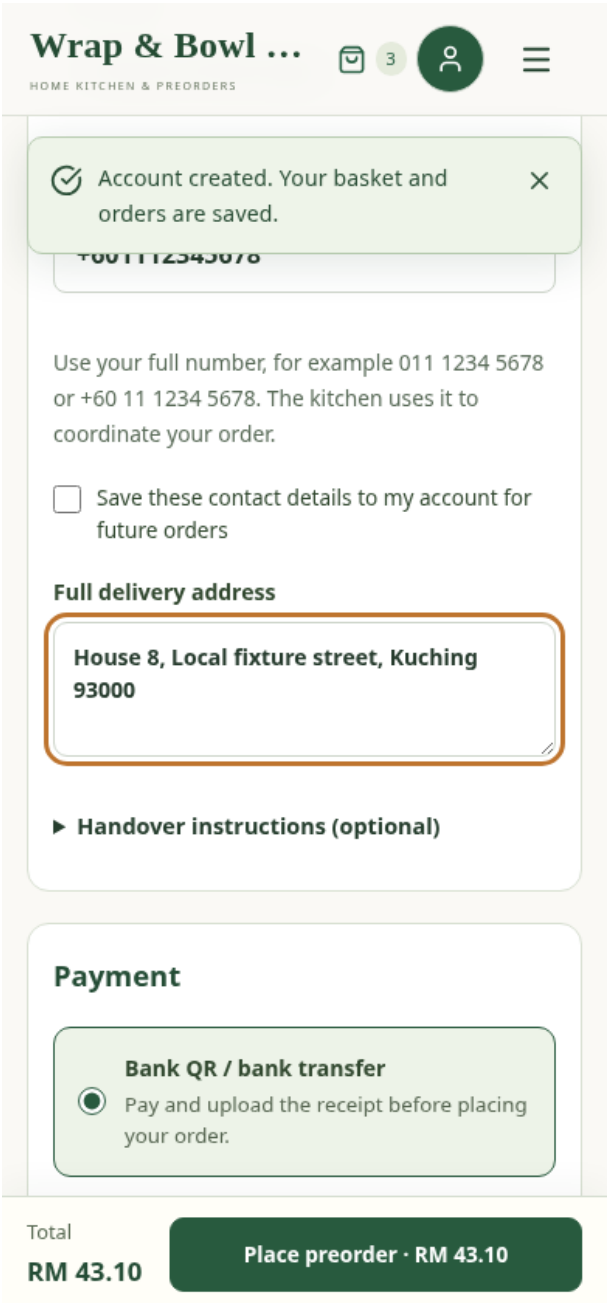



**Figure 4.10: Checkout receipt-evidence state and enabled placement controls (synthetic local proof).**

The image belongs to the authenticated account and is served from a private, no-store endpoint. Its selected price/fulfilment scope is checked again at placement. An uploaded receipt remains unverified and the order remains unpaid until staff compare it with bank records. If the customer has already transferred money and the amount changes, the interface directs them to review/contact the kitchen rather than paying again automatically. A file chooser cancellation preserves the already uploaded receipt.

### 4.4.7 MY ORDERS AND KITCHEN HELP

My orders shows accepted food, date/time, fulfilment information, payment state and whether the food has been received. Customer history is restricted to that account.

Changes use eligible audited operations; they do not directly overwrite price or stock from the browser. Where reviews are available, they concern food from an eligible received order. A customer cannot infer that payment is verified merely because the food has been marked received.

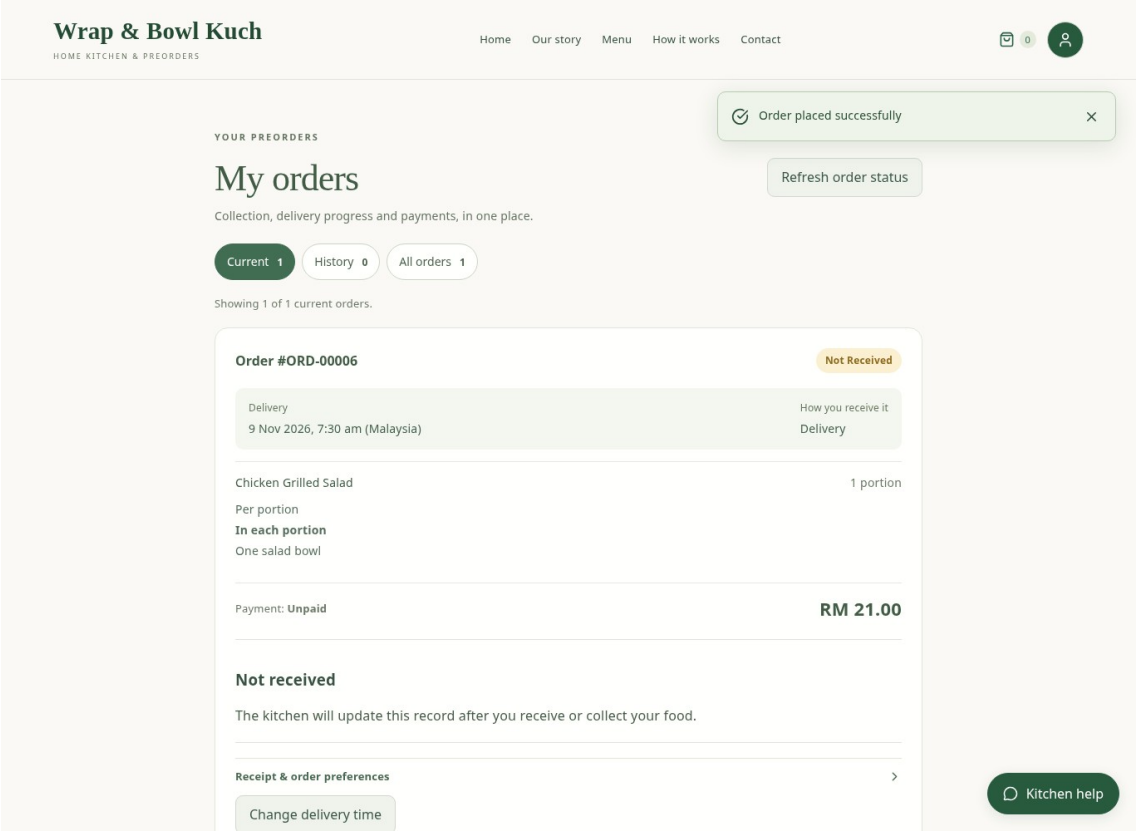



**Figure 4.11: Customer order confirmation with separate food-receipt and payment states.**

Kitchen help provides a compact route to supported questions, menu suggestions, order guidance and contact information. It should be treated as an assistant to the actual forms, not a source of authority to fabricate an order or promised delivery. External language-service availability can fail; core ordering remains subject to the same authoritative API. Clear close/back controls and readable phone layouts prevent the help panel from trapping the user.

## 4.5 OWNER INTERFACES

### 4.5.1 MENUS, CATEGORIES AND PHOTOS

Staff can create and edit menus, manage their categories, adjust offered days/date windows and maintain selling-unit contents. Pause/archive actions distinguish stopping future orders from deleting an unused record. A menu linked to history cannot simply be removed without respecting the server's protection of those relationships. Menu

names and current prices can change while accepted order lines retain their earlier values.

Menus support a public image URL and local-device image upload. The server applies image validation and sanitisation, so the feature is more than saving a file path from the owner's computer. The eight client menus initially use clearly illustrative salad, sandwich and rice images; the owner can replace these with client-provided photographs. Ingredient quantities and standard packaging are editable estimates that must be checked against actual production before accepting real business decisions based on them.

### 4.5.2 INGREDIENT DIRECTORY AND BATCH INVENTORY

Raw materials record a name, inventory unit, optional cost assumption and optional storage guidance. The batch form records the actual received lot, quantity, receipt date, storage location and evidence supporting its planning deadline. A source such as wet market helps describe provenance but does not generate a shelf life automatically. The FoodSafety.gov cold-storage chart is a reference link; selecting the link does not validate a batch's conditions or fill dates.

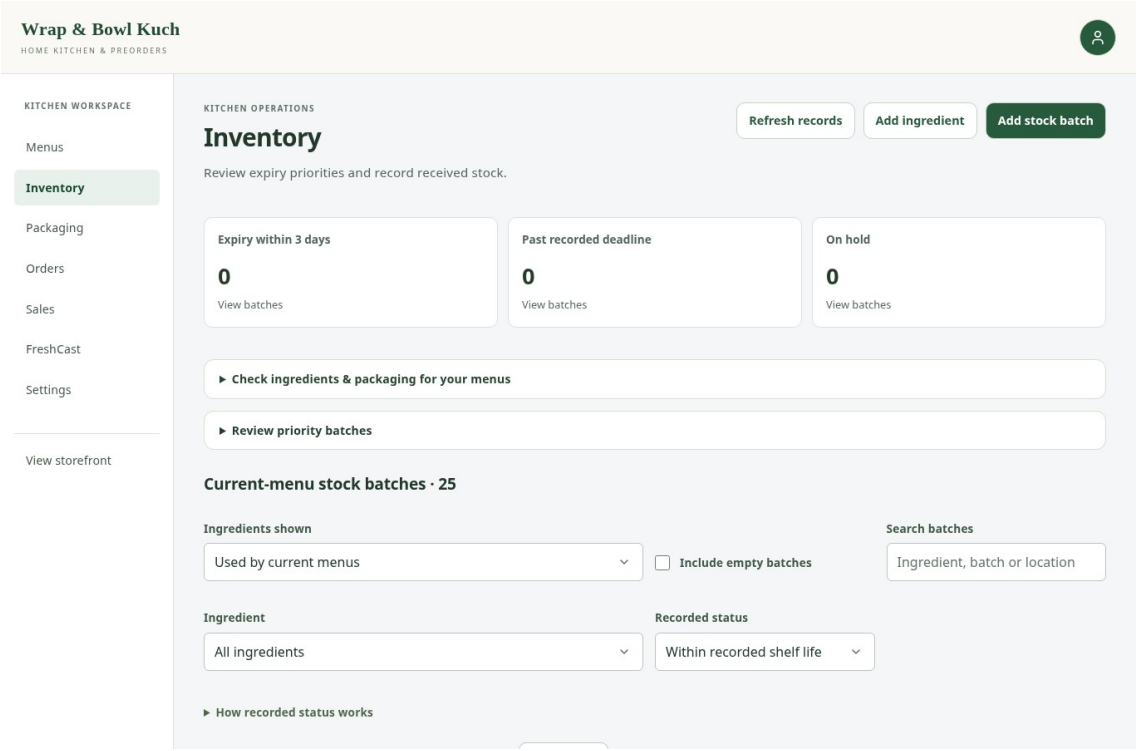



**Figure 4.12: Inventory overview with ingredient filters, recorded status and batch actions.**

For an unlabelled ingredient, the owner can use receipt-based documented storage life without first creating a compulsory guidance preset. If a matching preset has been saved for the ingredient/storage type, it can be selected to populate the unsaved batch

form. The copied evidence remains part of that batch and is not retrospectively changed when the preset is edited. This balances repeat-entry convenience with a batch-specific record.

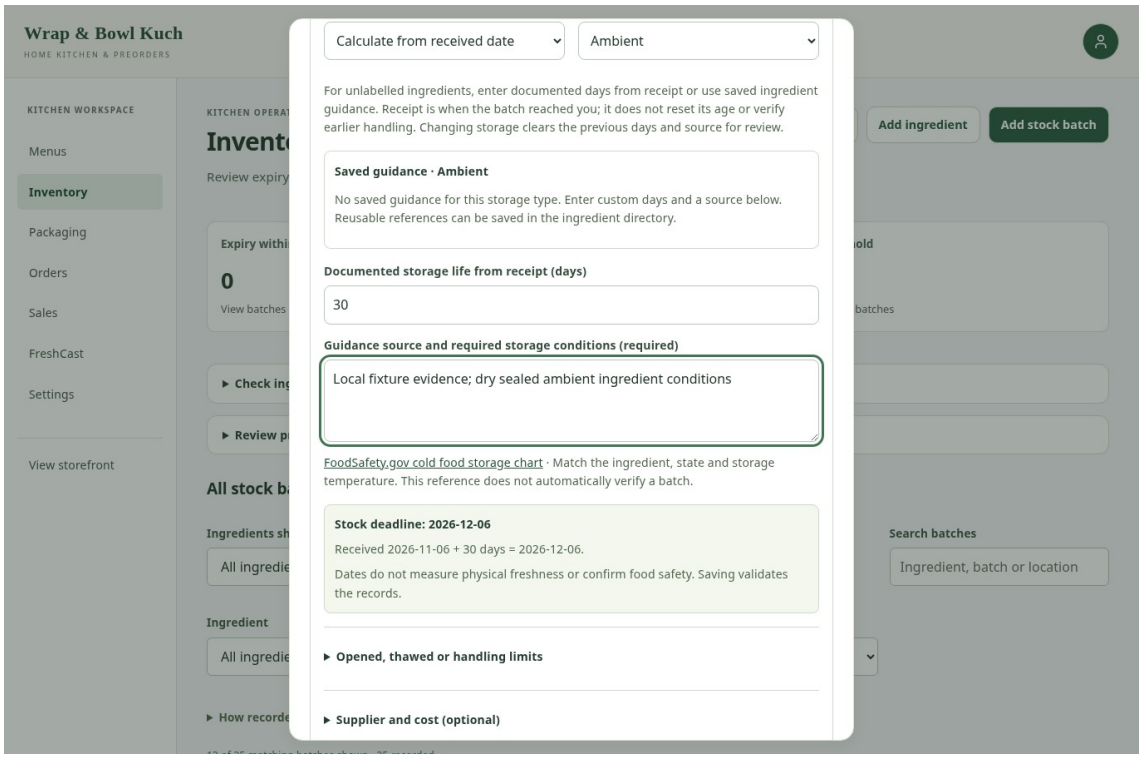



**Figure 4.13: Batch evidence form with conditional date inputs and a planning-deadline preview.**

Printed expiry/use-by is a supplier deadline; manufacture date is when the food was made; received date is when the kitchen obtained that batch. They are different events. Best-before is a quality date, although this system conservatively uses the recorded deadline as an allocation limit. Manufacture-plus-life uses the manufacture anchor, while receipt-plus-life uses receipt. An earlier original printed deadline, opening limit or thawing limit can cap a longer calculated life. This explains why entering 30 days may still produce a three-day effective deadline if an earlier recorded cap remains.

Inventory adjustments require an explanation where relevant and use a saved edit/version check. A stale form must not restore stock that an order has already consumed. The interface provides previews and field errors while the server remains authoritative. Ordinary low-impact quantity edits are distinct from explicit wastage, which also records cost evidence.

### 4.5.3 STOCK-SUPPORTED MENU QUANTITY AND TARGET CHECK

Stock-supported quantity is derived from current usable ingredient batches and assigned packaging. For each required material, usable stock is divided by the per-

selling-unit recipe quantity and rounded down; the limiting ingredient or packaging item determines the menu quantity. Menus share stock, so individual displayed quantities cannot be added together as independent guarantees. Checkout combines all recipe requirements before acceptance.

The inventory target check asks a different planning question: is recorded stock enough for the same requested number of units of every menu offered on a chosen date? With 3 units and 8 offered menus, it checks 24 units in total. It is not a daily customer-order report and does not create new stock, reserve orders or enter procurement automatically. Results show target, available quantity, shortfall and the included menus. The downloadable worksheet leaves physical receipt/evidence fields for actual owner entry.

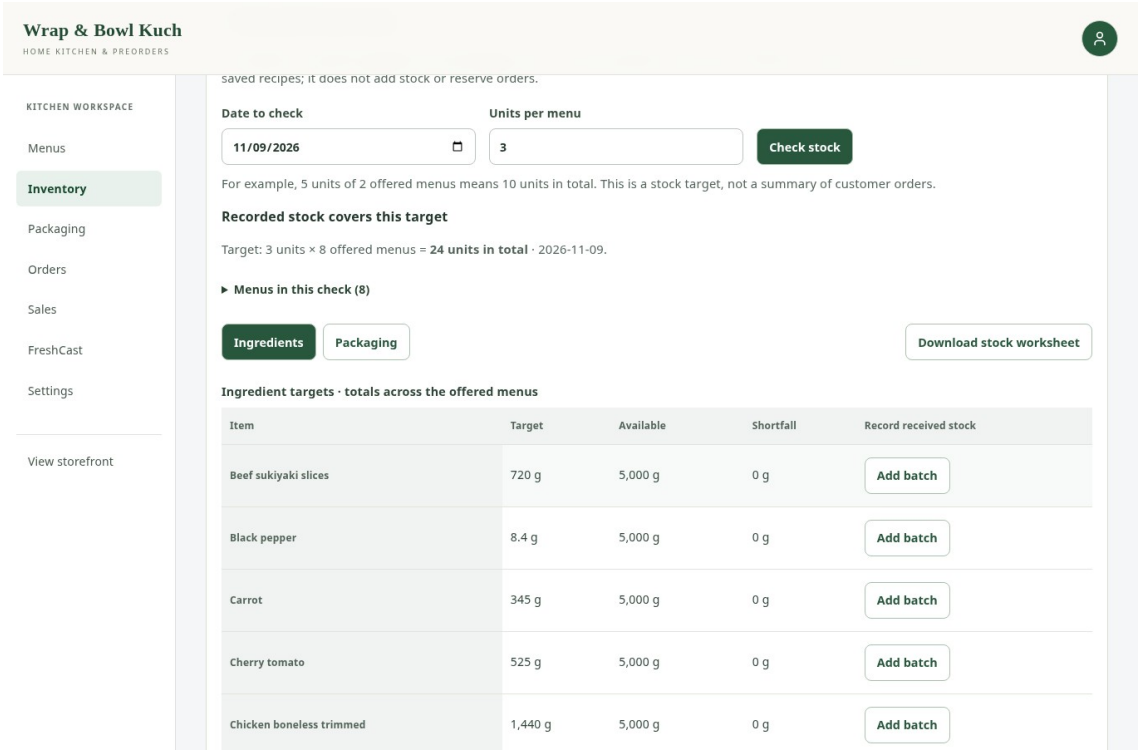



**Figure 4.14: Aligned stock-target controls and totals across the offered menus.**

### 4.5.4 PACKAGING STOCK AND MENU ASSIGNMENTS

Packaging has its own stock page, low-stock threshold, current pieces, unit cost and movement history. Receiving pieces and removing damaged/used pieces are deliberate operations. Removal first shows the proposed deduction and remaining balance. Menu packaging assigns containers, lids, cutlery, dressing cups, sheets and labels to one selling unit. This mapping is multiplied at placement, not manually re-entered for every customer order.

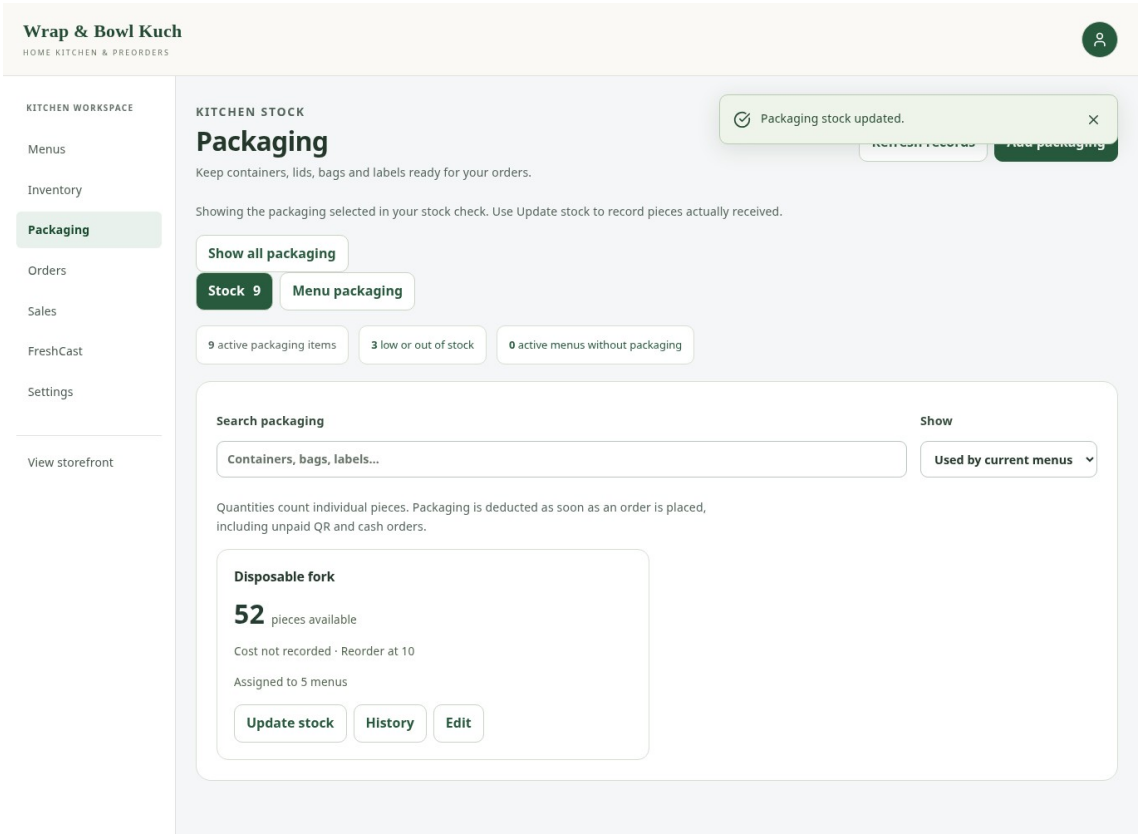



**Figure 4.15: Packaging stock balance and actions for current menu supplies.**

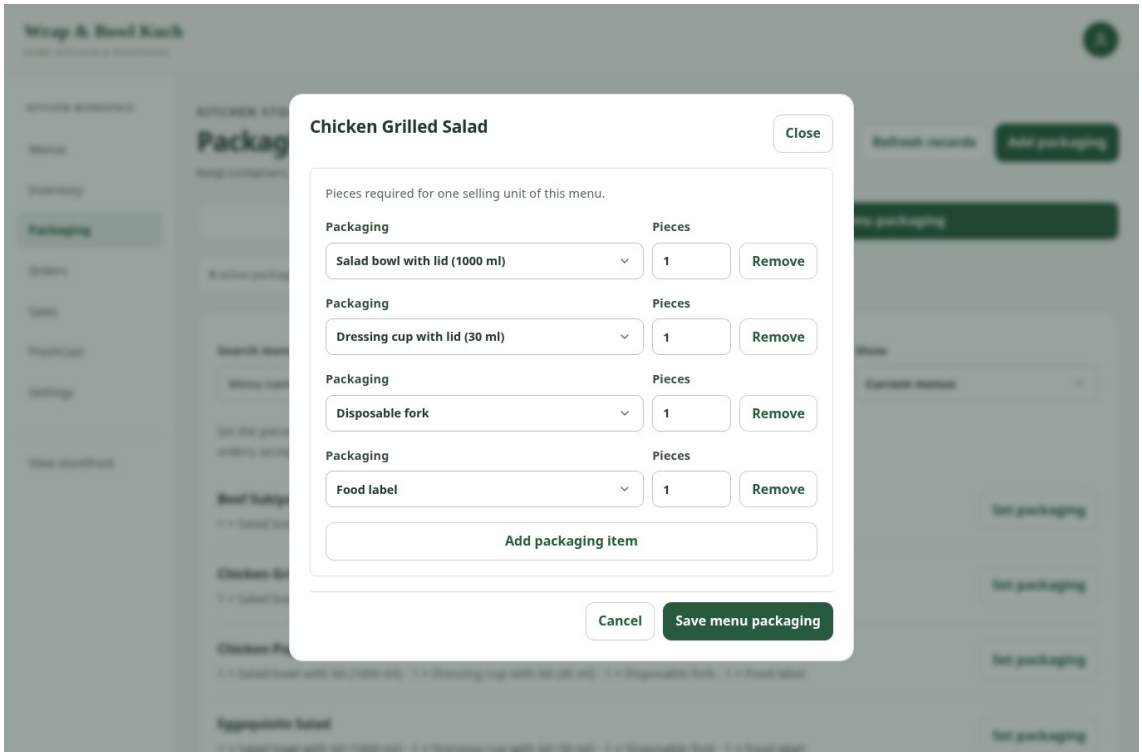



**Figure 4.16: Packaging-assignment dialog with per-selling-unit quantities and aligned controls.**

A salad bowl example uses its bowl/lid, dressing cup, fork and label. A Wanpaku sandwich uses a box, greaseproof sheet and label. These starting arrangements reflect the authorised project setup, not a confirmed packaging specification or supplier price

list. Known assigned packaging costs are included in current menu-cost estimates. Missing prices remain unknown; an explicit zero means genuinely no cost, not a hidden default.

### 4.5.5 ORDERS, PAYMENTS AND ACTUAL RECEIPT

The current owner queue groups Not received, Received, Cancelled and All orders, with date sorting/filtering and selected-order actions. The simple display is intentionally narrower than the historical database status enum. New orders do not need processing, cooking or courier-dispatch confirmations before the owner can record an eligible actual receipt. Bulk receipt actions identify the selected orders and recheck their current versions. A correction records the actor and before/after evidence.

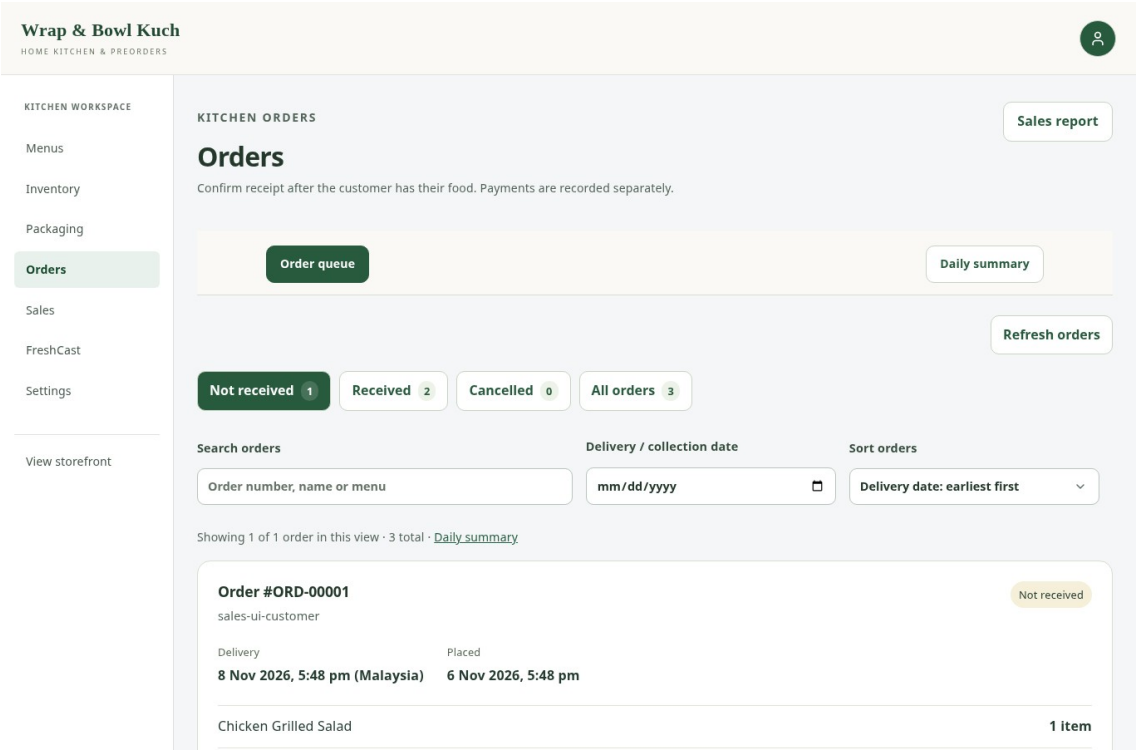



**Figure 4.17: Owner order queue with receipt groups, filters and deliberate per-order actions.**

Payment details open in a focused dialog. The owner sees the full expected amount, existing receipt/refund events and the customer proof link. A payment reference is a supplied bank/cash reference, not necessarily a database ID. The form makes clear that the owner is recording money verified outside the website. A refund action confirms an already completed external refund and does not send money. This protects the distinction between operational status and financial evidence.

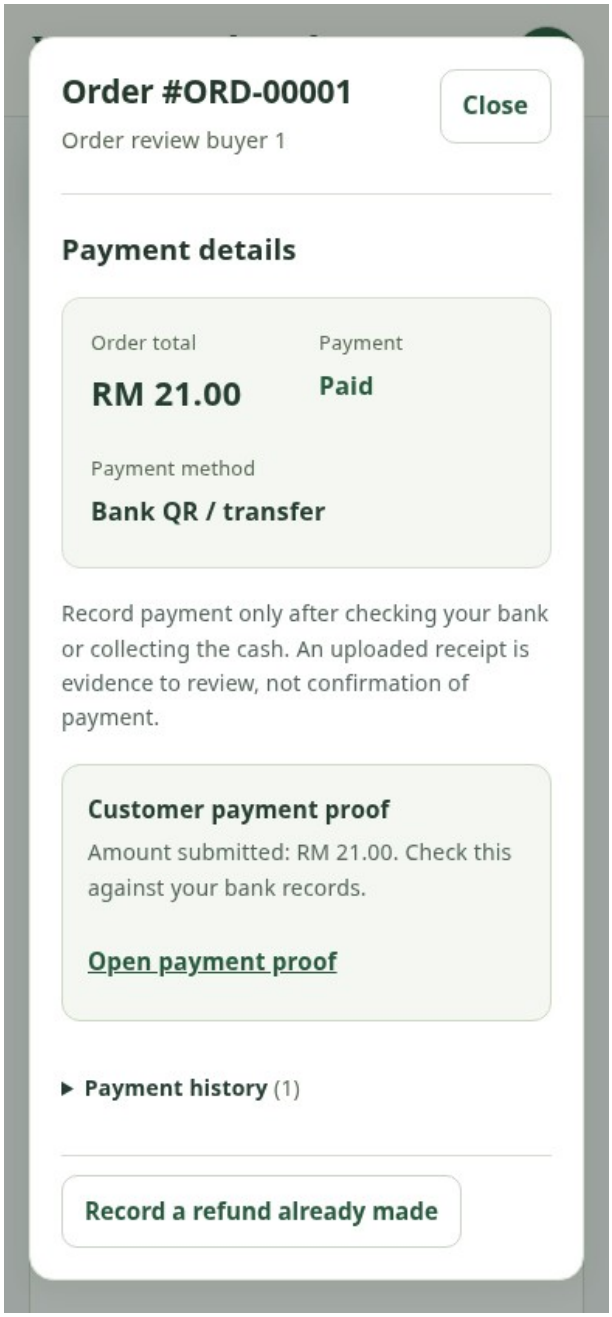



**Figure 4.18: Payment-review dialog showing proof, amount and payment history.**

Cancellation and unused-stock returns are separate from recording receipt. In the simple workflow, placement has already consumed recorded quantities for commitment purposes. An eligible unused cancellation can return allocations once; consumed/otherwise ineligible stock must not be restored blindly. The cancellation path and replay protections are verified by integration tests. No historical real customer/payment record was rewritten for this report.

### 4.5.6 DAILY ORDER SUMMARY AND PRINTING

Daily summary is located with Orders because it supports packing and handover: which accepted menus, in what quantities, for which date. It is not the same as a sales

dashboard. The report can include unpaid open orders according to its documented scope, while cancelled/generated analytical records are excluded as specified. Undated historical records are reported as such instead of assigning their placement date as an invented handover date.

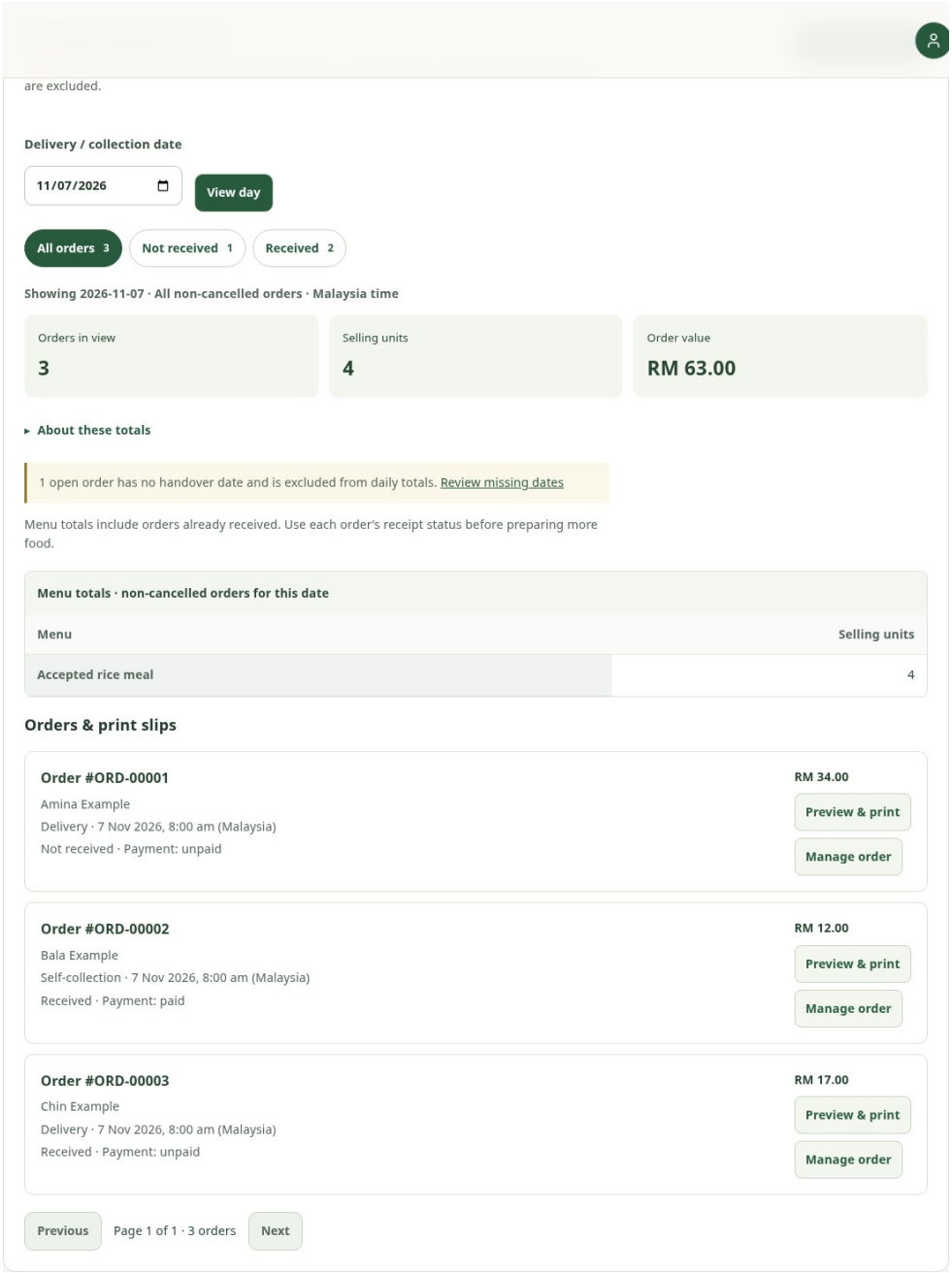
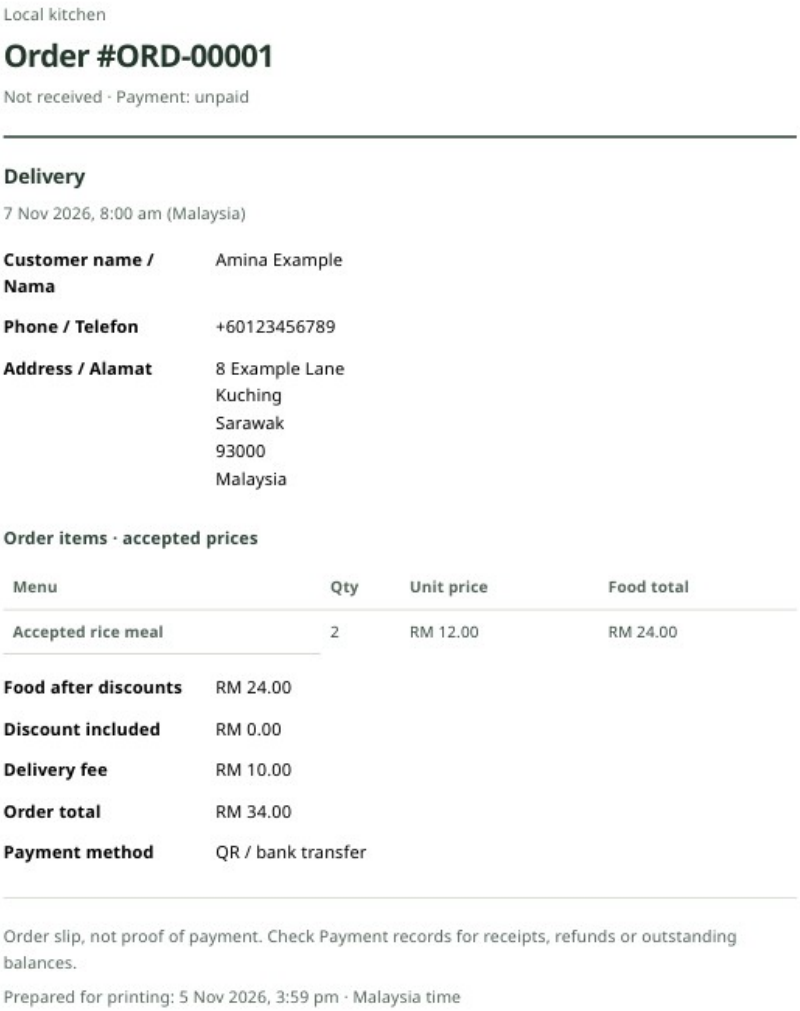



**Figure 4.19: Daily order/menu-unit summary for kitchen work.**

Each order can open a print preview containing the accepted customer name, full address where relevant, phone number, menu lines and agreed handover details. Collection slips use the collection snapshot rather than an old delivery address. The print action is explicit and read-only; a new tab uses safe browser attributes. An owner can use the browser's Print / Save PDF function without the website marking the food received.



Local kitchen

Order #ORD-00001

Not received · Payment: unpaid

Delivery

7 Nov 2026, 8:00 am (Malaysia)

| | |
|---|---|
| **Customer name / Nama** | Amina Example |
| **Phone / Telefon** | +60123456789 |
| **Address / Alamat** | 8 Example Lane<br>Kuching<br>Sarawak<br>93000<br>Malaysia |

Order items · accepted prices

| Menu | Qty | Unit price | Food total |
|---|---|---|---|
| Accepted rice meal | 2 | RM 12.00 | RM 24.00 |

| | |
|---|---|
| **Food after discounts** | RM 24.00 |
| **Discount included** | RM 0.00 |
| **Delivery fee** | RM 10.00 |
| **Order total** | RM 34.00 |
| **Payment method** | QR / bank transfer |

Order slip, not proof of payment. Check Payment records for receipts, refunds or outstanding balances.

Prepared for printing: 5 Nov 2026, 3:59 pm · Malaysia time

**Figure 4.20: Order-slip print preview; an A4 preview is intentionally different from a web-page viewport.**

### 4.5.7 SALES, COSTS, CONTRIBUTION AND WASTAGE

Sales overview uses the selected handover period and includes eligible paid-and-received orders. Food value, discount, net food sales and delivery income remain separate. Daily operational order value is not interchangeable with realised sales. CSV export respects report scope, bounds the date range/row count and escapes spreadsheet-formula text. Unknown dates or incomplete evidence are disclosed rather than silently counted as zero.

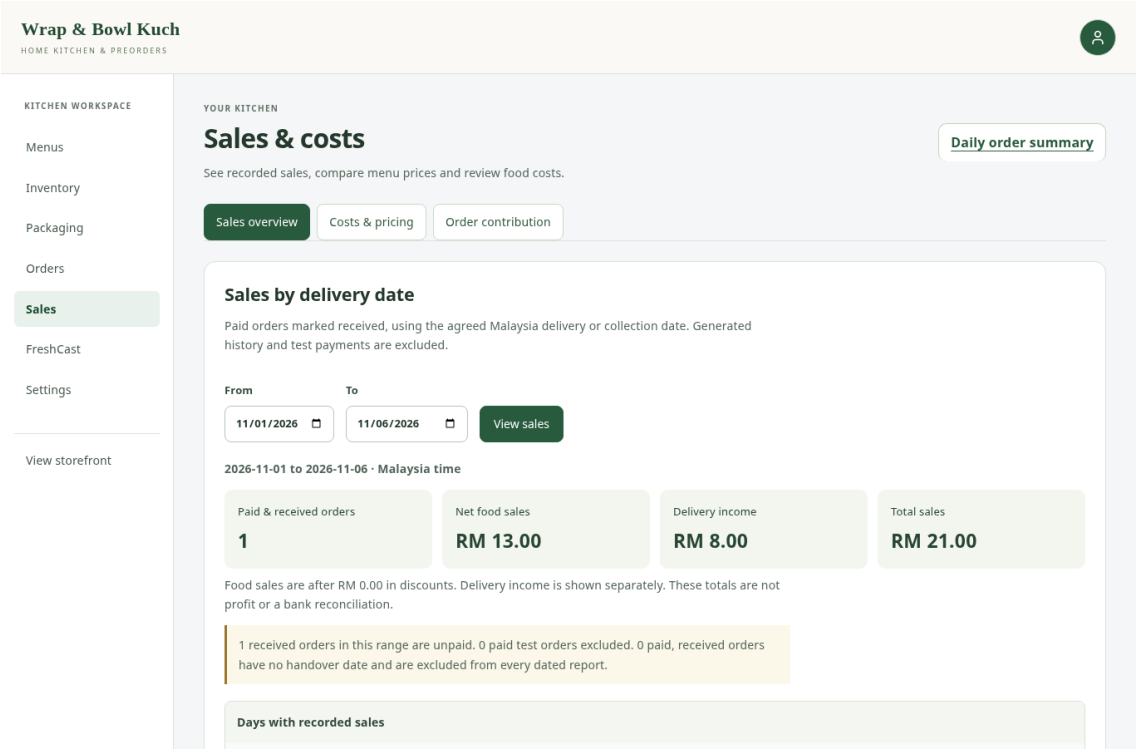



**Figure 4.21: Sales overview with date filtering and separate food/delivery values.**

Costs & pricing estimates recipe ingredients and assigned packaging at entered unit costs. A scenario compares a proposed selling price, quantity, discount and cost increase without saving a live promotion. Order contribution uses allocated ingredient batch costs and accepted packaging evidence. Its margin is a food-contribution margin, not net profit: labour, overhead, courier charges and payment fees can remain outside that food calculation. An unavailable contribution is evidence of missing required cost/payment records, not a numerical loss or a software-generated profit.

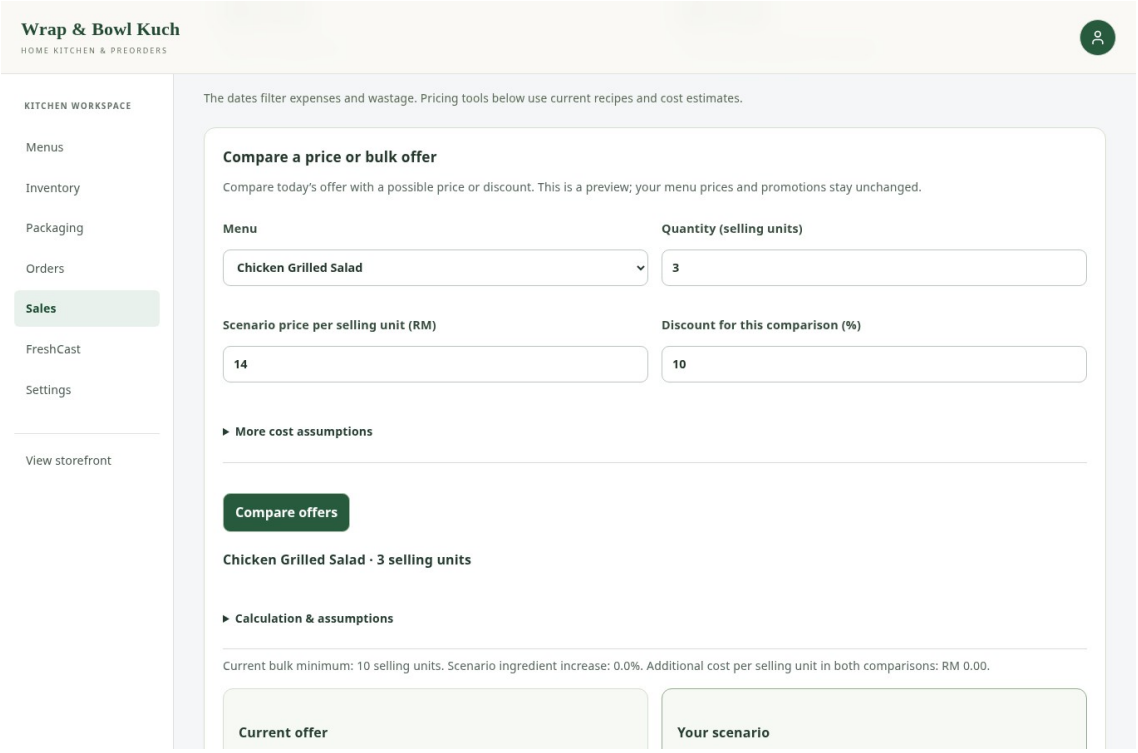



**Figure 4.22: Read-only price/bulk-offer scenario and progressively disclosed cost records.**

Ingredient cost entry supports understandable unit conversion. For example, RM12/kg is RM0.012/g; using RM12 as the per-gram cost would distort every recipe estimate. Packaging costs can be entered in pieces. Forcing an unknown value to zero would create misleading margins, so the application leaves that estimate unavailable until the necessary costs are supplied.

Operating expenses record overhead such as utilities, courier fees, labour and marketing. They should not duplicate ingredient/packaging purchases already counted as recipe costs. Wastage is relevant because discarded food changes both available inventory and the evidence behind purchasing decisions. The form reviews the batch, quantity, reason and remaining stock before deduction, then records the estimated cost at that time. It must not be used for food already deducted for an accepted order. Expense and wastage ledgers support review; they are not a complete general ledger or audited profit-and-loss statement.

### 4.5.8 SETTINGS AND SETUP CHECKS

Settings separates kitchen profile, business story, promotion, collection/delivery, payments, reminders and setup checks into manageable sections. Each save applies to its section, preserving unrelated unsaved drafts. Short delivery-area names are distinct from a descriptive service-area sentence and from a physical collection address. Listed

areas are customer choices and fee keys; they do not geocode or verify that an entered address lies inside a geographical boundary.

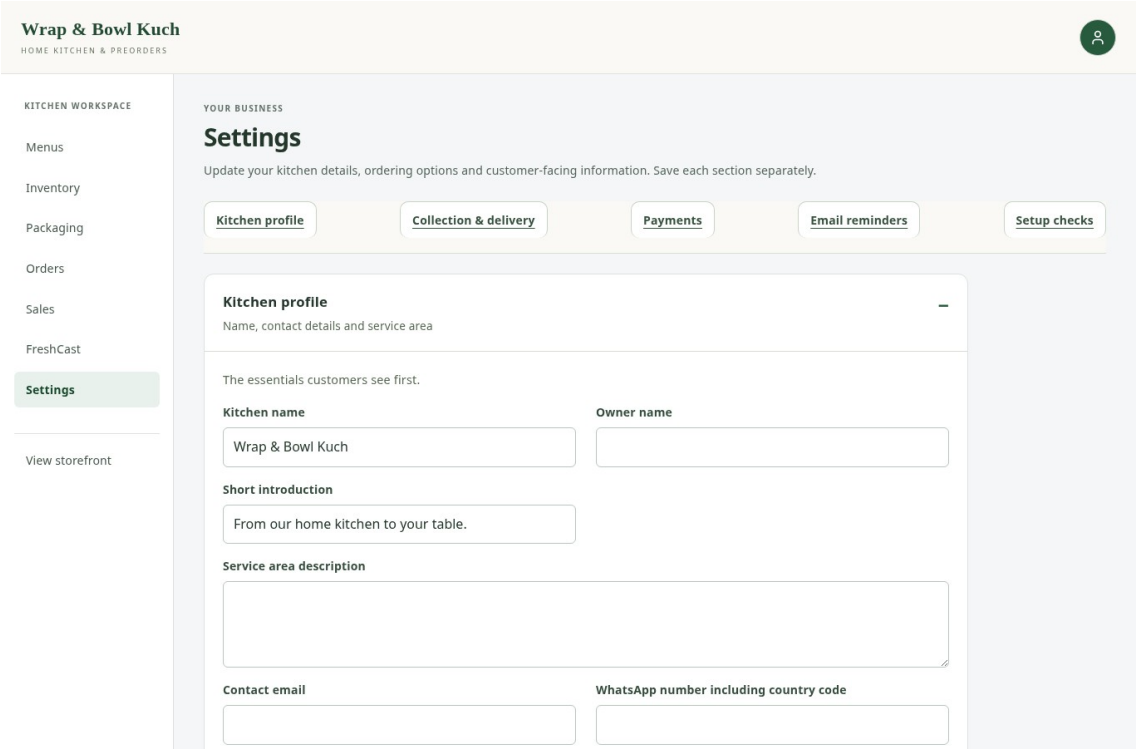



**Figure 4.23: Section-based kitchen settings with explicit save scope.**

Setup checks review recorded identity, recipe and cost requirements and identify missing information. They are not certificates of food safety or proof that an email or payment provider is working. The UI uses compact counts and optional detail rather than dense warnings. Technical website revision information remains under a less prominent support section so it is available for troubleshooting without dominating kitchen tasks.

## 4.6 CORE CALCULATIONS AND TRANSACTION RULES

### 4.6.1 EFFECTIVE DEADLINE AND DATE-BASED STATUS

The base deadline is either a printed date, manufacture date plus documented shelf-life days, or received date plus documented storage-life days. The effective planning deadline is the earliest valid applicable base/printed/opened/thawed limit. Date inputs cannot extend an earlier recorded deadline merely by supplying a longer duration. Held or expired lots are excluded from cooking stock, and uncertain handling can hold a batch even when its calculated date is in the future.

**Table 4.3: Illustrative date calculation using explicit evidence**

| Evidence | Example value | Consequence |
|---|---|---|
| Received date | 1 November 2026 | Receipt-based storage clock begins |
| Documented storage life | 30 days | Base planning deadline is 1 December 2026 |
| Original printed deadline | 4 November 2026 | Earlier supplier cap wins |
| Opened limit, if recorded | Opened 2 November; 1 documented day | Earlier handling deadline becomes 3 November |
| Handling breach | Known breach / held | Batch is excluded regardless of a future date |

The public average describes the proportion of each recorded storage window remaining. A printed-date batch uses receipt-to-expiry; a manufacture-based batch uses manufacture-to-expiry. Each non-empty received batch has equal weight. Expired and held batches contribute zero, unknown records are excluded and reported separately, and the score is bounded from 0 to 100. These rules are deterministic, not random. They do not measure microorganisms, temperature exposure or actual remaining edible life.

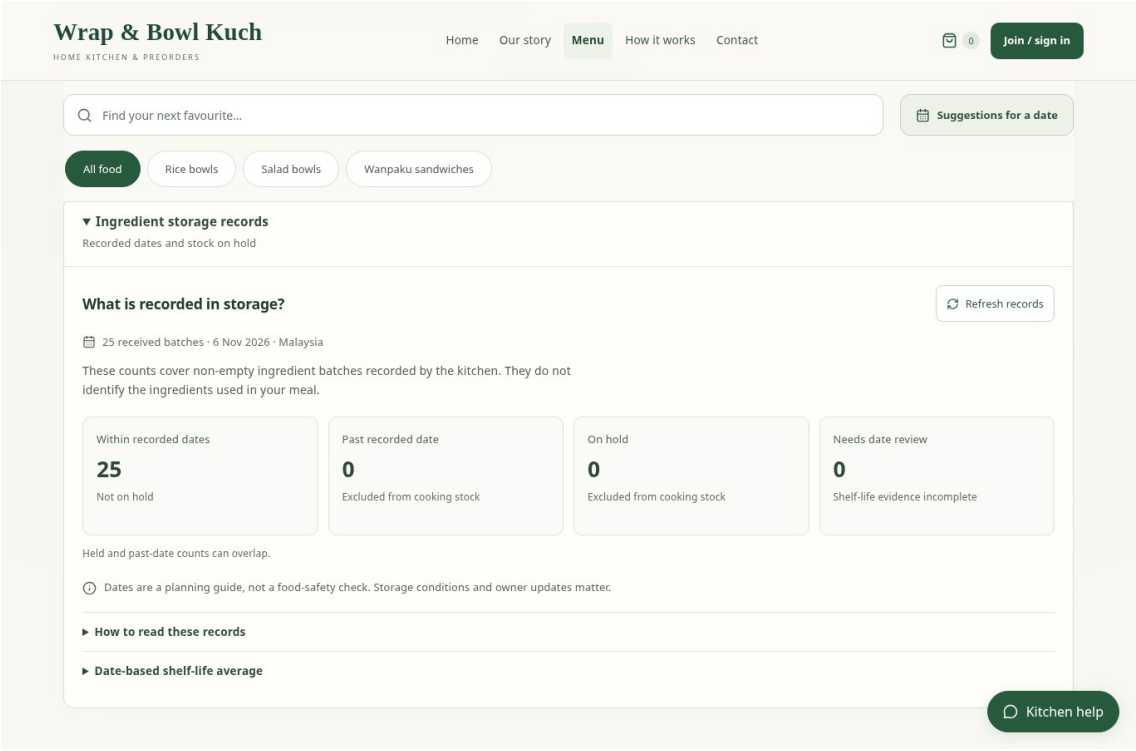



**Figure 4.24: Compact public recorded-stock counts with an optional date-based average explanation.**

### 4.6.2 INGREDIENT AND PACKAGING ALLOCATION

For ingredient r, total required quantity is the sum, across basket lines, of ordered units multiplied by the accepted recipe quantity for r. Eligible stock must already have been received, be positive, not held and cover the relevant date under the recorded deadline. Planned future purchases are not stock already available. FEFO consumes the eligible batches with the earliest expiry first, recording the quantities and batch-cost evidence.

Packaging requirement is the sum of ordered units multiplied by required pieces for each packaging type. Ingredients and packaging are rechecked and written in the same placement transaction. If any required item is short or a later write fails, the transaction rolls back. Repeating an accepted request identifier returns the same accepted result rather than deducting again. The order line's accepted name, price, recipe and packaging survive later catalogue changes.

### 4.6.3 PRICING, BULK PROMOTION AND BUDGET COMPARISON

Food subtotal is the sum of unit price multiplied by quantity. If the saved bulk threshold is met, the configured percentage reduces food subtotal. Delivery is then calculated under the selected area/fulfilment and any saved free-delivery policy. Collection has no delivery fee. The accepted fee/discount is snapshotted and a later settings edit affects new quotes, not earlier accepted terms.

For example, 10 items at RM12 with a 10% food discount and RM5 delivery produce RM120 food, RM12 discount and RM113 payable. This is an arithmetic illustration, not an advertised client promotion. The finder's whole-order budget comparison uses the same relevant pricing rules, preventing a recommendation from treating a per-item price as a complete order total. An exact RM100 total belongs within RM100, while RM100.01 belongs above it.

## 4.7 FRESHCAST AND SUPPORTING DECISION TOOLS

FreshCast remains a separate staff decision workspace. It distinguishes local database-based analysis, the external Genpact/CatBoost benchmark and an illustrative model-free walkthrough. Their data sources and meanings are not interchangeable. Local order history can describe the project scenario, but the fictional refreshed records do not establish accurate demand forecasting for Ayu's actual business. The external model was not retrained to claim otherwise during this revision.

Current recipes connect menu quantities to ingredient demand. Recorded batches and their deadlines support expiry-risk and shopping discussions. Scenario controls compare hypothetical actions without automatically buying stock or changing accepted orders. A statistical association, saved forecast or scenario recommendation is not a verified saving. The previous externally evaluated model result is retained with its original data boundary in Chapter 5; it is not presented as measured accuracy on Wrap & Bowl Kuch sales.

## 4.8 REMINDERS, SECURITY AND DEPLOYMENT

### 4.8.1 OWNER HANDOVER REMINDERS

One scheduled authenticated worker serves the relevant queue. Simple mode checks owner delivery/collection events within its documented 24-hour eligibility window, including a bounded recent-overdue interval, and does not send preparation reminders merely because historical preparation fields exist. Settings offers a status refresh, recent run evidence and an explicit owner test-email action. Refreshing status is read-only and never starts an email send.

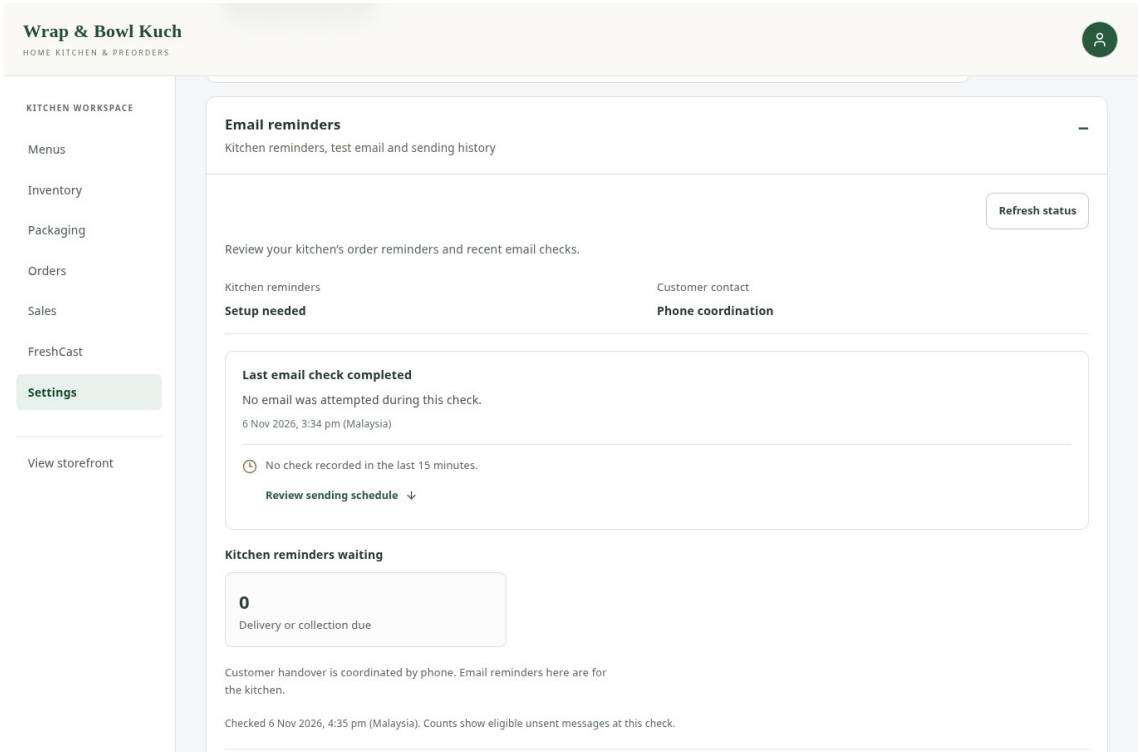



**Figure 4.25: Owner email configuration/status and optional execution details.**

The current customer account collects no email and is not automatically subscribed to email, SMS or WhatsApp. A phone number enables manual contact links where available; it does not authorise an automated WhatsApp service. Earlier explicitly consented email subscriptions are retained as compatibility behaviour. A GitHub

schedule can be delayed, and authenticated manual calls also appear in history. A successful developer test or an old recorded run is not proof that the live scheduler is currently punctual.

### 4.8.2 SECURITY AND RECOVERY

Server permissions, session/CSRF validation, private evidence routes, bounded uploads, request identifiers and transactional stock checks address concrete application risks. They do not constitute an independent penetration test. Customer passkey challenges are short-lived, one-use and bound to the correct session/action/origin. Sensitive changes require recent verification, and staff/customer roles remain separate. Changes of browser/account cancel or discard stale private reads.

Backups of the database, source and external model are different recovery objects. A source ZIP cannot restore orders and stock. Receiving bank details, private proof images, database credentials and passkey private keys must not be published in a report package. Build-time migration checks detect missing schema but do not apply production migrations automatically. The final report identifies source publication and local verification separately from an observed Ready deployment.

## 4.9 INTERFACE AND ACCESSIBILITY DECISIONS

The supplied food-app references informed compact product cards, visible prices, familiar quantity controls and a clear basket/checkout path. They were not copied as a competing marketplace. Nielsen’s visibility, consistency, error-prevention and recovery principles guided progressively disclosed settings, shared form spacing, clear selected states and field-local errors (Nielsen, 1994). Native labels, keyboard-operable controls, visible focus and explicit dialog return paths support the relevant accessibility concerns described in WCAG 2.2 (W3C, 2023); full WCAG conformance has not been certified.

The green identity remains a deliberate client/fresh-food theme. Neutral surfaces and restrained status colour prevent every informational message from looking like an alarm. Ordinary navigation uses active, hover and focus states instead of inherited prose underlines, while meaningful inline links can remain underlined. Phone controls stack rather than overflowing; touch actions remain large enough to operate. A four-card public storage summary loads only when opened, reducing unnecessary work for ordinary menu browsing.

The implementation also corrects lifecycle issues that a static screenshot cannot reveal. Dialogs share scroll-lock ownership so closing an older dialog cannot leave the

page frozen or unlock a remaining modal. Keyboard focus returns to a usable opener, and mobile navigation closes and restores focus when resizing to desktop. These decisions were checked through actual browser interactions as well as component rendering.

## 4.10 CONCLUSION

The current implementation supports the client's simpler morning preorder business while preserving traceable accepted terms, dated stock and financial distinctions. Its main contribution is the integrated workflow: a branded single-vendor menu, secure low-memory account access, date/area-aware ordering, immediate ingredient/packaging commitment and practical reporting. The following chapter evaluates those implemented behaviours and identifies the user evidence still needed before claiming operational or behavioural improvement.

# CHAPTER 5

# TESTING AND RESULTS

## 5.1 INTRODUCTION

Testing evaluated whether the implemented rules behave as specified and whether the visible workflows can be completed in developer-controlled scenarios. It did not treat a successful screenshot, generated order or test receipt as evidence of real business performance. The current verification identifies application source 6edde5e1c24c9c9329ec4e756ebee19703686d39 and a clean source tree. This differs from the earlier report's ab1a976 baseline and its test totals.

The supplied briefing emphasises suitable and verifiable testing, exception handling, security concerns and evaluation against the objectives. It does not explicitly mandate automated testing. The senior example provides useful behavioural test tables and objective-linked survey discussion; its participant counts and results cannot be reused for this project. This chapter therefore presents readable cases and outcomes first, supported by reproducible automated checks and developer browser evidence. Formal customer survey and client acceptance results have not yet been collected.

## 5.2 TESTING STRATEGY AND ENVIRONMENT

### 5.2.1 METHODS

Functional tests covered normal customer and owner tasks. Boundary and equivalence checks covered dates, times, phone formats, quantities, prices and file inputs. Integration tests checked that quotes, proof scope, order records, ingredients, packaging and payment state remain consistent. Failure-injection and replay checks tested rollback and idempotency. Role/session tests exercised unauthorised reads/writes and stale-account responses. Browser walkthroughs checked navigation, focus, layout and real control interactions beyond static rendering.

Automated tests are supporting evidence because stock deductions, scope changes and repeated submissions are difficult to verify thoroughly by repeatedly clicking a form. They do not replace manual exploratory testing or the client's judgement of workflow suitability. A test named after an old feature can remain in the compatibility regression suite without making that feature part of the current client interface.

### 5.2.2 ISOLATION AND EVIDENCE

The fresh report verification ran on 6 November 2026 using an in-memory SQLite database, in-memory email and disabled dotenv loading. It did not connect to production Supabase, send a real message, perform a bank transfer or book a rider. The backend used the project virtual environment and the frontend used the installed dependency versions/lockfile. Command logs, exit codes, elapsed times and source revision are retained in the evidence package.

**Table 5.1: Verification environment and interpretation**

| Evidence | Environment | Interpretation |
|---|---|---|
| Full Django application suite | Isolated in-memory SQLite; locmem email | Current functional/security regression results; explicit skips retained |
| Release/runtime harness | Local reduced runtime trees and mocked configuration/HTTP where relevant | Entrypoint, deployment-check and scheduler-runner behaviour, not a live Ready deployment |
| Frontend utilities and rendered components | Node/Vite test environment | Input/state/arithmetic/render contracts, not participant usability |
| Build and lint | Current source and installed frontend dependencies | Build succeeds and configured lint rules pass; bundle-size warning remains visible |
| Retained browser walkthroughs | Local built app, synthetic fixtures, desktop/phone Chromium; virtual authenticators | Observed developer interactions, not a physical-device user study |
| Client meetings | Project-author-supplied feedback and implemented revisions | Requirements refinement, not a fabricated interview or signed acceptance |
| Planned survey and acceptance form | Intended students/customer-role users and the client owner | Evaluation instruments prepared; responses pending |

SQLite cannot establish PostgreSQL row-locking behaviour. Seventy-four PostgreSQL-dependent checks were skipped in this run, and two optional real-model-bundle checks were also skipped. Earlier repository evidence records isolated PostgreSQL exercises of relevant stock/account/reminder behaviours, but those dated runs are not relabelled as executions of every test at the final UI revision. A new production-like concurrency run remains a release-evidence task if the underlying transaction logic is changed.

## 5.3 DECISION TABLES AND BOUNDARIES

### 5.3.1 ACCOUNT AND CHECKOUT ACCESS

**Table 5.2: Account eligibility and authentication decisions**

| Input/state | Expected behaviour | Reason |
|---|---|---|
| Public visitor browsing | Menu and business pages available | Browsing should not force registration |
| Browsing identity with no registered customer account | Cannot place or upload private proof | Name/phone alone must not authorise an order |
| Valid customer passkey | Own account and history available | Device response proves possession of the registered credential |
| Wrong session/origin/action, expired or replayed challenge | Reject | A challenge must be bound and single-use |
| Full Malaysian local or +60 phone | Normalise accepted contact | Formatting convenience without weakening authentication |
| Prefix 011 alone | Reject with useful field guidance | Prefix is not a complete phone number |
| Customer attempts staff action | Reject | Role boundary remains server-side |

### 5.3.2 DATE, FULFILMENT AND PAYMENT POLICY

**Table 5.3: Current handover and payment decisions**

| Condition | Expected result |
|---|---|
| Permitted future date within 90 days and valid menu schedule | Continue to stock/pricing check |
| Past time, date beyond horizon or unoffered weekday | Reject; clear stale quote and explain the relevant field |
| Default delivery at 07:29 or 11:01 | Outside 07:30–11:00 bounds |
| Default delivery at 07:30 or 11:00 | Within the time bounds, subject to all other rules |
| Unknown restricted delivery area | Request a configured area or kitchen contact; do not assume a free fee |
| Delivery QR/bank evidence | Private proof required; saved order remains unpaid until verification |
| Delivery cash | No advance receipt image; payment remains unpaid until cash is recorded |
| Collection cash | Reject; collection requires enabled QR evidence |

| Condition | Expected result |
|---|---|
| QR/proof selection changed after a quote | Recheck scope and amount; do not automatically pay again |

The minute boundaries concern the configured client default; they do not promise that the rider can physically arrive at an exact minute. Edited owner bounds are tested as settings rather than hardcoded assumptions. Calendar shortcuts show dates permitted by all basket menus, but do not bypass stock validation.

### 5.3.3 STOCK AND HISTORY INTEGRITY

**Table 5.4: Stock allocation and history decisions**

| Condition | Expected result |
|---|---|
| Sufficient received, unheld, date-eligible ingredients and packaging | Allocate all requirements once with the accepted order |
| Any ingredient or packaging shortage | No partial placement or stock deduction; preserve basket |
| Expired, held or future-received batch | Do not use it to cover the requirement |
| Repeated accepted request identifier | Return the same order rather than deduct again |
| Current menu renamed or recipe/mapping edited | Existing accepted name, price and allocation remain unchanged |
| Stale inventory edit after a consumption | Reject/review version conflict instead of restoring consumed stock |
| Eligible unused cancellation | Return recorded allocations once according to server policy |
| Timer reaches planned time | Do not mark food received or payment paid |

### 5.3.4 FRESHNESS AND BUDGET RULES

**Table 5.5: Date evidence and recommendation boundaries**

| Case | Expected behaviour |
|---|---|
| Manufacture plus documented life | Use manufacture date as the base anchor |
| Received plus documented storage life | Use receipt date; no compulsory manufacture date or preset |
| Earlier supplier/opened/thawed deadline | Earliest applicable deadline wins |
| Missing/contradictory evidence | Field error or needs-review state, never invented food safety |

| Case | Expected behaviour |
|---|---|
| Unknown average versus true zero | Display Not available versus 0%; do not merge them |
| Budget left blank | No price limit; within/above comparison disabled |
| Exact whole-order total equals budget | Within budget |
| Total is greater than budget | Above budget |
| Above/within alternatives requested | Partition eligible complete baskets before selecting the bounded result set |

## 5.4 EXECUTED FUNCTIONAL AND SECURITY CASES

The following cases map representative requirements to executed test methods from the fresh backend log. PASS means that the automated fixture assertions passed for the stated behaviour. It does not mean that a participant performed the task or that a real bank accepted a payment. The compact source locator is included so the result can be traced back to its test, with full logs in the companion evidence.

**Table 5.6: Executed functional cases TC-01–TC-05**

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-01<br>Obj. 1<br>Account preserves basket | Register a browsing customer through a valid ceremony. | Same user/basket retained; no customer website password/email required. | PASS<br>Automated fixture<br>See Appendix G |
| TC-02<br>Obj. 1<br>Passkey challenge boundaries | Replay a challenge and supply wrong origin or missing required verification. | Reject the invalid ceremony without granting account access. | PASS<br>Automated fixture<br>See Appendix G |
| TC-03<br>Obj. 1<br>Phone formats | Submit 011 alone, then a complete valid local number. | Incomplete prefix rejected; full contact accepted and normalised. | PASS<br>Automated fixture<br>See Appendix G |
| TC-04<br>Obj. 1<br>Account gate | Attempt placement/proof from a browsing identity with contact details. | Registered-account requirement enforced. | PASS<br>Automated fixture<br>See Appendix G |

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-05<br>Obj. 1<br>Delivery fees | Choose an area and submit an outdated client total. | Server fee authoritative; stale total cannot create an order. | PASS<br>Automated fixture<br>See Appendix G |

**Table 5.7: Executed functional cases TC-06–TC-10**

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-06<br>Obj. 1<br>Morning time boundaries | Check the minute boundaries around configured morning hours. | Quote/slot behaviour agrees at allowed and disallowed edges. | PASS<br>Automated fixture<br>See Appendix G |
| TC-07<br>Obj. 1<br>Cash delivery | Place eligible delivery cash preorder. | Saved contact/address/fee and stock allocations agree; payment remains unpaid. | PASS<br>Automated fixture<br>See Appendix G |
| TC-08<br>Obj. 1<br>QR versus collection policy | Configure bank details without a pickup QR. | Bank-only delivery option does not bypass collection QR requirement. | PASS<br>Automated fixture<br>See Appendix G |
| TC-09<br>Obj. 1<br>Bulk promotion consistency | Use qualifying quantity and compare quote, proof and accepted total. | Food discount, delivery and saved total match. | PASS<br>Automated fixture<br>See Appendix G |
| TC-10<br>Obj. 1,2<br>All eight menus allocated once | Place and replay the complete eight-menu client basket. | Ingredient and packaging deductions occur exactly once. | PASS<br>Automated fixture<br>See Appendix G |

**Table 5.8: Executed functional cases TC-11–TC-15**

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-11<br>Obj. 1,2<br>Shortage rollback | Submit a basket with insufficient assigned packaging. | No partial order/deduction; basket preserved. | PASS<br>Automated fixture<br>See Appendix G |

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-12<br>Obj. 1,2<br>No cooking-plan requirement | Place a valid simple-mode order despite legacy cooking constraints. | No preparation plan required; current date/stock policy still applies. | PASS<br>Automated fixture<br>See Appendix G |
| TC-13<br>Obj. 2<br>Date anchors differ | Use manufacture-based and receipt-based evidence. | Each documented duration uses its correct anchor. | PASS<br>Automated fixture<br>See Appendix G |
| TC-14<br>Obj. 2<br>Earlier cap wins | Combine calculated life, supplier cap and opened limit. | Earliest applicable recorded deadline is used. | PASS<br>Automated fixture<br>See Appendix G |
| TC-15<br>Obj. 2<br>Uncertain handling hold | Record uncertain handling alongside a future deadline. | Batch remains held/excluded. | PASS<br>Automated fixture<br>See Appendix G |

**Table 5.9: Executed functional cases TC-16–TC-20**

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-16<br>Obj. 2<br>Optional guidance | Create documented unlabelled receipt-based batch without a preset. | Valid manual evidence accepted without forcing manufacture/preset fields. | PASS<br>Automated fixture<br>See Appendix G |
| TC-17<br>Obj. 2<br>Read-only target totals | Check several menus sharing ingredients and packaging. | Requirements/shortfalls aggregate correctly; no stock/log/order writes. | PASS<br>Automated fixture<br>See Appendix G |
| TC-18<br>Obj. 2,4<br>Accepted packaging survives edits | Change a current menu mapping after acceptance. | Earlier allocated packaging quantities/costs remain unchanged. | PASS<br>Automated fixture<br>See Appendix G |
| TC-19<br>Obj. 4<br>Operational report scope | Create open, received, cancelled and generated records; request a daily summary. | Only documented operational scope contributes to totals. | PASS<br>Automated fixture<br>See Appendix G |

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-20<br>Obj. 4<br>Sales excludes test payments | Request sales with generated/test-payment records. | They do not become real-business sales. | PASS<br>Automated fixture<br>See Appendix G |

**Table 5.10: Executed functional cases TC-21–TC-25**

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-21<br>Obj. 4<br>Private formula-safe CSV | Export scoped report containing spreadsheet-sensitive text. | Complete authorised export; formula-like values escaped. | PASS<br>Automated fixture<br>See Appendix G |
| TC-22<br>Obj. 4<br>Collection slip contact | Preview a collection order with an older address on the account. | Slip uses accepted customer phone and collection terms. | PASS<br>Automated fixture<br>See Appendix G |
| TC-23<br>Obj. 4<br>Unknown costs | Leave an assigned packaging cost unknown. | Estimate unavailable rather than invented profit. | PASS<br>Automated fixture<br>See Appendix G |
| TC-24<br>Obj. 5<br>Whole-order budget boundary | Search at an exact budget and invalid input boundaries. | Exact total is within; invalid inputs rejected. | PASS<br>Automated fixture<br>See Appendix G |
| TC-25<br>Obj. 5<br>Budget partition before limit | Request within/above alternatives with more than three candidates. | Eligible budget ranges processed before bounded result selection. | PASS<br>Automated fixture<br>See Appendix G |

**Table 5.11: Executed functional cases TC-26–TC-26**

| Case / objective | Test action | Expected and observed behaviour | Recorded result |
|---|---|---|---|
| TC-26<br>Obj. 5<br>Complete basket pricing | Search with discount/fee/collection conditions. | Budget filtering uses the relevant whole-basket total. | PASS<br>Automated fixture<br>See Appendix G |

## 5.5 CURRENT REGRESSION RESULTS

The full backend suite discovered and ran 1,046 tests: 970 passed and 76 were skipped, with no failures or errors reported. The 76 skips are not counted as passes. They comprise 74 PostgreSQL-dependent checks and two checks requiring a provisioned evaluated/real model bundle. The source runner also completed 37 local release/runtime tests with no failures. These groups are reported separately because adding them into a single headline pass count would conceal their different scopes.

**Table 5.12: Fresh verification summary at source 6edde5e**

| Check group | Recorded result | Scope/limitation |
|---|---|---|
| Django application regression | 1,046 run; 970 passed; 76 skipped | SQLite; legacy and current features; no live records |
| Release/runtime harness | 37 passed | Local runtime/configuration checks; provider calls mocked where required |
| Frontend core utilities/rendering | Command passed | Includes existing compatibility behaviour |
| UI/UX and checkout suites | Commands passed | Forms, state, dates, payment selection and presentation |
| Recommendation/account/ stock/report suites | Commands passed | Relevant current client workflows and bounded responses |
| Predictive frontend checks | Command passed | Data-source/decision presentation, not a new forecast evaluation |
| Repository frontend lint | Passed, exit code 0 | Configured static checks, not proof of absence of all defects |
| Production frontend build | Passed, exit code 0 | Existing large-main-chunk warning remains |

The frontend command groups are not converted into an invented total number of end-to-end user tests. Some scripts contain several assertions, rendered states or nested test groups. The report therefore retains command-level outcomes and exact logs rather than conflating assertions with people, journeys or independent experiments.

## 5.6 BROWSER FINDINGS, DEFECTS AND RETESTING

### 5.6.1 TASK AND LAYOUT CHECKS

Retained developer browser evidence exercises native customer registration using a virtual authenticator, staff/customer dialog returns, checkout continuity, menu

alternatives, whole-basket pricing, image-proof selection and private owner views. Phone-width checks cover compact cards, stacked form controls and reachable footer/basket actions. These observations supplement the backend tests by checking what a user can actually focus, click, read and recover from.

A late stock-check refinement measured input/button bottom-edge differences of 0 pixels at 1366, 1024 and 768 pixel widths. At 390 and 320 pixels the controls stack with equal widths and no detected page overflow. The same browser check confirmed that 5 units across eight offered Monday menus display 40 units, editing to 3 clears the stale result, and rerunning displays 24. Only GET requests were used for that check.

The public storage disclosure made zero summary requests before opening, then reused the dated snapshot on reopening. Explicit refresh and retry were tested alongside malformed responses, real zero, unknown averages, empty records and overlapping held/expired counts. This improves both comprehension and unnecessary-request behaviour, but it is not a laboratory measure of physical freshness.

### 5.6.2 DEFECTS AND CORRECTIONS

**Table 5.13: Representative defects, corrections and observed retests**

| Defect | Correction | Observed result |
|---|---|---|
| Overlapping dialogs could restore overflow:hidden after both closed | Shared lock ownership with final-owner cleanup | Both closing orders, repeated switches, staff login and wheel scrolling passed |
| Stock button sat lower than its adjacent inputs | Scoped grid and reset label/control margins | Equal bottom edges at desktop/tablet widths; stacked phone layout |
| Per-menu stock target could be mistaken for order demand | Example, total-unit calculation and included-menu list | 5 × 8 = 40 and changed 3 × 8 = 24 confirmed |
| Manual invalid weekday left an old availability result visible | Invalidate selected quote and explain offered-day intersection | No invalid quote submission; shortcut selection recovers |
| Receipt chooser cancellation could lose a valid upload | Preserve existing proof when no replacement file selected | Existing evidence remains attached; invalid replacement still reports an error |
| Account navigation obscured the return from staff to customer | Explicit back/return controls and preserved unsent draft | Customer/staff switch and checkout return path passed |
| Stale or malformed stock summary could imply healthy/zero records | Validate response and show retry without invented counts | Unknown, empty, zero and bad responses remain distinct |

These are actual developer findings and retests, not evidence that intended users have accepted the final interface. The student survey should still ask whether date selection, quantity, area fee and payment-proof status are understood without assistance.

### 5.6.3 SECURITY AND PERFORMANCE INTERPRETATION

Permission tests reject unauthorised staff operations and cross-customer evidence access. Passkey tests cover challenge reuse, wrong session/origin/action, required user verification, credential signatures and account isolation. Upload tests check real image decoding and private reads. Placement tests cover rollback, scope changes and replay. CSV checks prevent user-supplied text from becoming a spreadsheet formula.

Performance-oriented checks verify bounded searches/exports and selected query behaviour, while lazy disclosures reduce unnecessary reads. The build warning identifies a remaining opportunity to split the main JavaScript bundle. No controlled concurrent-load benchmark, production latency distribution or independent penetration test was conducted for this report. Those limitations must not be replaced by unsupported claims that the application is infinitely scalable or fully secure.

## 5.7 FORECAST EVALUATION AND DATA LIMITS

The application retains the external Genpact/CatBoost material described in the previous report. Its archived holdout evidence recorded 28.23494% WAPE compared with 34.80762% for its stated baseline, a difference of approximately 6.57 percentage points. These are the earlier external-data results, not a model rerun or a measurement on Wrap & Bowl Kuch orders. The current revision did not retrain the model, modify the evaluated artifact or turn generated order history into real client sales.

The local operational records used for development are fictional scenario data, as confirmed by the project author. They permit deterministic demonstrations of charts, stock allocation and report inclusion rules, but cannot establish genuine forecast accuracy, profit, food-waste reduction or customer conversion. Two optional real-model-bundle tests were skipped in the fresh SQLite run. The source/evidence distinction is visible so a reader can separate implemented decision support from externally evaluated and still-unmeasured claims.

## 5.8 EVALUATION AGAINST THE UNCHANGED OBJECTIVES

**Table 5.14: Objective evidence and remaining validation**

| Objective | Implemented and developer-verified evidence | Remaining empirical evidence |
|---|---|---|
| 1: Flexible date-based preordering and scheduling | Offered dates/cutoffs, 90-day horizon, minute-based delivery/collection, area fees and accepted-term changes | Client task completion and practicality of the morning handover workflow |
| 2: Shelf-life/storage-duration monitoring | Evidence-based deadlines, handling caps, holds, date-derived status and eligible FEFO allocation | Owner comprehension and accuracy of actual stock/evidence entry; no food-safety certification |
| 3: Personalised single-vendor storefront | Client identity/story, own categories/menu/images and no competitor listings | Customer recognition and clarity ratings; owner branding acceptance |
| 4: Sales analytics supporting decisions | Daily operations versus paid/received sales, print slips, CSV, known-cost contribution and what-if pricing | Owner's interpretation and usefulness for actual decisions with real cost records |
| 5: Recommendations reducing fatigue/improving conversion | Date-free suggestions and optional complete-basket finder with mix/same, budget and eligibility rules | Comparative decision time, completion/abandonment and non-leading customer feedback |

The implementation and test evidence supports functional achievement, while the behavioural effects in Objectives 3–5 require participant or operational observations. In particular, successful recommendation generation is not itself proof that decision fatigue decreased or paid purchase conversion increased.

## 5.9 CLIENT AND CUSTOMER EVALUATION PLAN

### 5.9.1 WHAT HAS AND HAS NOT BEEN COLLECTED

The project author met the client, discussed requested changes and implemented follow-ups. No completed interview transcript, structured answer sheet, signed acceptance record or student survey dataset has been provided for this revision. The report records the supplied feedback as informal requirements input and does not retrospectively create quotes or scores. Formal evaluation results remain pending.

The briefing's annotated example links surveys to objectives, but it does not establish that this student must use the senior's exact sample size, location or participant mix. The supervisor should confirm the intended recruitment and required institutional forms. Appendices E and F supply a client task/acceptance record and a student

customer-usability questionnaire. A full interview is not necessary merely to make the earlier meeting appear formal; a brief present-tense follow-up can document which current tasks the client actually tried and which changes remain.

### 5.9.2 ROLE-APPROPRIATE TASKS

The owner should inspect menu/recipe and packaging settings, receive a batch, interpret a deadline/hold, check a stock target, review an order/payment, confirm actual receipt, open a daily summary/print preview and interpret a sales/cost report. The test should use a non-production fixture or supervised reversible scenario without a real bank transfer. The evaluator records success without help, success with help, failure or not attempted, together with the client's actual comments.

Students should evaluate the customer journey: identify the kitchen, find menus for a permitted date, understand a selling unit, use optional recommendations, compare complete-basket totals, review the basket, understand passkey sign-in, select area/time and recognise that uploaded evidence is not confirmed payment. Students should not rate whether ingredient formulas, staff reports or real kitchen handling are accurate unless they were specifically qualified and shown those functions.

### 5.9.3 RECOMMENDATION COMPARISON

A practical comparison can ask each consenting participant to select the same total number of items under a similar budget and offered-date constraint. One task uses ordinary browsing; a matched task uses recommendations. Alternate task order across participants to reduce learning bias, use comparable menus/constraints and record whether help was given. Start and stop timing at defined points, record task completion/abandonment, then ask balanced clarity and effort questions.

The analysis should report valid response counts, missing items and recruitment context. For time data, report medians and spread as well as any justified comparisons; for rating questions, show the full distribution rather than only favourable percentages. A test-checkout completion rate is not real paid conversion. If no actual transaction study is conducted, report purchase intention and task completion with those names, not increased revenue.

### 5.9.4 CONSENT, PRIVACY AND ACCEPTANCE

Participation should be voluntary, with a short purpose/withdrawal notice. Use anonymous respondent codes and avoid collecting payment receipts, private passkey

material or unnecessary phone/email details. If the passkey step is tested on a participant device, explain its account effect and cleanup/backup options; do not imply that the website reads fingerprints. An unsupported device is an important access finding and should be recorded, not silently removed as a failed respondent.

The client form distinguishes an accepted task, an issue needing revision and a function not tried. A signature or date is left for the real client and is not generated by the developer. After collection, replace the pending results subsection with the actual sample, method, distributions, observed issues, changes and retest evidence. Keep the original anonymised response export separately from the report.

## 5.10 DISCUSSION IN RELATION TO THE LITERATURE

The proposal literature identifies a need for accessible digital support around ordering, food quality records and small-business decisions. The implemented single-kitchen storefront addresses the integration problem by connecting the customer order to accepted recipes, dated batches and packaging rather than treating each as a disconnected list. The date/stock/payment test results show that these links can be implemented consistently in the tested scenarios; they do not establish that the broader commercial problem has disappeared.

The client revision also demonstrates a contextual trade-off. Detailed cooking-task scheduling added expressive power but imposed data-entry and planning work the client did not want. Retaining dated fulfilment while removing that operational requirement better reflects this case study. Similar trade-offs appear in registration: contact-only forms feel simple, but private history needs authentication. Passkeys reduce memorised credentials while introducing device/support and recovery considerations that require evaluation.

Consistent with the literature's quality concerns, documented shelf life makes stock decisions more traceable but cannot substitute for proper handling or temperature measurement. Recommendation filters reduce the size of a choice set, but actual decision effort and conversion remain empirical questions. The proposed contribution is this implemented combination for the client context, not a claim of a universally novel algorithm or proven business uplift.

## 5.11 CONCLUSION

Current developer verification supports the specified account, pricing, date, stock, packaging, payment-evidence, receipt and reporting behaviours. The full SQLite run,

release checks and selected frontend suites passed with explicit skips and deployment/model limitations. Client feedback has informed the system, but formal acceptance and student customer evaluation have not yet been completed. The final chapter therefore distinguishes the implemented achievement from the further evidence and operational work required.

# CHAPTER 6

# CONCLUSION AND SUGGESTIONS

## 6.1 INTRODUCTION

This final chapter evaluates the implemented project against the five unchanged objectives and discusses its strengths, limitations and next steps. Chapters 1–3 establish the rationale and updated design; Chapter 4 explains the client-oriented implementation; Chapter 5 supplies developer verification and a clear plan for the evaluation still to be collected. Chapter 6 is the final main chapter in the supplied report/example structure, followed by references and appendices.

The final case-study application is Wrap & Bowl Kuch, a home-based business in Kuching. Its customer-facing workflow supports morning preorders, area-based delivery charges, optional collection, a name/phone passkey account and private QR/bank-transfer evidence or delivery cash. Its staff workflow connects menus to ingredient and packaging records, order/payment review, actual receipt, daily print summaries, sales/cost information and configurable branding. These changes were made to fit the client's requests rather than adding technology solely for presentation.

## 6.2 ACHIEVEMENT OF THE ORIGINAL OBJECTIVES

### 6.2.1 FLEXIBLE DATE-BASED FOOD PREORDERING AND SCHEDULING

The application supports permitted future dates, menu weekdays and specific-date windows, cutoffs, configurable delivery/collection hours and accepted handover information. Upcoming common dates help customers choose around the contents of the whole basket. Delivery fees are calculated for an owner-managed area list and saved with the order. Server rechecks and request replay protection coordinate these choices with stock at acceptance.

Scheduling in the current client workflow refers to handover scheduling. The owner is not required to model cooking resources or confirm preparation plans. The client explicitly requested removal of that operational workflow. Retained historical scheduling modules therefore do not define how the final case-study checkout should be explained or assessed. Functional support for the objective is demonstrated by the implementation and tests; punctual physical delivery remains dependent on actual preparation, rider availability and customer coordination.

### 6.2.2 INGREDIENT SHELF LIFE AND STORAGE DURATION

The batch model records received/manufacture/printed dates, documented duration and storage evidence, plus earlier opening/thawing limits and handling holds. Effective deadlines and date-derived indicators follow deterministic rules. Unlabelled ingredients can use documented receipt-based guidance without a compulsory preset. Current eligible stock and FEFO allocation connect this monitoring to orders rather than displaying an unrelated status badge.

This achieves the monitoring feature at the level of recorded evidence and calendar rules. The system does not measure physical spoilage, validate a supplier's date or certify safe handling. Using a general reference chart responsibly still requires matching the ingredient, condition and purpose of the guidance. Owner evaluation should check whether those distinctions are understood in the batch form.

### 6.2.3 PERSONALISED SINGLE-VENDOR STOREFRONT

The storefront presents one kitchen's name, story, founder introduction, menu/category structure, food images and contact/fulfilment terms. It applies the actual client identity and advertised menu without placing competitor listings beside it. The owner can change those details through section-based settings and can replace illustrative images with authorised food photographs.

The implemented controls support the branding objective. Stronger recognition, trust or loyalty has not been measured through the fictional development records. The planned student questionnaire therefore asks whether users can identify the kitchen, understand its offer and find the relevant information, while the owner acceptance form checks control over the published presentation.

### 6.2.4 SALES ANALYTICS FOR BUSINESS DECISIONS

Daily summaries and print slips support packing/handover, while period sales applies its documented paid-and-received scope. Food, discounts and delivery income are separated. Ingredient and packaging cost estimates, order contribution, expenses, wastage and read-only price scenarios support practical questions about quantities and prices. Missing costs remain unavailable, preventing a blank purchase cost from becoming an invented profit.

These features provide data and calculations for decisions. They are not an audited accounting system and do not prove net profitability. Genuine supplier prices, operating

expenses, actual paid/received records and client interpretation are needed before judging real financial benefit. The external forecast benchmark must also remain separate from any claimed accuracy on the client's own sales.

### 6.2.5 MENU RECOMMENDATIONS, DECISION FATIGUE AND CONVERSION

The system implements complementary date-free discovery and an optional complete-basket finder. The finder supports mixed or repeated-menu alternatives, a total item count, an optional whole-order budget, category preference and current eligibility rules. It explains selected signals and asks before replacing an existing basket. Within/above budget partitioning and whole-basket totals address the lecturer's requested comparison while maintaining purchase consistency.

The functionality needed to investigate this objective is implemented and developer-verified. Reduced decision fatigue and increased conversion are not demonstrated by passing algorithm tests. The planned comparison of ordinary browsing and recommendations can measure selection time, task completion and perceived effort. Real paid conversion or revenue claims require a separately designed operational study.

## 6.3 DEVELOPMENT CHALLENGES AND RESPONSES

### 6.3.1 FITTING THE CLIENT WITHOUT CHANGING THE OBJECTIVES

The earlier system's more detailed preparation model did not match the client's desired workload. The project responded by simplifying the active flow while preserving the recipe and history data needed for stock and analysis. This shows why feature quantity is not the same as client fit. The report records the revised scope explicitly instead of describing removed requirements as still mandatory or silently changing the objectives to match the code.

### 6.3.2 LOW-MEMORY ACCOUNTS AND PRIVATE HISTORY

The client wanted customers to enter familiar contact details rather than memorise a username/password. Allowing a name and phone alone to authenticate would expose private history to anyone who knew those details. Passkeys offer a practical compromise: a short profile plus device-mediated verification. The remaining challenge is compatible access, synchronisation and recovery; an unsupported browser or lost credential is a user-experience issue requiring honest guidance.

### 6.3.3 STOCK CONSISTENCY ACROSS RECIPES AND PACKAGING

Food quantities and packaging pieces cannot be handled as independent afterthoughts. A complete basket can share ingredients across several menus. Placement therefore captures accepted requirements, rechecks current records and allocates the union transactionally. Shortage, rollback, cancellation and replay cases were important because an extra click or failed later write must not consume quantities twice. The stock-target checker adds a clear planning aid without pretending that a calculated need is a received purchase.

### 6.3.4 HISTORICAL TERMS AND CHANGING CATALOGUES

Order lines intentionally keep accepted names and prices as well as their menu foreign key. Recipe and packaging snapshots extend the same principle. The duplicated historical description is useful rather than a normalisation mistake: it prevents a menu rename or new recipe from rewriting what the customer bought. It also permits an old order to remain interpretable when a current product link is absent. This accuracy is the primary justification; avoiding a join for displayed snapshot fields is a secondary benefit, not a universal performance guarantee.

### 6.3.5 MANUAL PAYMENT AND PHYSICAL HANDOVER

The website can store a proof image and a payment event, but it cannot know that a transfer settled without reliable bank evidence. Likewise, an elapsed time cannot prove that a customer received food. The project keeps manual verification explicit and provides contact/print information around external banking and courier tasks. This is proportionate to the client scope, while still requiring the owner to carry out those checks.

### 6.3.6 INTERFACE CONSISTENCY AND RECOVERY

Repeated feedback identified dense explanations, mismatched controls, duplicated forms and unclear return paths. The refinements used progressive disclosure, common form sizing, clearer dates and separate state/error messages. The scrollbar defect further showed that a page can look correct in a screenshot but fail after a dialog transition. Browser lifecycle testing was therefore a useful complement to conventional behavioural case tables.

## 6.4 STRENGTHS OF THE IMPLEMENTED SYSTEM

The project brings a single-vendor storefront, dated food ordering, evidence-based inventory records, packaging allocations and owner reports into one coherent data flow. Customers can review prices and quantities before checkout, use optional suggestions and see accepted terms afterwards. Owners can maintain the current catalogue while preserving earlier purchase facts. Explicit scope and transactional rules reduce the chance of a misleading success state or an untraceable stock change.

Another strength is the separation of concepts that are easily confused in a small business: current menu versus accepted order, stock target versus physical stock, payment proof versus verified payment, received food versus paid order, operational order value versus sales, and contribution versus net profit. These distinctions make the data more interpretable and support later evaluation and maintenance.

## 6.5 LIMITATIONS

**Table 6.1: Limitations and their practical implications**

| Limitation | Practical implication |
|---|---|
| Formal owner acceptance and student evaluation pending | Client fit and behavioural improvement are not yet empirically established |
| Recipes, packaging and operational history include authorised estimates/fictional records | Actual production quantities, costs and sales must replace assumptions before business reliance |
| Calendar dates and owner evidence do not measure physical safety | Correct storage, handling and professional food-safety practices remain necessary |
| No live bank or courier API | Payment verification and rider coordination remain manual; no automatic delivery-event claim |
| Passkeys depend on device/browser support and retained credentials | Backup/synchronisation and an accessible support policy are important |
| No automatic customer email/SMS/WhatsApp subscription | A name/phone account does not guarantee advance reminders reach the customer |
| Scheduler/SMTP and production deployment not observed as continuously reliable | Local tests and status history cannot prove live punctuality or inbox receipt |
| Current full run used SQLite with PostgreSQL/model skips | Database locking and optional model execution need their own environment/evidence |
| Selected browser checks are not full accessibility/load/penetration certification | Further independent and representative testing is required |
| Main JavaScript bundle retains a size warning | Further code splitting and measured performance work are possible |

The client's real ingredient freshness, delivery punctuality, customer no-shows and profitability cannot be solved completely through interface changes. The website can reduce ambiguity, record commitments and surface outstanding work. It cannot make an owner cook on time, ensure a rider finds the address, or guarantee that the customer answers a call. Safe in-person handover and agreed instructions remain operational responsibilities.

## 6.6 PRIORITISED FUTURE WORK

### 6.6.1 COMPLETE AND ANALYSE THE USER EVALUATION

The immediate priority is to use the prepared instruments with the client and consenting customer-role participants. Record the actual tasks tried, assistance, unsupported-device cases, comments and remaining defects. Compare matched browsing/recommendation tasks where feasible. Report the sample and missing responses transparently, and revise the interface based on observed difficulty rather than favourable scores alone. Insert genuine results into Chapter 5 only after collection.

### 6.6.2 VALIDATE BUSINESS INPUTS

Review all estimated recipes, selling-unit yields, packaging mappings, purchase costs, advertised areas/fees and collection arrangements with the owner. Receive real stock with truthful evidence and distinguish opening balances from later purchase receipts. Keep physical counts and the system reconciled. The implementation should not refill quantities automatically just to make an out-of-stock message disappear.

### 6.6.3 IMPROVE RELIABILITY AND ACCESSIBILITY EVIDENCE

A production-like PostgreSQL concurrency run should accompany transaction changes. Add measured performance targets, real supported-device tests, keyboard/screen-reader review and monitored backup restoration. Keep a single intended reminder schedule active and inspect actual worker results without assuming that configuration alone proves operation. Improve JavaScript splitting based on measured loading behaviour rather than speculative removal of repository folders.

### 6.6.4 CONSIDER INTEGRATIONS ONLY WHEN JUSTIFIED

A bank/acquirer or courier integration could reduce manual work if approved documentation, access, cost and trustworthy events become available. Such work requires verified webhooks, replay protection, failure recovery and an agreed operating

process. A WhatsApp service also needs the appropriate provider setup, consent and message policy. These are optional future projects, not missing claims to be disguised as implemented features in the current FYP.

### 6.6.5 EVALUATE FORECASTS WITH REPRESENTATIVE DATA

Collect enough genuine, correctly dated client demand and stock outcomes to establish an appropriate baseline before assessing local forecasts. Separate training, validation and holdout periods and retain provenance. A useful forecast should improve a relevant decision under realistic constraints, not simply reproduce a generated pattern. Recommendation outcomes similarly need their own behavioural evaluation rather than being inferred from the external Genpact model.

## 6.7 HANDOVER AND MAINTENANCE

The owner guide in Appendix H summarises routine setup, stock receiving, payment review, receipt confirmation, reports and troubleshooting. Staff should keep credentials and private receiving/payment records out of public source and screenshots. Customer account guidance should explain backup passkeys and the distinction between contact data and authentication. Private database recovery copies must be maintained separately from the source download and model archive.

Future releases should identify the tested source revision, required migrations and observed deployment status. The frozen earlier report/integration baseline must not be confused with current main. A missing migration should be applied and verified against the correct database rather than bypassing the release check. After release, verify actual customer/staff flows on the stable production origin because passkey origins, provider configuration and deployment settings can differ from a local test environment.

## 6.8 CONCLUSION

The project implements the original objectives through a client-oriented single-vendor preorder application with dated ingredient monitoring, editable branding, practical sales/cost reporting and explainable menu recommendations. The final changes simplify the client's daily work without abandoning account security, stock integrity or accepted history. Current developer tests provide verifiable evidence of the specified behaviours, with explicit environment and evaluation limits.

The next evidence required is formal client acceptance and customer-role survey/task evaluation. Completing that work will allow the report to assess usability

and decision support with actual responses rather than assumption. The resulting project is therefore an implemented and developer-verified system with a defined path to client validation, not a claim that all delivery, food-safety or commercial risks have been eliminated.

# REFERENCES


Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). Manifesto for Agile Software Development. Agilemanifesto.org

Bernama. (2022, April 7). Malaysia's e-commerce income can breach RM1 trillion again this year - MDEC. https://www.bernama.com/en/business/news.php?id=2069812

Department of Statistics Malaysia. (2025, November 26). Malaysia digital economy (#276): Contribution of ICT to GDP in 2024. https://www.dosm.gov.my/uploads/release-content/file_20260327194302.pdf

Django Software Foundation. (2026). Meet Django. Djangoproject.com

Django Software Foundation. (n.d.). Database transactions. Django documentation. https://docs.djangoproject.com/en/5.2/topics/db/transactions/

Django Software Foundation. (n.d.). User authentication in Django. Django documentation. https://docs.djangoproject.com/en/5.2/topics/auth/

Eu, E. Z. R., & Sameeha, M. J. (2021). Consumers' perceptions of healthy food availability in online food delivery applications and its association with food choices among public university students in Malaysia. Frontiers in Nutrition, 8, 674427. https://doi.org/10.3389/fnut.2021.674427

Food poisoning turns fatal. (2024, June 12). The Star. https://www.thestar.com.my/news/nation/2024/06/12/food-poisoning-turns-fatal

foodpanda. (2012). Backgrounder: foodpanda APAC. https://www.foodpanda.com/wp-content/uploads/2021/01/EN-Backgrounder-foodpanda-APAC.pdf

FoodSafety.gov. (n.d.). Cold food storage chart. https://www.foodsafety.gov/food-safety-charts/cold-food-storage-charts

Gogliettino, M., Balestrieri, M., Ambrosio, R. L., Anastasio, A., Smaldone, G., Proroga, Y. T. R., Moretta, R., Rea, I., De Stefano, L., Agrillo, B., & Palmieri, G. (2020). Extending the shelf- life of meat and dairy products via PET-modified packaging activated with the antimicrobial peptide MTP1. Frontiers in microbiology, 10, 2963. https://www.frontiersin.org/journals/microbiology/articles/10.3389/fmicb.2019.02963/full

Google. (2026). Firebase documentation. Google Cloud. https://firebase.google.com/docs

Gou, Z., Gamage, W., Lau, S. S. Y., & Lau, S. S. Y. (2018). An investigation of thermal comfort and adaptive behaviors in naturally ventilated residential buildings In

tropical climates: A pilot study. Buildings, 8(1), 1-17. https://doi.org/10.3390/buildings8010005

Grab. (2018, May 28). Grab launches GrabFood beta and unveils vision to be an everyday app. Grab Press Centre. https://www.grab.com/sg/press/business/grab-launches-grabfood-beta-and-unveils-vision-to-be-an-everyday-app/

Hansen, C. (2026). Preorder and freshness: Wrap & Bowl Kuch [Computer software and verification evidence, revision 6edde5e]. GitHub. https://github.com/CHplus2/preorder-freshness/tree/6edde5e

i-CATS University College. (2026). IT Project 2 briefing on 17 September [Annotated briefing supplied by the project author].

Islam, A. K. M. Z., & Ferworn, A. (2020). A comparison between agile and traditional software development methodologies. Global Journal of Computer Science and Technology, 20(2), 7-42. https://globaljournals.org/GJCST_Volume20/2-A-Comparison-between-Agile.pdf

Kandoli, L. (2022). Effect of food shelf life on food quality. Journal of Advances in Humanities and Social Sciences, 8(1), 38-42. http://www.tafpublications.com/gip_content/paper/jahss-8.1.5.pdf

Leau, Y. B., Loo, W. K., Tham, W. Y., & Tan, S. F. (2012). Software development life cycle AGILE vs traditional approaches. In International Conference on Information and Network Technology (Vol. 37, No. 1, pp. 162-167). https://ku-fpg.github.io/files/agile-traditional.pdf

Lok, Y. H., Liou, Z. Y., Teoh, K. B., Lim, J. T., Lim, K. E., Lim, P. E., & Kee, D. M. H. (2024). Influencing Factors on Customer Behavioral Intentions to Use a Food Delivery App: A Study of GrabFood in Malaysia. Advances in Global Economics and Business Journal, 5(1), 1-14. https://www.agebj.org/index.php/agebj/article/view/92

Nielsen, J. (1994, April 24). 10 usability heuristics for user interface design. Nielsen Norman Group. https://www.nngroup.com/articles/ten-usability-heuristics/

OpenJS Foundation. (2026). Express documentation (Version 4.21). https://expressjs.com/

OWASP Foundation. (n.d.). File upload cheat sheet. OWASP Cheat Sheet Series. https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html

Pallianysamy, S., Abdul-Mutalib, N. A., Abidin, U. F. U. Z., Rashid, N. K. M., & Ahmad, N. H. (2025). Home-based Foods in Malaysia: A Food Safety Perspective. Pertanika Journal of Tropical Agricultural Science, 48(3). http://pertanika2.upm.edu.my/resources/files/Pertanika%20PAPERS/JTAS%20Vol.%204 8%20(3)%20May.%202025/01%20JTAS-3151-2024.pdf

PayPal. (n.d.). Orders API v2. PayPal Developer. https://developer.paypal.com/docs/api/orders/v2/

Peres, F. A. P., Bondarczuk, B. A., Gomes, L. D. C., Jardim, L. D. C., Corrêa, R. G. D. F., & Baierle, I. C. (2025). Advances in food quality management driven by Industry 4.0: A systematic review-based framework. Foods, 14(14), 2429. https://www.mdpi.com/2304-8158/14/14/2429

PostgreSQL Global Development Group. (2026). PostgreSQL 18.4 documentation. https://www.postgresql.org/docs/

Prokhorenkova, L., Gusev, G., Vorobev, A., Dorogush, A. V., & Gulin, A. (2018). CatBoost: Unbiased boosting with categorical features. Advances in Neural Information Processing Systems, 31. https://arxiv.org/abs/1706.09516

Royce, W. W. (1970). Managing the development of large software systems. Proceedings of IEEE WESCON, 26(August), 1-9.

Sixty-five Sungai Petani primary school pupils suffer food poisoning. (2024, May 21). The Star. https://www.thestar.com.my/news/nation/2024/05/21/65-sungai-petani-primary-school-pupils-suffer-food-poisoning

Sommerville, I. (2016). Software Engineering (10th ed.). Pearson Education Limited.

Statista. (2024). Online food delivery-Malaysia. https://www.statista.com/outlook/emo/online-food-delivery/malaysia

Susanto, D. A., Suef, M., Karningsih, P. D., & Prasetya, B. (2026). Quality Management System Model for Food SMEs. Sustainability, 18(2), 890. https://www.mdpi.com/2071-1050/18/2/890

Tan, H., & Kim, V. W. E. (2021). Examining the factors that influence consumer satisfaction with online food delivery in Klang Valley, Malaysia. Journal of Management Theory and Practice, 2(2). https://doi.org/10.37231/jmtp.2021.2.2.115

Teo, S. C., Liew, T. W., & Lim, H. Y. (2024). Factors influencing consumers' continuance purchase intention of local food via online food delivery services. Cogent Business & Management, 11(1). https://doi.org/10.1080/23311975.2024.2316919

Tsang, Y. P., Choy, K. L., Wu, C. H., Ho, G. T. S., & Lam, H. Y. (2019, August). An internet of things (IoT)-based shelf life management system in perishable food e-commerce businesses. In 2019 Portland International Conference on Management of Engineering and Technology (PICMET) (pp. 1-8). IEEE. https://ieeexplore.ieee.org/abstract/document/8893684

Violi, A., De Maio, A., & Fattoruso, G. (2024). Inventory management and delivery of perishable products with stochastic demands and risks consideration. Procedia

Computer Science, 232, 2941-2949. https://www.sciencedirect.com/science/article/pii/S1877050924002886

World Wide Web Consortium (W3C). (2021, April 8). Web Authentication: An API for accessing public key credentials, Level 2. https://www.w3.org/TR/webauthn-2/

World Wide Web Consortium (W3C). (2023, October 5). Web Content Accessibility Guidelines (WCAG) 2.2. https://www.w3.org/TR/WCAG22/

Wrap & Bowl Kuch. (n.d.). Menu poster and founder introduction [Client materials supplied by the project author].

Zhou, X., Tang, J., Jacobs, T. L., & Saguy, I. S. (2025). Transforming food supply chains through digital tracking and monitoring technologies. Trends in Food Science & Technology, 163, 105142. https://doi.org/10.1016/j.tifs.2025.105142

## APPENDIX A — SUPERVISOR CONSULTATION RECORD

Attach the actual completed institutional consultation log. The supplied briefing indicates eight consultations. This report revision does not generate attendance, meeting dates, supervisor approval or signatures.

## APPENDIX B — ORIGINALITY / SIMILARITY RECORD

Attach the genuine institution-approved similarity/originality record and required submission forms after the author has reviewed the report. The briefing specifies a plagiarism check and inclusion of the summary page. No similarity percentage or approval is claimed here.

# APPENDIX C — DECLARATION OF INTEGRITY

The source declaration is retained below for author review. Use the current official institutional form if required; the report does not supply a signature or certify originality on the author's behalf.

I hereby declare that the submitted work together with this form is my original work. The following are further details of this declaration which I approve.

All references and sources relevant to this submission (from prints or the Internet) are inspirations to my written idea and creativity as well as my understanding of used references and sources.

I did not copy any other people's work, be it from amongst the people I know in real life or from random sources on the Internet. Similarities are expected for common facts but I understand that my worded works should not be too similar to other people's works at an acceptable amount set by the college.

If I fail to satisfy my pledge, I am responsible for any errors or negligence that might occur in this submission and I accept the final decision made by the lecturer and the university college. I understand the risk of non-compliance to this pledge may result in a reduction of marks or failure.

# APPENDIX D — CURRENT APPLICATION DATA DICTIONARY

Generated from the current Django model metadata without reading production rows. The application contains 35 models, in addition to Django's built-in user/session/auth tables. Diagrams show selected attributes; this dictionary lists all model columns. A retained field is not evidence that a legacy payment or Planner feature is enabled in the client interface. Nullable means the database accepts no value; application validation can impose further conditions.

### MenuPhoto — myapp_menuphoto

| Column | Django type | Storage / relationship properties |
|---|---|---|
| digest | CharField | PK; Not null; Max 64 |
| content | BinaryField | Not null |
| width | PositiveIntegerField | Not null |
| height | PositiveIntegerField | Not null |
| created_at | DateTimeField | Not null |

### Category — myapp_category

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| name | CharField | Unique; Not null; Max 100 |
| created_at | DateTimeField | Not null |

### Product — myapp_product

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| selling_details | JSONField | Not null |
| delivery_weekdays | JSONField | Not null |
| delivery_dates | JSONField | Not null |
| order_cutoff_time | TimeField | Nullable |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| order_cutoff_days_before | PositiveSmallIntegerField | Not null |
| selling_status | CharField | Not null; Max 10 |
| packaging_cost | DecimalField | Nullable; 2 decimal places |
| lead_hours | PositiveIntegerField | Not null |
| preparation_minutes | PositiveIntegerField | Not null |
| preparation_tasks | JSONField | Not null |
| max_preparation_days | PositiveIntegerField | Not null |
| batch_size | PositiveIntegerField | Not null |
| additional_batch_minutes | PositiveIntegerField | Not null |
| packing_minutes_per_portion | PositiveIntegerField | Not null |
| max_early_minutes | PositiveIntegerField | Not null |
| daily_capacity | PositiveIntegerField | Not null |
| social_url | CharField | Not null; Max 200 |
| name | CharField | Not null; Max 200 |
| description | TextField | Not null |
| ai_summary | TextField | Not null |
| category_id | ForeignKey | FK → Category; SET_NULL; Nullable |
| price | DecimalField | Not null; 2 decimal places |
| image_url | CharField | Not null; Max 200 |
| created_at | DateTimeField | Not null |
| updated_at | DateTimeField | Not null |

### RawMaterial — myapp_rawmaterial

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| estimated_unit_cost | DecimalField | Nullable; 6 decimal places |
| storage_guidance | JSONField | Not null |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| name | CharField | Unique; Not null; Max 100 |
| unit | CharField | Not null; Max 10 |
| created_at | DateTimeField | Not null |

### ProductIngredient — myapp_productingredient

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| product_id | ForeignKey | FK → Product; CASCADE; Not null |
| raw_material_id | ForeignKey | FK → RawMaterial; CASCADE; Not null |
| quantity_required | DecimalField | Not null; 3 decimal places |

### InventoryItem — myapp_inventoryitem

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| supplier | CharField | Not null; Max 160 |
| source_type | CharField | Not null; Max 20 |
| label_date_type | CharField | Not null; Max 20 |
| original_expiry_date | DateField | Nullable |
| opened_date | DateField | Nullable |
| after_open_days | PositiveIntegerField | Nullable |
| thawed_date | DateField | Nullable |
| after_thaw_days | PositiveIntegerField | Nullable |
| handling_history | CharField | Not null; Max 20 |
| handling_note | CharField | Not null; Max 500 |
| unit_cost | DecimalField | Nullable; 6 decimal places |
| expiry_basis | CharField | Not null; Max 20 |
| manufactured_date | DateField | Nullable |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| shelf_life_days | PositiveIntegerField | Nullable |
| storage_type | CharField | Not null; Max 20 |
| guidance_note | CharField | Not null; Max 300 |
| quarantined | BooleanField | Not null |
| raw_material_id | ForeignKey | FK → RawMaterial; PROTECT; Not null |
| quantity | DecimalField | Not null; 3 decimal places |
| batch_code | CharField | Not null; Max 100 |
| storage_location | CharField | Not null; Max 200 |
| received_date | DateField | Not null |
| expiry_date | DateField | Not null |
| created_at | DateTimeField | Not null |
| updated_at | DateTimeField | Not null |

**Wallet — myapp_wallet**

Retained compatibility/supporting entity; not part of the client's required checkout flow.

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| user_id | OneToOneField | FK → User; CASCADE; Unique; Not null |
| balance | DecimalField | Not null; 10 decimal places |
| wallet_address | CharField | Unique; Not null; Max 255 |
| is_external | BooleanField | Not null |
| created_at | DateTimeField | Not null |
| updated_at | DateTimeField | Not null |

**WalletTransaction — myapp_wallettransaction**

Retained compatibility/supporting entity; not part of the client's required checkout flow.

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| wallet_id | ForeignKey | FK → Wallet; CASCADE; Not null |
| amount | DecimalField | Not null; 10 decimal places |
| type | CharField | Not null; Max 20 |
| reference | CharField | Not null; Max 255 |
| created_at | DateTimeField | Not null |

### Address — myapp_address

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| recipient_name | CharField | Not null; Max 150 |
| user_id | ForeignKey | FK → User; CASCADE; Not null |
| line1 | CharField | Not null; Max 255 |
| line2 | CharField | Not null; Max 255 |
| city | CharField | Not null; Max 100 |
| state | CharField | Not null; Max 100 |
| postal_code | CharField | Not null; Max 20 |
| country | CharField | Not null; Max 100 |
| phone | CharField | Not null; Max 20 |
| is_default | BooleanField | Not null |

### CartItem — myapp_cartitem

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| user_id | ForeignKey | FK → User; CASCADE; Not null |
| product_id | ForeignKey | FK → Product; CASCADE; Not null |
| quantity | PositiveIntegerField | Not null |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| added_at | DateTimeField | Not null |

### Order — myapp_order

Includes accepted fulfilment/pricing fields and retained historical preparation fields. Current simple-mode presentation is Received / Not received with cancellation; the stored status choices remain broader.

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| ordering_mode | CharField | Not null; Max 12 |
| stock_returned_at | DateTimeField | Nullable |
| fulfilment_snapshot | JSONField | Not null |
| checkout_request_id | UUIDField | Unique; Nullable; Max 32 |
| payment_method | CharField | Not null; Max 20 |
| payment_instructions | JSONField | Not null |
| preparation_end_at | DateTimeField | Nullable |
| preparation_plan | JSONField | Not null |
| delivery_address | JSONField | Not null |
| delivery_at | DateTimeField | Nullable |
| preparation_at | DateTimeField | Nullable |
| delivery_method | CharField | Not null; Max 20 |
| inventory_deducted | BooleanField | Not null |
| user_id | ForeignKey | FK → User; PROTECT; Not null |
| address_id | ForeignKey | FK → Address; SET_NULL; Nullable |
| total_amount | DecimalField | Not null; 2 decimal places |
| discount_amount | DecimalField | Not null; 2 decimal places |
| shipping_fee | DecimalField | Not null; 2 decimal places |
| status | CharField | Not null; Max 20 |
| payment_status | CharField | Not null; Max 20 |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| created_at | DateTimeField | Not null |
| updated_at | DateTimeField | Not null |

### OrderItem — myapp_orderitem

product_id links to the current menu where available; product_name and unit_price preserve the accepted historical name/price.

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| recipe_source | CharField | Not null; Max 30 |
| preparation_snapshot | JSONField | Not null |
| packaging_unit_cost | DecimalField | Nullable; 6 decimal places |
| order_id | ForeignKey | FK → Order; CASCADE; Not null |
| product_id | ForeignKey | FK → Product; SET_NULL; Nullable |
| product_name | CharField | Not null; Max 200 |
| unit_price | DecimalField | Not null; 2 decimal places |
| quantity | PositiveIntegerField | Not null |
| subtotal | DecimalField | Not null; 2 decimal places |

### InventoryLog — myapp_inventorylog

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| inventory_item_id | ForeignKey | FK → InventoryItem; SET_NULL; Nullable |
| change | DecimalField | Not null; 3 decimal places |
| reason | CharField | Not null; Max 255 |
| reference | CharField | Not null; Max 255 |
| admin_id | ForeignKey | FK → User; SET_NULL; Nullable |
| created_at | DateTimeField | Not null |

### Storefront — myapp_storefront

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| ordering_mode | CharField | Not null; Max 12 |
| delivery_start_time | TimeField | Not null |
| delivery_end_time | TimeField | Not null |
| pickup_start_time | TimeField | Not null |
| pickup_end_time | TimeField | Not null |
| delivery_area_fees | JSONField | Not null |
| delivery_enabled | BooleanField | Not null |
| pickup_enabled | BooleanField | Not null |
| pickup_location | CharField | Not null; Max 250 |
| pickup_instructions | TextField | Not null; Max 1000 |
| delivery_open_hour | PositiveSmallIntegerField | Not null |
| delivery_close_hour | PositiveSmallIntegerField | Not null |
| pickup_open_hour | PositiveSmallIntegerField | Not null |
| pickup_close_hour | PositiveSmallIntegerField | Not null |
| delivery_areas | JSONField | Not null |
| delivery_fee_myr | DecimalField | Not null; 2 decimal places |
| free_delivery_minimum_myr | DecimalField | Nullable; 2 decimal places |
| manual_payment_enabled | BooleanField | Not null |
| bank_transfer_instructions | TextField | Not null; Max 1000 |
| duitnow_qr_url | CharField | Not null; Max 200 |
| kitchen_open_hour | PositiveSmallIntegerField | Not null |
| kitchen_close_hour | PositiveSmallIntegerField | Not null |
| founder_name | CharField | Not null; Max 100 |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| founder_intro | CharField | Not null; Max 250 |
| founding_story | TextField | Not null |
| food_philosophy | TextField | Not null |
| signature_food | CharField | Not null; Max 200 |
| service_area | CharField | Not null; Max 200 |
| whatsapp_number | CharField | Not null; Max 20 |
| founder_image_url | CharField | Not null; Max 200 |
| name | CharField | Not null; Max 100 |
| tagline | CharField | Not null; Max 200 |
| story | TextField | Not null |
| social_url | CharField | Not null; Max 200 |
| contact_email | CharField | Not null; Max 254 |
| delivery_buffer_minutes | PositiveIntegerField | Not null |
| bulk_minimum | PositiveIntegerField | Not null |
| bulk_discount_percent | PositiveIntegerField | Not null |

### Review — myapp_review

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| product_id | ForeignKey | FK → Product; CASCADE; Not null |
| user_id | ForeignKey | FK → User; CASCADE; Not null |
| rating | PositiveSmallIntegerField | Not null |
| comment | CharField | Not null; Max 1000 |
| created_at | DateTimeField | Not null |

### OrderReminder — myapp_orderreminder

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| order_id | ForeignKey | FK → Order; CASCADE; Not null |
| event | CharField | Not null; Max 20 |
| sent_at | DateTimeField | Not null |

### KitchenBlock — myapp_kitchenblock

Retained compatibility/supporting entity; not part of the client's required checkout flow.

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| start_at | DateTimeField | Not null |
| end_at | DateTimeField | Not null |
| reason | CharField | Not null; Max 160 |

### OperatingExpense — myapp_operatingexpense

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| request_id | UUIDField | Unique; Not null; Max 32 |
| voided | BooleanField | Not null |
| date | DateField | Not null |
| category | CharField | Not null; Max 20 |
| amount | DecimalField | Not null; 2 decimal places |
| note | CharField | Not null; Max 300 |
| created_at | DateTimeField | Not null |

### WasteRecord — myapp_wasterecord

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| request_id | UUIDField | Unique; Not null; Max 32 |
| inventory_item_id | ForeignKey | FK → InventoryItem; PROTECT; Not null |
| quantity | DecimalField | Not null; 3 decimal places |
| estimated_cost | DecimalField | Nullable; 2 decimal places |
| reason | CharField | Not null; Max 300 |
| created_at | DateTimeField | Not null |

### OrderIngredient — myapp_orderingredient

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| order_item_id | ForeignKey | FK → OrderItem; CASCADE; Not null |
| raw_material_id | ForeignKey | FK → RawMaterial; PROTECT; Not null |
| material_name | CharField | Not null; Max 100 |
| unit | CharField | Not null; Max 10 |
| quantity_per_portion | DecimalField | Not null; 3 decimal places |

### IngredientConsumption — myapp_ingredientconsumption

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| allocation_stage | CharField | Not null; Max 12 |
| returned_at | DateTimeField | Nullable |
| order_item_id | ForeignKey | FK → OrderItem; PROTECT; Not null |
| inventory_item_id | ForeignKey | FK → InventoryItem; PROTECT; Not null |
| material_name | CharField | Not null; Max 100 |
| batch_code | CharField | Not null; Max 100 |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| unit | CharField | Not null; Max 10 |
| quantity | DecimalField | Not null; 3 decimal places |
| unit_cost | DecimalField | Nullable; 6 decimal places |
| recorded_expiry | DateField | Not null |
| created_at | DateTimeField | Not null |

### PaymentEvent — myapp_paymentevent

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| order_id | ForeignKey | FK → Order; PROTECT; Not null |
| request_id | UUIDField | Unique; Not null; Max 32 |
| kind | CharField | Not null; Max 10 |
| outcome | CharField | Not null; Max 12 |
| amount | DecimalField | Not null; 2 decimal places |
| method | CharField | Not null; Max 20 |
| reference | CharField | Not null; Max 200 |
| note | CharField | Not null; Max 500 |
| source | CharField | Not null; Max 30 |
| resolves_id | OneToOneField | FK → PaymentEvent; PROTECT; Unique; Nullable |
| actor_id | ForeignKey | FK → User; SET_NULL; Nullable |
| created_at | DateTimeField | Not null |

### AuthAttempt — myapp_authattempt

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| key | CharField | Not null; Max 64 |
| created_at | DateTimeField | Not null |

### OrderAmendment — myapp_orderamendment

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| order_id | ForeignKey | FK → Order; PROTECT; Not null |
| actor_id | ForeignKey | FK → User; SET_NULL; Nullable |
| before | JSONField | Not null |
| after | JSONField | Not null |
| created_at | DateTimeField | Not null |

### RecommendationSession — myapp_recommendationsession

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | UUIDField | PK; Not null; Max 32 |
| owner_key | CharField | Not null; Max 64 |
| variant | CharField | Not null; Max 20 |
| context | JSONField | Not null |
| products | JSONField | Not null |
| created_at | DateTimeField | Not null |

### RecommendationEvent — myapp_recommendationevent

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| session_id | ForeignKey | FK → RecommendationSession; CASCADE; Not null |
| event | CharField | Not null; Max 20 |
| product_id | ForeignKey | FK → Product; SET_NULL; Nullable |
| order_id | OneToOneField | FK → Order; SET_NULL; Unique; Nullable |
| request_id | UUIDField | Unique; Not null; Max 32 |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| created_at | DateTimeField | Not null |

### PackagingItem — myapp_packagingitem

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| name | CharField | Unique; Not null; Max 100 |
| quantity | PositiveIntegerField | Not null |
| reorder_level | PositiveIntegerField | Not null |
| unit_cost | DecimalField | Nullable; 6 decimal places |
| is_active | BooleanField | Not null |
| created_at | DateTimeField | Not null |
| updated_at | DateTimeField | Not null |

### ProductPackaging — myapp_productpackaging

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| product_id | ForeignKey | FK → Product; CASCADE; Not null |
| packaging_item_id | ForeignKey | FK → PackagingItem; PROTECT; Not null |
| quantity_per_item | PositiveIntegerField | Not null |

### OrderPackaging — myapp_orderpackaging

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| order_item_id | ForeignKey | FK → OrderItem; PROTECT; Not null |
| packaging_item_id | ForeignKey | FK → PackagingItem; PROTECT; Not null |
| packaging_name | CharField | Not null; Max 100 |
| quantity | PositiveIntegerField | Not null |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| unit_cost | DecimalField | Nullable; 6 decimal places |
| allocated_at | DateTimeField | Nullable |
| released_at | DateTimeField | Nullable |

### PackagingMovement — myapp_packagingmovement

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| packaging_item_id | ForeignKey | FK → PackagingItem; PROTECT; Not null |
| order_packaging_id | ForeignKey | FK → OrderPackaging; PROTECT; Nullable |
| request_id | UUIDField | Unique; Nullable; Max 32 |
| kind | CharField | Not null; Max 10 |
| change | IntegerField | Not null |
| balance_after | PositiveIntegerField | Not null |
| reason | CharField | Not null; Max 200 |
| actor_id | ForeignKey | FK → User; SET_NULL; Nullable |
| created_at | DateTimeField | Not null |

### GuestCustomer — myapp_guestcustomer

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| user_id | OneToOneField | FK → User; CASCADE; Unique; Not null |
| created_at | DateTimeField | Not null |

### CheckoutProof — myapp_checkoutproof

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | UUIDField | PK; Not null; Max 32 |
| user_id | ForeignKey | FK → User; CASCADE; Not null |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| order_id | OneToOneField | FK → Order; PROTECT; Unique; Nullable |
| content | BinaryField | Not null |
| digest | CharField | Not null; Max 64 |
| scope_digest | CharField | Not null; Max 64 |
| amount | DecimalField | Not null; 2 decimal places |
| created_at | DateTimeField | Not null |

### CustomerAccount — myapp_customeraccount

Customer authentication support. Public credentials and session-bound challenges are stored; biometric templates, device PINs and private keys are not.

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| user_id | OneToOneField | FK → User; CASCADE; Unique; Not null |
| user_handle | UUIDField | Unique; Not null; Max 32 |
| name | CharField | Not null; Max 150 |
| phone | CharField | Not null; Max 30 |
| created_at | DateTimeField | Not null |
| updated_at | DateTimeField | Not null |

### CustomerPasskey — myapp_customerpasskey

Customer authentication support. Public credentials and session-bound challenges are stored; biometric templates, device PINs and private keys are not.

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | BigAutoField | PK; Not null |
| account_id | ForeignKey | FK → CustomerAccount; CASCADE; Not null |
| credential_id | CharField | Unique; Not null; Max 1400 |
| public_key | BinaryField | Not null |

| Column | Django type | Storage / relationship properties |
|---|---|---|
| sign_count | PositiveBigIntegerField | Not null |
| label | CharField | Not null; Max 80 |
| device_type | CharField | Not null; Max 32 |
| backed_up | BooleanField | Not null |
| created_at | DateTimeField | Not null |
| last_used_at | DateTimeField | Nullable |

**PasskeyCeremony — myapp_passkeyceremony**

Customer authentication support. Public credentials and session-bound challenges are stored; biometric templates, device PINs and private keys are not.

| Column | Django type | Storage / relationship properties |
|---|---|---|
| id | UUIDField | PK; Not null; Max 32 |
| session_digest | CharField | Not null; Max 64 |
| user_id | ForeignKey | FK → User; CASCADE; Nullable |
| action | CharField | Not null; Max 12 |
| challenge | CharField | Not null; Max 64 |
| origin | CharField | Not null; Max 255 |
| rp_id | CharField | Not null; Max 253 |
| context | JSONField | Not null |
| expires_at | DateTimeField | Not null |
| consumed_at | DateTimeField | Nullable |

# APPENDIX E — CLIENT FOLLOW-UP AND ACCEPTANCE FORM

Use this after the client tries the current system. The earlier meetings remain informal requirements feedback; this form must not be backdated or filled with invented answers. Use isolated/reversible test records. No actual bank transfer, customer message or unsafe stock change is required.

| Field | Response |
|---|---|
| Client / business | Ayu Shafinaz / Wrap & Bowl Kuch (confirm with client) |
| Evaluator and date | _____ |
| Website revision / device | _____ |
| Permission to record feedback | [ ] Agreed [ ] Not agreed |

Task outcome codes: I = completed independently; H = completed with help; F = failed; N = not attempted. Record actual comments rather than assuming that a demonstration implies acceptance.

| Task / objective | Task to observe | Outcome / issue |
|---|---|---|
| C1 / O3 | Find and change the kitchen story or a menu detail in a supervised test record. | I / H / F / N<br>Comments:<br>_____ |
| C2 / O1, O3 | Set/check delivery areas, their fees, 07:30–11:00 hours and enabled collection. | I / H / F / N<br>Comments:<br>_____ |
| C3 / O2 | Create a received ingredient batch using the appropriate printed or documented-life mode; explain its effective deadline. | I / H / F / N<br>Comments:<br>_____ |
| C4 / O2 | Explain why a held, expired or future-received batch is excluded from available stock. | I / H / F / N<br>Comments:<br>_____ |
| C5 / O1, O2 | Check 3 units of every offered menu; identify the total units and a shortfall without adding stock. | I / H / F / N<br>Comments:<br>_____ |
| C6 / O1, O4 | Check packaging assignment and the ingredient/packaging movement for a prepared test order. | I / H / F / N<br>Comments:<br>_____ |
| C7 / O1, O4 | Review a synthetic receipt, distinguish unverified proof from paid status, and explain the cash versus collection rule. | I / H / F / N<br>Comments:<br>_____ |

| Task / objective | Task to observe | Outcome / issue |
|---|---|---|
| C8 / O1 | Confirm actual food receipt for eligible test orders; explain why payment is separate. | I / H / F / N<br>Comments:<br>_____ |
| C9 / O1, O4 | Open a daily menu summary and print preview; check the saved name, address and phone. | I / H / F / N<br>Comments:<br>_____ |
| C10 / O4 | Read a sales period and a price scenario; distinguish sales, known-cost contribution and net profit. | I / H / F / N<br>Comments:<br>_____ |
| C11 / O5 | Review one-menu/mixed suggestions and their whole-order budget totals. | I / H / F / N<br>Comments:<br>_____ |
| C12 / O1, O3 | Try customer/staff access and discuss whether passkeys and contact entry are practical for this business. | I / H / F / N<br>Comments:<br>_____ |

1. Which parts fit your actual ordering, stock and handover routine?

Response:

_____

2. Which steps were unclear, unnecessary or difficult? Please refer to a task or screen.

Response:

_____

3. Are the current menu prices/days, estimated recipes, packaging and area fees correct? List any corrections; estimates must not be treated as verified production data.

Response:

_____

4. Is requiring a passkey account acceptable for your customers? What device/access or support concern remains?

Response:

_____

5. What must change before you would use this with genuine customer orders?

Response:

_____

6. Which decisions could the sales/stock information help you make, and what information is still missing?

Response:

_____

Overall decision: [ ] Suitable for the tested use [ ] Suitable after listed changes [ ] Not yet suitable [ ] Not enough tried to decide.

| **Agreed change / priority** | **Responsible person** | **Retest date / actual result** |
|---|---|---|
| _____ | _____ | _____ |
| _____ | _____ | _____ |
| _____ | _____ | _____ |

Client name / optional signature: _____ Actual date: _____

A signature records the client's actual assessment of the tasks tried; it is not academic marking, a food-safety certificate or approval of untested features.

# APPENDIX F — STUDENT CUSTOMER-USABILITY QUESTIONNAIRE

Purpose: evaluate the customer experience of a home-kitchen preorder website. Participation is voluntary. Do not enter real bank details or upload a personal payment receipt. Use the supervised test setup and assigned contact details. Stop before making a real payment or placing an actual customer order. Responses are for this academic project, not for marketing.

Before recruitment, confirm the supervisor's ethics/consent requirements. Suggested eligibility: a consenting adult able to try the customer tasks. No institutional ethics approval, respondent count or result is asserted by this blank instrument.

| **Field** | **Response** |
|---|---|
| Anonymous participant code | _____ |
| Date / device / browser | _____ |
| Consent | [ ] I have read the purpose and agree to take part. [ ] I do not agree (stop). |
| Experience | [ ] Never [ ] Occasionally [ ] Frequently order food online |
| Passkey access | [ ] Supported/tested [ ] Not supported [ ] Not attempted |

Customer tasks: identify the kitchen and selling units; select 3 items for an offered date and a feasible whole-order budget; inspect area fee, basket and checkout guidance; explain account access and payment-proof status. Use ordinary browsing and the date suggestions in matched tasks. Stop before actual payment/order submission.

| **Task** | **Method / order** | **Success / help / time** |
|---|---|---|
| A: select 3 items within RM60, agreed area and offered date | Ordinary browsing; task order assigned by evaluator | I / H / F / N<br>Seconds: _____ |
| B: select 3 items within a comparable budget/date constraint | Suggestions; counterbalance order across participants | I / H / F / N<br>Seconds: _____ |

For S1–S10: 1 = strongly disagree, 2 = disagree, 3 = neither agree nor disagree, 4 = agree, 5 = strongly agree; N/A = not tried or not applicable. Do not rate a function you did not try.

| Item / objective | Statement | Response |
|---|---|---|
| S1 / O3 | I could identify which kitchen I was ordering from. | 1 2 3 4 5 N/A |
| S2 / O3 | The menu price and the meaning of a meal, box or pack were clear. | 1 2 3 4 5 N/A |
| S3 / O1 | I could find an offered date and an allowed delivery/collection time. | 1 2 3 4 5 N/A |
| S4 / O1 | I could see the delivery fee and complete order total before placing an order. | 1 2 3 4 5 N/A |
| S5 / O1 | I understood that uploading a receipt does not mean the kitchen has verified payment. | 1 2 3 4 5 N/A |
| S6 / O1 | The customer account and passkey sign-in instructions were understandable. | 1 2 3 4 5 N/A |
| S7 / O1, O3 | I could return from a dialog or recover from an input error without losing my place. | 1 2 3 4 5 N/A |
| S8 / O5 | The suggestions were relevant to the date, quantity and preferences in my task. | 1 2 3 4 5 N/A |
| S9 / O5 | I understood the difference between Allow a mix and One menu only. | 1 2 3 4 5 N/A |
| S10 / O5 | I could tell whether the suggested complete basket was within or above my budget. | 1 2 3 4 5 N/A |

S11 / O5: Compared with ordinary browsing, the suggestions required: [ ] Much less effort [ ] Less effort [ ] About the same [ ] More effort [ ] Much more effort [ ] Not compared.

S12 / O5: If the real menu, price and date suited you, how likely would you be to preorder? [ ] Very unlikely [ ] Unlikely [ ] Unsure [ ] Likely [ ] Very likely. This is purchase intention, not a paid conversion result.

S13 / O2 comprehension: Do the recorded batch dates alone guarantee that food is physically fresh and safe? [ ] Yes [ ] No [ ] Not sure. Do not provide the correct answer before the participant responds.

S14: Which screen or step was most confusing, and why?

Response:

_____

S15: What would you change to make ordering easier?

Response:

_____

S16: Was there anything you could not test, including passkey/device access?

Response:

_____

Evaluator notes: record missing answers, assistance, invalid/unoffered dates and unsupported devices. Report anonymous valid counts and rating distributions. Students evaluate the customer tasks, not the accuracy of hidden staff calculations. Do not invent responses or present intention as real purchases.

# APPENDIX G — IMPLEMENTATION AND TEST EVIDENCE MAP

The report source is 6edde5e1c24c9c9329ec4e756ebee19703686d39. Fresh checks used a clean source tree. The companion evidence includes verification.json, verbose backend/release logs, frontend command logs, schema metadata and editable diagram sources. These artefacts contain test evidence, not private production rows or bank credentials.

TC-01 — Account preserves basket

myapp.test_passkeys.PasskeyTests.test_registration_upgrades_same_user_basket_orders_and_no_password_email

TC-02 — Passkey challenge boundaries

myapp.test_passkeys.PasskeyTests.test_challenge_single_use_wrong_origin_and_uv_rejected

TC-03 — Phone formats

myapp.test_passkeys.PasskeyTests.test_registration_rejects_prefix_only_then_accepts_full_011_mobile

TC-04 — Account gate

myapp.test_guest_checkout.GuestCheckoutTests.test_browsing_guest_cannot_place_or_upload_even_with_contact_details

TC-05 — Delivery fees

myapp.test_guest_checkout.GuestCheckoutTests.test_area_fee_is_authoritative_and_stale_client_total_does_not_place

TC-06 — Morning time boundaries

myapp.test_guest_checkout.GuestCheckoutTests.test_delivery_window_minute_edges_and_slot_search_are_consistent

TC-07 — Cash delivery

myapp.test_guest_checkout.GuestCheckoutTests.test_cash_delivery_saves_name_phone_full_address_fee_and_stock

TC-08 — QR versus collection policy

myapp.test_guest_checkout.GuestCheckoutTests.test_bank_only_configuration_cannot_replace_pickup_qr_rule

TC-09 — Bulk promotion consistency

myapp.test_guest_checkout.GuestCheckoutTests.test_bulk_promotion_matches_quote_proof_and_placed_order

TC-10 — All eight menus allocated once

myapp.test_stock_readiness.ClientRecipeAllocationTests.test_all_eight_client_recipes_and_packaging_deduct_once_using_current_materials

TC-11 — Shortage rollback

myapp.test_packaging_stock.PackagingCheckoutTests.test_checkout_shortage_rolls_back_order_and_preserves_basket

TC-12 — No cooking-plan requirement

myapp.test_simple_ordering.SimpleOrderingTests.test_default_has_no_preparation_plan_and_ignores_capacity_and_worker_closures

TC-13 — Date anchors differ

myapp.test_expiry_evidence.ExpiryEvidenceTests.test_manufacture_and_storage_have_different_anchors

TC-14 — Earlier cap wins

myapp.test_expiry_evidence.ExpiryEvidenceTests.test_printed_limit_caps_calculated_life_and_opening_can_shorten_it

TC-15 — Uncertain handling hold

myapp.test_expiry_evidence.ExpiryEvidenceTests.test_uncertain_handling_is_held_even_with_a_future_computed_deadline

TC-16 — Optional guidance

myapp.test_expiry_evidence.ExpiryEvidenceTests.test_unlabelled_storage_needs_no_manufacture_or_raw_material_preset

TC-17 — Read-only target totals

myapp.test_stock_readiness.StockReadinessTests.test_combines_shared_recipes_and_packaging_and_uses_checkout_stock_filter

TC-18 — Accepted packaging survives edits

myapp.test_packaging_stock.PackagingStockTests.test_accepted_mapping_and_cost_survive_later_menu_edits

TC-19 — Operational report scope

myapp.test_client_reports.ClientReportTests.test_daily_has_all_active_statuses_but_not_cancelled_or_generated

TC-20 — Sales excludes test payments

myapp.test_client_reports.ClientReportTests.test_generated_and_completed_test_payments_do_not_become_sales

TC-21 — Private formula-safe CSV

myapp.test_client_reports.ClientReportTests.test_csv_is_complete_private_and_formula_safe

TC-22 — Collection slip contact

myapp.test_client_reports.ClientReportTests.test_pickup_slip_has_customer_phone_and_collection_snapshot_not_old_address

TC-23 — Unknown costs

myapp.test_pricing.PricingPreviewTests.test_unpriced_assigned_packaging_blocks_profit_estimate

TC-24 — Whole-order budget boundary

myapp.test_basket_guide.BasketGuideTests.test_exact_whole_order_total_is_within_and_input_ranges_are_validated

TC-25 — Budget partition before limit

myapp.test_basket_guide.BasketGuideTests.test_budget_ranges_are_partitioned_before_three_result_limit

TC-26 — Complete basket pricing

myapp.test_basket_guide.BasketGuideTests.test_current_discount_free_delivery_and_pickup_are_quoted_before_budget_filter

Representative current source locations: backend/myapp/models.py; services/simple_ordering.py; services/packaging.py; services/freshness.py; views/checkout_proofs.py; views/passkeys.py; views/client_reports.py;

views/stock_readiness.py; frontend/src/pages/customer/SimpleCheckoutPage.jsx; frontend/src/components/GuidedMenu.jsx; frontend/src/pages/admin/SimpleOrdersPage.jsx; frontend/src/components/StockReadiness.jsx. Paths identify implementation evidence; they are not instructions to expose the application's private configuration.

# APPENDIX H — SHORT OWNER OPERATING GUIDE

### H.1 Prepare the kitchen profile

Sign in through Kitchen staff sign in. Review the name/story, approved public contact and actual service description. Save each section separately.

### H.2 Confirm delivery and collection

Use short area names with their own fees; keep descriptive sentences in the service description. Check minute-based morning hours. Enable collection explicitly and provide a genuine arrangement. Test a new quote after saving.

### H.3 Configure receiving payment

Use business-approved QR/bank receiving details. Collection needs a QR image. Verify the displayed recipient and test only with authorised evidence; uploading an image does not verify a transfer.

### H.4 Review menus and packaging

Check prices, offered weekdays/dates, selling units and contents. Confirm estimated recipes and packaging pieces with the actual kitchen. Enter known costs or leave them unknown; do not use zero merely to remove a warning.

### H.5 Receive ingredients

Select the correct ingredient/unit and actual received date. Choose printed or documented-life evidence, actual storage and any opened/thawed limits. Review the earliest planning deadline before saving. Hold uncertain handling.

### H.6 Receive packaging

Record actual received whole pieces and reason. Map the pieces used by one selling unit of each menu. Review removals and movement history; do not duplicate an automatic order allocation.

### H.7 Check a stock target

Choose a date and units per offered menu. Read the total units and included-menu list. Replenish only quantities actually obtained; the check and worksheet do not create inventory.

### H.8 Accept and review orders

Use the order queue and date filters. Placement already allocates ingredients and packaging. Review shortages and bank/cash evidence without repeating deductions manually.

### H.9 Record payment and receipt separately

Compare a transfer with actual bank records before confirming money received. Confirm food received only after handover/collection. Review the exact selected set before bulk confirmation. Use corrections/cancellation with reasons rather than rewriting history.

### H.10 Print and interpret reports

Open Daily summary for packing quantities and Preview & print for customer name/address/phone. Use Sales for paid-and-received periods. Contribution needs known costs and is not net profit; a price scenario changes no live price.

### H.11 Review reminders and contact

Refresh status to inspect recent worker activity; it does not send email. An explicit test-email button sends only its stated test. Current phone/passkey customers are not automatically subscribed to email/SMS/WhatsApp. Arrange external rider/customer contact as agreed.

### H.12 Recover and maintain

Keep database backups separate from source/model downloads. Use backup/synchronised passkeys for customer access. Record the website revision when reporting an error. Verify deployment and required migrations instead of bypassing release checks.